DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

RABAB KHAN RONGON

M.Sc. ENGINEERING THESIS

[MIST MONOGRAM
28 mm × 30 mm]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY
DHAKA, BANGLADESH

[MONTH YEAR]

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

RABAB KHAN RONGON (SN. [STUDENT NUMBER])

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science in Engineering (M.Sc. Engineering) in Computer Science and Engineering

[MIST MONOGRAM
28 mm × 30 mm]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY
DHAKA, BANGLADESH

[MONTH YEAR]

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

M.Sc. Engineering Thesis
By
RABAB KHAN RONGON (SN. [STUDENT NUMBER])

Approved as to style and content by the Board of Examination on [DATE OF EXAMINATION]:

_____
[NAME OF SUPERVISOR]
[Designation], [Department]
[Institution], Dhaka

Chairman (Supervisor)
Board of Examination

_____
[NAME OF MEMBER]
[Designation], [Department]
[Institution], Dhaka

Member
Board of Examination

_____
[NAME OF EXTERNAL MEMBER]
[Designation], [Department]
[Institution]

Member (External)
Board of Examination

_____
[NAME OF HEAD OF THE DEPARTMENT]
[Designation], Department of Computer Science and Engineering
MIST, Dhaka

Member (Ex-officio)
Head of the Department

Department of Computer Science and Engineering, MIST, Dhaka

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

DECLARATION

I hereby declare that the study reported in this thesis entitled as above is my own original work and has not been submitted before anywhere for any degree or other purposes. Further I certify that the intellectual content of this thesis is the product of my own work and that all the assistance received in preparing this thesis and sources have been acknowledged and/or cited in the reference Section.

_____

Rabab Khan Rongon

Department of Computer Science and Engineering, MIST, Dhaka

ABSTRACT

Development of a Sensitivity-Aware Reinforcement Learning Framework for Layer-Wise CNN Pruning

Neural-network pruning reduces model size and computation, but deciding how much each layer should lose is difficult, and weight sparsity alone does not establish a deployment benefit. This thesis develops and evaluates a sensitivity-aware reinforcement-learning framework for the layer-wise pruning of convolutional neural networks. The study covers the CIFAR-10 and CIFAR-100 datasets, three small convolutional networks and one laptop CPU.

A Proximal Policy Optimisation (PPO) agent assigns a pruning ratio to each layer or layer group, guided by layer sensitivity. Successive experiments with pre-specified decision criteria added a centred sensitivity prior, archive-based selection of the policies visited during search, an accuracy-constrained reward and an equal post-pruning recovery protocol. A later deployment extension realised the pruning structurally, measured CPU latency and package energy directly, and trained a deployment-oriented structured agent.

The learned allocations substantially outperformed uniform and random allocation, by up to approximately 31.4 percentage points of test accuracy over uniform pruning. The deterministic layer-adaptive magnitude-based pruning (LAMP) method nevertheless remained more accurate before equal recovery. Sensitivity guidance improved search and exploration measures, not final accuracy. The agent found high-ranking policies but rarely ended on them, and archive selection restored their retention on validation ranking. The accuracy-constrained reward reduced destructive over-pruning, although feasibility certified on validation data held on the test set in only about two thirds of runs. Equal five-epoch recovery narrowed the differences between PPO and LAMP to within a pre-registered ±0.5 percentage-point decision margin in every setting. In the deployment analysis, parameter reduction proved a weak predictor of measured latency compared with the reduction in multiply–accumulate operations. Structured pruning reduced directly measured CPU package energy, but with accuracy losses beyond the pre-registered limit. The deployment-oriented structured agent reduced parameter memory by 63.4% and multiply–accumulate operations by 36.0% at a 1.12 percentage-point

validation-accuracy cost. It missed its accuracy criterion, and its direct energy remained unresolved because the measurement setup failed its stability qualification.

The novelty of the thesis lies not in a new pruning criterion but in treating adaptive pruning as a chain of connected problems (allocation, search guidance, policy retention, reward alignment, recovery and hardware validation) and in examining each with its own experiment and evidence. The results show that an improvement at one stage does not guarantee success at the next. Each stage must therefore be validated before a compression result can be called a deployment-efficiency result.

Keywords: neural network pruning; reinforcement learning; proximal policy optimisation; layer-wise pruning; sensitivity-aware pruning; structured pruning; model compression; deployment efficiency

সারসংক্ষেপ

Development of a Sensitivity-Aware Reinforcement Learning Framework for Layer-Wise CNN Pruning

নিউরাল নেটওয়ার্ক প্রুনিং (neural network pruning) মডেলের আকার ও গণনার পরিমাণ কমায়, কিন্তু প্রতিটি স্তর (layer) থেকে কতটুকু অপসারণ করা উচিত তা নির্ধারণ করা কঠিন, এবং কেবল ওজন-বিরলতা (weight sparsity) দিয়ে বাস্তব প্রয়োগে (deployment) সুবিধা প্রমাণিত হয় না। এই থিসিসে কনভোলিউশনাল নিউরাল নেটওয়ার্কের (convolutional neural network) স্তরভিত্তিক প্রুনিংয়ের জন্য একটি সংবেদনশীলতা-সচেতন (sensitivity-aware) রিইনফোর্সমেন্ট লার্নিং (reinforcement learning) কাঠামো উদ্ভাবন ও মূল্যায়ন করা হয়েছে। গবেষণার পরিধিতে রয়েছে CIFAR-10 ও CIFAR-100 ডেটাসেট, তিনটি ছোট কনভোলিউশনাল নেটওয়ার্ক এবং একটি ল্যাপটপ সিপিইউ (CPU)।

একটি প্রক্সিমাল পলিসি অপটিমাইজেশন (proximal policy optimisation, PPO) এজেন্ট স্তর-সংবেদনশীলতার (layer sensitivity) নির্দেশনায় প্রতিটি স্তর বা স্তরগুচ্ছকে একটি প্রুনিং অনুপাত (pruning ratio) নির্ধারণ করে। পূর্বনির্ধারিত সিদ্ধান্ত-মানদণ্ডসহ ধারাবাহিক পরীক্ষার মাধ্যমে এতে যুক্ত করা হয়েছে একটি কেন্দ্রীকৃত সংবেদনশীলতা পূর্বধারণা (centred sensitivity prior), অনুসন্ধানের সময় পরিদর্শিত নীতিগুলোর (policies) আর্কাইভভিত্তিক নির্বাচন (archive-based selection), একটি নির্ভুলতা-সীমাবদ্ধ পুরস্কার (accuracy-constrained reward) এবং প্রুনিং-পরবর্তী সমান পুনরুদ্ধার প্রোটোকল (equal recovery protocol)। পরবর্তী একটি প্রয়োগমুখী সম্প্রসারণে প্রুনিংকে কাঠামোগতভাবে (structured pruning) বাস্তবায়ন করা হয়েছে, সিপিইউ লেটেন্সি (latency) ও প্যাকেজ শক্তি (package energy) সরাসরি পরিমাপ করা হয়েছে, এবং একটি প্রয়োগমুখী কাঠামোগত এজেন্ট (deployment-oriented structured agent) প্রশিক্ষণ দেওয়া হয়েছে।

শেখা বণ্টনগুলো (allocations) সুষম (uniform) ও দৈব (random) বণ্টনের চেয়ে উল্লেখযোগ্যভাবে ভালো ফল দিয়েছে; সুষম প্রুনিংয়ের তুলনায় টেস্ট নির্ভুলতায় (test accuracy) এই ব্যবধান সর্বোচ্চ প্রায় ৩১.৪ শতাংশ-বিন্দু (percentage points)। তবে নির্ধারণমূলক লেয়ার-অ্যাডাপ্টিভ ম্যাগনিচুড-ভিত্তিক প্রুনিং (layer-adaptive magnitude-based pruning, LAMP) পদ্ধতি সমান পুনরুদ্ধারের আগে অধিক নির্ভুল ছিল। সংবেদনশীলতা নির্দেশনা অনুসন্ধান ও অন্বেষণের (exploration) পরিমাপ উন্নত করেছে, চূড়ান্ত নির্ভুলতা নয়। এজেন্ট উচ্চ-র‍্যাংকের নীতি খুঁজে পেলেও কদাচিৎ সেগুলোতে শেষ করেছে, এবং

আর্কাইভ নির্বাচন ভ্যালিডেশন র‍্যাংকিংয়ের (validation ranking) ভিত্তিতে সেগুলোর ধারণ (retention) পুনরুদ্ধার করেছে। নির্ভুলতা-সীমাবদ্ধ পুরস্কার ধ্বংসাত্মক অতিপ্রুনিং (over-pruning) কমিয়েছে, যদিও ভ্যালিডেশন ডেটায় প্রত্যয়িত সম্ভাব্যতা (feasibility) টেস্ট সেটে মাত্র প্রায় দুই-তৃতীয়াংশ রানে বজায় ছিল। পাঁচ ইপকের (epoch) সমান পুনরুদ্ধার প্রতিটি সেটিংয়ে PPO ও LAMP-এর পার্থক্যকে পূর্বনিবন্ধিত (pre-registered) ±০.৫ শতাংশ-বিন্দু সিদ্ধান্ত-সীমার (decision margin) মধ্যে নামিয়ে এনেছে। প্রয়োগ বিশ্লেষণে দেখা গেছে, গুণ-যোগ অপারেশন (multiply–accumulate operations) হ্রাসের তুলনায় প্যারামিটার হ্রাস (parameter reduction) পরিমাপকৃত লেটেন্সির একটি দুর্বল পূর্বাভাসক। কাঠামোগত প্রুনিং সরাসরি পরিমাপকৃত সিপিইউ প্যাকেজ শক্তি কমিয়েছে, তবে পূর্বনিবন্ধিত সীমার চেয়ে বেশি নির্ভুলতা-ক্ষতির বিনিময়ে। প্রয়োগমুখী কাঠামোগত এজেন্ট ১.১২ শতাংশ-বিন্দু ভ্যালিডেশন-নির্ভুলতা (validation accuracy) ব্যয়ে প্যারামিটার মেমরি ৬৩.৪% এবং গুণ-যোগ অপারেশন ৩৬.০% কমিয়েছে। এটি তার নির্ভুলতা-মানদণ্ড পূরণ করেনি, এবং পরিমাপ ব্যবস্থাটি স্থিতিশীলতা যোগ্যতা-যাচাইয়ে (stability qualification) ব্যর্থ হওয়ায় এর সরাসরি শক্তি-ফলাফল অমীমাংসিত রয়ে গেছে।

এই থিসিসের নতুনত্ব কোনো নতুন প্রুনিং মানদণ্ডে নয়; বরং অভিযোজিত প্রুনিংকে (adaptive pruning) পরস্পর-সংযুক্ত সমস্যার একটি শৃঙ্খল হিসেবে বিবেচনা করা (বণ্টন, অনুসন্ধান-নির্দেশনা, নীতি-ধারণ, পুরস্কার-সামঞ্জস্য, পুনরুদ্ধার ও হার্ডওয়্যার যাচাই) এবং প্রতিটিকে তার নিজস্ব পরীক্ষা ও প্রমাণ দিয়ে পরীক্ষা করাই এর মূল অবদান। ফলাফল দেখায় যে এক ধাপের উন্নতি পরবর্তী ধাপের সাফল্য নিশ্চিত করে না। তাই কোনো সংকোচন-ফলাফলকে (compression result) প্রয়োগ-দক্ষতার (deployment efficiency) ফলাফল বলার আগে প্রতিটি ধাপ আলাদাভাবে যাচাই করতে হবে।

ACKNOWLEDGEMENTS

[ACKNOWLEDGEMENTS TO BE WRITTEN BY THE AUTHOR. MIST TP.13 lists, in order: (a) the supervisor [SUPERVISOR] and co-supervisor, if any; (b) peers and colleagues in the laboratory; (c) other academic, technical or support staff of the Department of Computer Science and Engineering; (d) collaborators in other departments, institutions or organisations; (e) any financial assistance (fellowships, grants or contracts); (f) family and friends.]

TABLE OF CONTENTS

Abstract i
Abstract in Bangla (সারসংক্ষেপ) iii
Acknowledgements v
Table of Contents vi
List of Figures xi
List of Tables xii
List of Main Notation and Abbreviations xv
CHAPTER 1: INTRODUCTION 1
1.1 Background 1
1.2 Motivation 3
1.3 Problem Statement 4
1.4 Research Gap 5
1.5 Research Questions 6
1.6 Research Objectives 7
1.7 Research Contributions 8
1.8 Scope and Boundaries 11
1.9 Thesis Organisation 12
1.10 Chapter Summary 12
CHAPTER 2: LITERATURE REVIEW 13
2.1 Neural Network Compression 13
2.2 Unstructured and Structured Pruning 14
2.3 Layer-Wise Adaptive Pruning 16
2.4 Magnitude-Based and Deterministic Pruning 17
2.5 LAMP and Related Layer-Adaptive Methods 18
2.6 Reinforcement Learning for Neural Network Compression 19
2.7 PPO-Based Optimisation 23
2.8 Sensitivity-Aware Pruning 24
2.9 Search and Policy Selection in Pruning 25
2.10 Post-Pruning Recovery and Fine-Tuning 26
2.11 Hardware-Aware Neural Network Compression 27
2.12 Parameter Count vs MACs/FLOPs vs Real Latency 27
2.13 Energy-Efficient and Sustainable Neural Network Inference 28
2.14 Research Gap 30
CHAPTER 3: PROPOSED FRAMEWORK 33
3.1 General 33
3.2 Problem Formulation 37
3.3 Layer-Wise Pruning Representation 38
3.4 Reinforcement Learning Formulation 38
3.5 State Representation 39

3.6 Action Space 40
3.7 Initial Reward Function 41
3.8 Layer Sensitivity Analysis 42
3.9 Sensitivity-Guided Action Prior 43
3.10 Centred Sensitivity Prior 43
3.11 Policy-Retention Problem 44
3.12 Archive-Based Policy Selection 45
3.12.1 Archive (Phase 4) 45
3.12.2 Selection rule (Phase 4) 45
3.12.3 Elite behavioural cloning (Phase 4 only) 46
3.12.4 Constraint-aware selection (Phase 5) 46
3.13 Reward Misalignment and Accuracy-Constrained Reward 47
3.13.1 Why R0 can encourage excessive pruning 47
3.13.2 Accuracy-constrained reward (Phase 5) 47
3.14 Final Core Framework 48
3.15 Equal Recovery and Fine-Tuning Protocol 49
3.16 Deployment-Oriented Structured Extension 50
3.16.1 From masks to physical structure 50
3.16.2 Measurement: three levels of deployment efficiency 51
3.16.3 Two uses of the extension 51
3.17 Overall Algorithm 53
3.18 Computational Workflow and Reproducibility 55
3.19 Chapter Summary 56
CHAPTER 4: EXPERIMENTAL SETUP 57
4.1 Experimental Design Overview 57
4.2 Datasets 57
4.3 Dataset Partitioning and Validation Protocol 57
4.3.1 Training, validation and test 57
4.3.2 Two validation configurations 58
4.3.3 Use of each split by phase 58
4.4 Neural Network Architectures 59
4.5 Dense Reference Models 60
4.6 Pruning Units and Policy Spaces 61
4.6.1 Masked pruning units (core framework) 61
4.6.2 Physically compacted structured units (deployment extension) 62
4.6.3 Policy spaces and enumeration 62
4.7 Baseline Pruning Methods 63
4.7.1 Core-framework baselines 63
4.7.2 Deployment-extension comparators 64
4.7.3 Matching rules 64
4.8 PPO Training Configuration 65

4.9 Experimental Conditions by Phase 66
4.9.1 Phase 3 68
4.9.2 Phase 4 68
4.9.3 Phase 5 69
4.9.4 Phase 6 69
4.9.5 Paper 3 69
4.9.6 Phase 7A 70
4.9.7 Phase 7B 71
4.9.8 Phase 7B energy confirmation 71
4.10 Sensitivity Conditions and Controls 72
4.11 Archive and Selection Protocols 72
4.12 Reward Configurations 73
4.13 Recovery and Fine-Tuning Protocol 73
4.14 Evaluation Metrics 74
4.15 Statistical Analysis and Decision Criteria 75
4.15.1 Paired tests (Phases 3–6) 75
4.15.2 The ±0.5 pp rule of Phase 6 76
4.15.3 Deployment-extension statistics 77
4.15.4 Pre-registered decision rules 78
4.16 Hardware and Software Environment 79
4.17 Structured Pruning and Latency Protocol 79
4.18 Energy Measurement Protocol 80
4.18.1 Backend 80
4.18.2 Phase 7A 81
4.18.3 Phase 7B energy surrogate 81
4.18.4 Phase 7B confirmatory measurement and its stopping rule 81
4.18.5 Energy confirmation study 82
4.19 Reproducibility and Pre-Registration 82
4.20 Chapter Summary 83
CHAPTER 5: RESULTS OF THE CORE FRAMEWORK 84
5.1 General 84
5.2 Dense Reference Performance 84
5.3 Phase 3 — PPO versus Baselines 85
5.4 Phase 3 — Effect of Sensitivity Guidance 89
5.5 Phase 4 — Centred Sensitivity and Exploration 91
5.6 Phase 4 — Policy Retention and Archive Selection 92
5.7 Phase 4 — Reward Misalignment 95
5.8 Phase 5 — Accuracy-Constrained Reward 96
5.9 Phase 5 — Validation-to-Test Transfer 101
5.10 Phase 6 — Equal Fine-Tuning 102
5.11 Answers to RQ1–RQ5 106

5.12 Chapter Summary 107
CHAPTER 6: DEPLOYMENT AND SUSTAINABILITY ANALYSIS 109
6.1 General 109
6.2 From Compression to Deployment Efficiency 109
6.3 Structured Physical Pruning Setup 110
6.4 Exhaustive CPU Latency Analysis 111
6.5 Parameter Reduction versus MAC Reduction versus Latency 112
6.6 Accuracy-Feasible Latency Headroom 113
6.7 Phase 7A — Structured Transfer and Energy Evaluation 116
6.8 Phase 7A — Accuracy–Energy Trade-off 118
6.9 Motivation for Phase 7B 120
6.10 Phase 7B — Deployment-Oriented Structured PPO 120
6.11 Phase 7B — Memory and Computation Results 121
6.12 Phase 7B — Accuracy Trade-off 122
6.13 Direct Energy Confirmation and Measurement Failure 124
6.14 Three-Level Sustainability Assessment 125
6.15 Answer to RQ6 126
6.16 Chapter Summary 127
CHAPTER 7: DISCUSSION 128
7.1 General 128
7.2 What Adaptive Layer-Wise PPO Actually Contributes 128
7.3 The Role of Sensitivity Information 129
7.4 Search Quality versus Policy Retention 130
7.5 Reward Design and Over-Pruning 131
7.6 Validation-to-Test Transfer 132
7.7 Why LAMP Remains a Strong Benchmark, and Why Equal Fine-Tuning Changes the Rankings 133
7.7.1 LAMP 133
7.7.2 Equal fine-tuning 133
7.8 Compression versus Real Deployment Efficiency 134
7.9 Accuracy–Efficiency Trade-offs and the Three Levels of Efficiency 135
7.10 Interpreting the Sustainability Experiments 135
7.11 What the Negative Results Establish 137
7.12 Generalisability 138
7.13 Limitations 138
7.14 Overall Scientific Contribution 139
7.15 Chapter Summary 140
CHAPTER 8: CONCLUSION AND FUTURE WORK 142
8.1 General 142
8.2 Conclusions by Research Question 142

RQ1. Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP? 142
RQ2. Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?.. 143
RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention? 143
RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off? 143
RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods? 144
RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy? 144
8.3 Main Contributions 145
8.4 Overall Thesis Conclusion 146
8.5 Limitations 146
8.6 Future Work 147
Short-term 147
Medium-term 148
Long-term 148
8.7 Final Remarks 149
REFERENCES 150
APPENDIX A: ARCHITECTURES, HYPERPARAMETERS AND REPRODUCIBILITY A-1
APPENDIX B: CORE PPO AND SENSITIVITY SUPPLEMENTARY RESULTS B-1
APPENDIX C: ARCHIVE, REWARD AND RECOVERY DIAGNOSTICS C-1
APPENDIX D: STRUCTURED PRUNING AND HARDWARE PROTOCOL D-1
APPENDIX E: ENERGY EVALUATION AND QUALIFICATION RECORDS E-1

LIST OF FIGURES

Figure 3.1: Overall thesis framework 35

Figure 5.1: Test accuracy against total sparsity (Phase 3) 87

Figure 5.2: Final-policy pruning ratio per unit (Phase 3) 88

Figure 5.3: Unit sensitivity, pruning tolerance and PPO allocation (Phase 3) 90

Figure 5.4: Share of runs whose selected policy lies in the top-k of the VAL-RL (I) landscape (Phase 4, Stage C) 94

Figure 5.5: ResNet-8 reward alignment: test-accuracy drop below dense against selected-policy sparsity 98

Figure 5.6: Rank score of the sampled policy over 512 episodes, C1 (centred prior) vs C2 (no prior) (Phase 5, VAL-RL) 100

Figure 5.7: Median selected sparsity of C0, C1 and C2 against the maximum feasible sparsity on VAL-RL ($q \geq \tau$), by setting (Phase 5) 102

Figure 5.8: PPO − LAMP test accuracy before and after equal 5-epoch fine-tuning (Phase 6) 105

Figure 6.1: Measured latency reduction against (a) physical parameter reduction and (b) MAC reduction, for all 1,296 structured policies of the CIFAR-10 SimpleCNN and ResNet-8 tables (Paper 3) 112

Figure 6.2: ResNet-8 (CIFAR-10): table speed-up against retained validation accuracy (Paper 3) 115

Figure 6.3: Phase 7A: VAL-RL accuracy after recovery against measured package energy per 1,000 inferences 119

Figure 6.4: Phase 7B: VAL-SELECT accuracy after the equal 5-epoch recovery against MACs per image (left) and parameter memory (right) 123

LIST OF TABLES

Table 1.1: Research questions, objectives, contributions and evidence 10
Table 2.1: Pruning granularity 14
Table 2.2: Comparison of representative pruning-allocation and deployment-aware pruning works 21
Table 3.1: Development of the framework across phases 34
Table 3.2: Notation 36
Table 3.3: Pruning units of the core framework (unstructured) 38
Table 3.4: State features by phase 40
Table 3.5: Mature core framework (as frozen after Phase 5, evaluated in Phase 6) 48
Table 3.6: Three levels of deployment efficiency 51
Table 3.7: PPO configuration (identical in Phases 3, 4, 5 and 7B) 55
Table 4.1: Datasets and split configurations 58
Table 4.2: Data-split usage by phase 59
Table 4.3: Network architectures and pruning units (core framework) 60
Table 4.4: Dense reference models (accuracy, %) 61
Table 4.5: Baseline methods and matching rules 64
Table 4.6: PPO configuration by phase 65
Table 4.7: Phase-by-phase experimental design 67
Table 4.8: Reward configurations 73
Table 4.9: Evaluation metrics 74
Table 4.10: Pre-registered decision criteria 78
Table 4.11: Hardware and software environments 79
Table 5.1: Dense reference models (accuracy %, seed-0 reference) 85
Table 5.2: Phase 3: P0 minus baseline, test accuracy (pp, mean of 20 paired runs) 86
Table 5.3: Retention: the best policy encountered, the terminal policy (S0) and the archive-selected policy (S2A) 93
Table 5.4: Phase 4 pre-registered criteria 96
Table 5.5: ResNet-8 reward alignment across Phases 4 and 5 (test accuracy %, mean of 20 runs; sparsity = mean selected masked sparsity) 97
Table 5.6: Phase 5 by setting (test accuracy %, mean of 20 runs; differences in pp) 98
Table 5.7: Phase 5 pre-registered criteria 100
Table 5.8: Validation-to-test feasibility of the selected Phase-5 policies ($q \geq 0.98$; runs out of 120) 101
Table 5.9: Phase 6: test accuracy before → after equal fine-tuning, and the PPO–LAMP and PPO–Global gaps (pp; mean of 20 paired runs; 95% t-CI) 103

Table 5.10: Phase 6 pre-registered criteria 105
Table 5.11: Sensitivity guidance across Phases 3–5: exploration endpoints versus final-model endpoints (settings supported / opposite, of 6) 106
Table 6.1: Exhaustive structured latency tables: cost metrics against measured latency gain 111
Table 6.2: Two CIFAR-10 SimpleCNN policies: parameter reduction, MAC reduction and measured speed-up 113
Table 6.3: Validation of the 1-epoch accuracy proxy against 5-epoch recovery 114
Table 6.4: ResNet-8 (CIFAR-10): calibrated accuracy-feasible latency headroom 114
Table 6.5: Phase 7A: accuracy, compression, latency and measured energy 117
Table 6.6: Phase 7B: accuracy, memory and computation after the equal 5-epoch recovery (SimpleCNN, CIFAR-10) 121
Table 6.7: Phase-7B confirmatory energy pilot (dense model only; limit CV ≤ 3%) 124
Table 6.8: Three-level deployment-efficiency assessment 126
Table 7.1: Negative and partial results and what each establishes 137
Table 7.2: Stages of the diagnostic framework, the question each isolates, and the thesis evidence 140
Table 8.1: Thesis contributions and their boundaries 145
Table A.1: Pruning units, weight shapes, parameters and MACs per image (CIFAR-10 output layer) A-2
Table A.2: Dense references over three training seeds A-3
Table A.3: Equal recovery protocol (Phase 6; reused in Paper 3, Phases 7A and 7B) A-3
Table A.4: Pre-registration and freeze commits A-4
Table A.5: Software and hardware environments A-4
Table B.1: Phase 3 mean test accuracy and sparsity by setting and method (20 runs; baselines at each PPO run's zero count) B-1
Table B.2: Normalised loss sensitivity S_l by setting and the Phase-5 centred prior coefficients B-3
Table C.1: Phase 4 search quality by variant (VAL-RL (I) ranks among 1,296 policies; medians over 20 runs) C-2
Table C.2: Phase 5 selected policies by setting and condition (20 runs each) C-4
Table C.3: Phase 6 test accuracy before and after equal recovery (means of 20 runs) C-5
Table D.1: H0 calibration: per-configuration decision quantities D-1
Table D.2: Latency-table session log (anchor drift; limit 10%) D-2
Table D.3: Accuracy-proxy validation (1-epoch against 5-epoch recovery) D-2
Table D.4: ResNet-8 threshold calibration and latency-room decision D-3
Table E.1: Phase 7A energy: session summaries (five sessions per architecture) E-1
Table E.2: Phase 7B: all recovered final models (5-epoch recovery) E-2

Table E.3: Dense-only stability pilots and qualification (energy per 1,000 inferences, J).E-3

Table E.4: Energy-confirmation qualification blocks (descriptive diagnostics) E-3

LIST OF MAIN NOTATION AND ABBREVIATIONS

Main notation

N_L	number of pruning units; $N_L = 4$ for every architecture
r_l	pruning ratio assigned to unit l, $r_l = r(a_l) \in \mathcal{R}$
r (bold) / r(a)	**r**: the policy vector $[r_1, \ldots, r_{NL}]$. r(a): the map from action index to ratio
S_l	normalised sensitivity of unit l, $S_l \in [0, 1]$
$\tilde{S}_l$ (centred sensitivity)	$S_l - \bar{S}$, where $\bar{S} = (1/N_L)\ \Sigma\ S_j$ (Eq. 3.11); the centred prior uses $c_l = 1.0 \cdot \tilde{S}_l$
s_l	state observed by the agent before deciding unit l
a_l	action at unit l, $a_l \in \{0, \ldots, 5\}$
π_θ	PPO policy with parameters θ, $\pi_\theta(a \mid s)$ = softmax of the shifted logits (Eq. 3.10)
A_b	dense-reference accuracy on the reward split (VAL-RL)
A_p	accuracy of the pruned network $M_{\mathbf{r}}$ on the same split as A_b
q	retained accuracy A_p / A_b
ρ	masked weight sparsity, % of zero weights over all Conv2d/Linear weights (Eq. 3.2)
R	episode reward. Variants: R_0 (Eq. 3.7), R1a/R1b/R2 (Phase-4 Stage A), R_c (Eq. 3.15), R_{7B} (Eq. 3.19); $\hat{R}_t$ is the PPO return target
τ	retained-accuracy threshold of the constrained reward, τ = 0.98 (Phase 5 and the selection rule of Eq. 3.14)
latency, T(**r**)	measured batch-1 CPU latency per inference (ms); $T_{table}(\mathbf{r})$ is the frozen Paper-3 table value used by the 7B surrogate
energy, E(**r**)	directly measured CPU package energy per 1,000 inferences (J)
$\mathcal{R}$	ratio set {0, 0.1, 0.2, 0.3, 0.4, 0.6}
M, $M_{\mathbf{r}}$	dense reference network; network pruned by policy **r**
$\bar{S}$	mean sensitivity over units
c_l	prior coefficient (P1: $0.5 \cdot S_l$; centred: $1.0 \cdot \tilde{S}_l$)

C_l, k_l	channels before structured pruning; channels kept
D	number of distinct actions in **a**
$\mathcal{A}$	archive of unique visited policies
$\mathcal{C}$	top-K archived candidates (Eq. 3.14), $K_c = 10$
ε	accuracy tie-breaker weight in R_c and R_{7B} (0.01)
κ	penalty slope for infeasible policies ($\kappa = 10$)
α	weight of the predicted-saving term in R_{7B} ($\alpha = 4$)
a, b	surrogate intercept and slope (−0.117; 13.762)
η	weight of the behavioural-cloning (elite) loss, Phase-4 S2 only
$\hat{A}_t$, V_φ, $\boldsymbol{\mathcal{H}}$, c_v, c_e	PPO advantage, value function, entropy, value and entropy coefficients
MAC(**r**)	multiply–accumulate operations per image
ρ^*_s, ρ_{60}	R1b target sparsity; sparsity of the maximal policy [5,5,5,5] (R2)

General abbreviations

AC	Alternating current (mains power)
AI	Artificial intelligence
ARM	ARM processor architecture
AUC	Area under the curve (trajectory AUC of a search measure)
CI	Confidence interval
CNN	Convolutional neural network
CPU	Central processing unit
CSV	Comma-separated values
CV	Coefficient of variation
DRAM	Dynamic random-access memory
EDP	Energy–delay product
FLOP	Floating-point operation (FLOPs: floating-point operations)

GAE	Generalised advantage estimation
GPU	Graphics processing unit
HW	Hardware (table header, Chapter 2)
JSON	JavaScript Object Notation
LLM	Large language model
MAC	Multiply–accumulate operation (MACs: multiply–accumulate operations)
MKL	Math Kernel Library (Intel oneMKL)
MLP	Multilayer perceptron
NAS	Neural architecture search
NCHW	Tensor layout: batch (N), channels (C), height (H), width (W)
NLP	Natural language processing
OS	Operating system
PDH	Performance Data Helper (Windows performance-counter interface)
pp	Percentage point(s)
PP0	RAPL power plane 0 (processor cores)
PPO	Proximal Policy Optimisation
RAPL	Running Average Power Limit (Intel energy-counter interface)
ReLU	Rectified linear unit
RGB	Red–green–blue colour channels
RL	Reinforcement learning
RQ	Research question
SB3	Stable-Baselines3
SD	Standard deviation
SHA-256	Secure Hash Algorithm, 256-bit digest
SNR	Signal-to-noise ratio

TDP	Thermal design power
VAL-RL	Validation split used for sensitivity, reward and PPO training (Chapter 4)
VAL-SELECT	Validation split used only to select among candidates already produced (Chapter 4)

Method, model and dataset names

AMC	AutoML for Model Compression
BESA	Blockwise Parameter-Efficient Sparsity Allocation
CHEX	CHannel EXploration
CIFAR-10, CIFAR-100	Image-classification datasets of the Canadian Institute for Advanced Research [10 and 100 classes]
DDPG	Deep Deterministic Policy Gradient
DECORE	Deep Compression with Reinforcement Learning
ERK	Erdős–Rényi-kernel layer-wise sparsity allocation
GNN	Graph neural network
HALP	Hardware-Aware Latency Pruning
HAP	Hessian-Aware Pruning
HAQ	Hardware-Aware Automated Quantization
LAMP	Layer-Adaptive Magnitude-based Pruning (score)
LeNet-5, ResNet-8, SimpleCNN	Network architectures (Chapter 4)
LSTM	Long short-term memory
MNIST, PASCAL VOC, ImageNet	Datasets named in the literature review
OPT, LLaMA, Vicuna, Mistral, ViT, MobileNet V1/V2	Model families named in the literature review
OWL	Outlier Weighed Layerwise sparsity
PQ Index	Compressibility index named in the literature review

QLP	Deep Q-Learning for Pruning
SMCP	Soft Masking for Cost-Constrained Channel Pruning
SNIP	Single-shot Network Pruning

Experiment labels (thesis notation)

C10, C100	CIFAR-10, CIFAR-100 (in tables) (Chapter 5)
C0, C1, C2	Phase-5 conditions (R0 + centred prior; constrained reward + centred prior; constrained reward, no prior) (Chapter 4 §4.9.3)
F, K	Phase-4 final pipeline (R0, P2, S2A) and control (R0, P0, S0) (Chapter 4 §4.9.2)
H0, H1, H2-A	Paper-3 stages: latency calibration, structured-pruning validation, exhaustive latency tables (Chapter 4 §4.9.5)
O1–O6	Specific research objectives (Chapter 1 §1.6)
P0–P3 (Phase 3)	No prior; correct prior; shuffled prior; constant prior (Chapter 4 §4.9.1)
P0–P5 (Phase 4)	No prior; Phase-3 prior; centred prior; permuted centred; magnitude-matched conservative; sign-reversed (Chapter 4 §4.9.2)
R0, R1a, R1b, R2, R_c	Reward functions (Chapter 4 §4.12)
RQ1–RQ6	Research questions (Chapter 1 §1.5)
S0, S1, S2, S2A	Phase-4 search variants (terminal; archive; elite cloning; elite cloning + archive) (Chapter 4 §4.9.2)
TC1–TC6	Thesis contributions 1–6 (written "Contribution 1" … "Contribution 6" in prose) (Chapter 1 §1.7; Chapter 8 §8.3)
7A, 7B, 7B-EC	Phase 7A, Phase 7B, Phase-7B energy confirmation study (Chapter 4 §4.9)

CHAPTER 1
INTRODUCTION

1.1 Background

The cost of deployment. Deep neural networks reach high predictive accuracy on many tasks, but the models that do so are often large. Deploying them has a cost on four fronts:

(a) the parameter memory needed to store and load them;

(b) the arithmetic they perform for every input;

(c) the latency of each prediction;

(d) the energy drawn by the processor that runs them.

These costs matter most at inference time, which is repeated for every query. Inference energy and its trends have become a subject of study in their own right (Desislavov, Martínez-Plumed and Hernández-Orallo, 2023; Luccioni, Jernite and Strubell, 2024).

Pruning. Network compression reduces these costs while keeping accuracy, through pruning, quantisation, distillation and efficient architecture design (Menghani, 2023). *Pruning* removes parts of a trained network that contribute little to its predictions. It is one of the oldest compression techniques (LeCun, Denker and Solla, 1989; Han et al., 2015), and it remains among the most studied; a recent survey counts thousands of pruning papers in a few years (Cheng, Zhang and Shi, 2024).

What is removed:

(a) **Unstructured (masked) pruning** sets individual weights to zero. The tensors keep their shapes, and the zeros are recorded in a mask.

(b) **Structured pruning** removes whole channels, filters or neurons, so the network becomes physically smaller and its layers perform less arithmetic (He and Xiao, 2024).

Only the second shrinks the dense computation that general-purpose hardware actually executes (Hoefler et al., 2021).

How much each layer should lose. The simplest rule prunes every layer by the same fraction. Layers, however, differ in their share of the parameters, in their share of the

computation, and in how much accuracy is lost when they are pruned. This motivates *layer-wise adaptive* pruning, in which each layer receives its own ratio. Approaches to choosing the ratios include:

(a) **Magnitude-based rules,** which remove the smallest weights within each layer or across the whole network.

(b) **Layer-adaptive scores,** such as LAMP, which rescale weight magnitudes so that a single global threshold produces a non-uniform allocation (Lee et al., 2021).

(c) **Sensitivity-aware methods,** which measure how strongly each layer's pruning affects the loss.

(d) **Learned allocators,** which search for the ratios directly. Reinforcement learning (RL) agents that choose a ratio for each layer, rewarded by the accuracy and compression of the resulting network, are a well-known example (He et al., 2018).

This thesis uses Proximal Policy Optimisation (PPO) (Schulman et al., 2017) as such an allocator, and studies what it contributes, what it needs in order to work, and whether its compression becomes real deployment benefit.

Why a learned allocator, and what is hard about it. The quantity a learned allocator optimises need not be differentiable or expressible as a per-weight score. It can be validation accuracy after pruning, a hard accuracy constraint, or a measured hardware cost, and the same agent can be pointed at a different objective without redesigning the method. This flexibility has costs:

(a) **Evaluation cost.** Every candidate must be evaluated, so the search costs far more than a one-pass rule.

(b) **Sensitivity to design details.** Policy-gradient methods are sensitive to implementation and training choices (Engstrom et al., 2020; Andrychowicz et al., 2021).

(c) **Sensitivity to the reward.** The outcome depends on how the reward is written.

Whether the flexibility is worth the cost has to be measured against strong deterministic alternatives.

1.2 Motivation

The thesis is motivated by five linked problems. Each concerns a different step between "a network can be pruned" and "a pruned network is a better deployment choice".

Problem A: uniform pruning ignores heterogeneous layer sensitivity. A single ratio for all layers treats a small early convolution, on which every later layer depends, like a large fully connected layer with many redundant weights. An adaptive allocation should do better, but *how much* better, and against *which* alternatives? Strong deterministic layer-adaptive methods already exist, so a learned allocator must be compared with them under controlled conditions, not only with uniform pruning.

Problem B: a search may find strong policies without retaining them. An RL agent evaluates many candidate allocations but returns its final policy. If its distribution drifts away from the best candidates it has seen, a good search can still end on a mediocre allocation, and reporting only the final policy cannot distinguish weak search from weak retention.

Problem C: a reward can encourage excessive pruning. RL pruning rewards typically trade accuracy against sparsity at a fixed rate. Such a reward has no point at which accuracy loss becomes unacceptable. A more effective optimiser may then find exactly the over-pruned models the reward implicitly prefers.

Problem D: comparisons before equal recovery may overstate differences. Pruned networks are usually fine-tuned before use, and fine-tuning can repair much of the damage (Renda, Frankle and Carbin, 2020; Le and Hua, 2021). A method that looks better immediately after pruning may simply damage the network less before repair; its advantage after equal recovery is a separate question.

Problem E: sparsity does not necessarily mean speed or energy savings. A masked network still runs dense computations on a CPU. Even under structured pruning, the parameters removed and the computation saved need not coincide, and cost indicators can disagree (Dehghani et al., 2022). Speed and energy must be measured, with a reliable instrument.

These problems form a chain: a study that solves only the first can still return poor allocations, optimise the wrong trade-off, overstate its advantage, or claim efficiency it does not deliver. The design controls each problem:

(a) **A:** every baseline is built at exactly the sparsity the agent chose;

(b) **B:** every policy the agent visits is recorded and ranked against an exhaustive landscape;

(c) **C:** rewards are compared on the same landscapes, and the accuracy boundary is made explicit;

(d) **D:** every method receives the same fixed-mask recovery with a shared data stream;

(e) **E:** pruning is realised physically and measured on hardware, with the measuring instrument qualified before any comparison.

1.3 Problem Statement

Existing pruning methods often optimise compression and accuracy through fixed rules or single saliency criteria. Reinforcement learning offers a flexible mechanism for learning non-uniform layer-wise allocations, but its effectiveness depends on four things beyond the learning algorithm itself:

(a) the quality of the guidance given to the search;

(b) whether good policies found during the search are retained;

(c) whether the reward encodes the intended accuracy–compression trade-off;

(d) how the pruned models are recovered before comparison.

Moreover, the fraction of zero weights does not by itself establish deployment efficiency. This thesis therefore investigates whether a PPO agent can learn adaptive layer-wise pruning policies, what sensitivity, retention, reward design and equal recovery contribute, and whether the resulting compression becomes memory, computation, latency and measured energy benefits.

Research approach. The investigation was a sequence of experimental phases, each with conditions, seeds, metrics and decision criteria committed before its confirmatory data were collected (Chapter 4). Four design choices make the results auditable:

(a) **Exhaustive ground truth.** Four pruning units with six ratios each give 1,296 policies per setting. Every policy was evaluated once, so the rank of any policy the agent visits or returns is known exactly, and search and retention are measured against ground truth.

(b) **Strict data governance.** The training images update weights only. One validation split drives the reward, and a second, disjoint split is used only to select among candidates already produced. The test set is read only for frozen models.

(c) **Matched comparisons.** In the core framework, every baseline is built at exactly the number of zeroed weights of the PPO model it is compared with. After pruning, every compared model receives an identical recovery protocol.

(d) **A separate extension.** A later extension realises pruning physically and measures latency and energy directly, under pre-registered stability checks for the measuring instrument.

1.4 Research Gap

Chapter 2 identifies six narrower gaps. The claim is not that RL has never been applied to pruning; it has (He et al., 2018). The gaps concern how such methods are evaluated and what their results establish.

(i) **Adaptive allocation against strong deterministic allocation.** Layer-specific allocation matters, but learned allocators are rarely compared with uniform, random, magnitude-based and layer-adaptive baselines built at exactly the same sparsity. → RQ1.

(ii) **Exploration effects versus final-model effects of sensitivity.** Sensitivity is widely used to guide pruning, but its effect on the efficiency of a search is seldom separated from its effect on final accuracy, or compared with uninformative controls. → RQ2.

(iii) **Retention of strong policies found during search.** RL pruning is usually judged by its final policy. Whether stronger policies were encountered and lost has not been examined systematically for pruning. → RQ3.

(iv) **Incentives for excessive sparsity in pruning rewards.** Hand-weighted accuracy–sparsity rewards can be exploited by capable optimisers (Pan, Bhatia and Steinhardt, 2022). Whether better pruning search exposes this, and whether an explicit accuracy constraint controls it, has not been tested directly. → RQ4.

(v) **Comparisons without equal recovery.** Pruning methods are often compared under inconsistent experimental conditions (Blalock et al., 2020), and the retraining schedule alone can change which pruned network performs better (Le and Hua, 2021). Whether differences between a learned allocator and deterministic methods, measured before recovery or under different recovery budgets, survive identical post-pruning fine-tuning therefore remains to be tested. → RQ5.

(vi) **Compression versus deployment efficiency.** Several questions remain empirical:

(a) how well parameter and MAC reductions predict measured CPU latency across a whole pruning space;

(b) how much latency headroom remains under an accuracy constraint;

(c) whether a structured realisation of a learned allocation reduces directly measured energy without material accuracy loss.

Energy measurement on general-purpose CPUs adds its own reliability problem. → RQ6.

1.5 Research Questions

RQ1–RQ5 concern the **core framework**, which uses masked (unstructured) pruning. **RQ6** concerns the **deployment extension**, added after the core framework had been completed and evaluated.

> **RQ1.** Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP?
>
> **RQ2.** Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?

RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention?

RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off?

RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods?

RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy?

The questions were not all fixed at the outset; each answer exposed the next problem (Chapter 3). Improving the search revealed a retention problem, retention revealed reward misalignment, a corrected reward made a fair recovery comparison meaningful, and the deployment question followed.

1.6 Research Objectives

General objective. To develop and systematically evaluate a sensitivity-aware reinforcement-learning framework for adaptive layer-wise neural-network pruning, including its search behaviour, recovery characteristics and deployment implications.

Specific objectives:

(a) **O1.** Develop a PPO-based framework for non-uniform layer-wise pruning, and evaluate it against uniform, random, global-magnitude, ERK and LAMP allocations at identical sparsity.

(b) **O2.** Evaluate whether sensitivity guidance improves the pruning search and the final model quality, using shuffled, constant, magnitude-matched and sign-reversed controls.

(c) **O3.** Investigate policy-retention behaviour, and evaluate archive-based selection of the policies visited during search.

(d) **O4.** Analyse reward misalignment, and evaluate an accuracy-constrained reward with validation-based selection.

(e) **O5.** Assess how an equal, fixed-mask post-pruning recovery protocol affects comparative conclusions between PPO and deterministic methods.

(f) **O6.** Evaluate whether pruning-related compression translates into memory, computation, latency and directly measured operational-energy improvements when realised as physically structured networks.

Each objective was addressed by completed, pre-registered experiments (Chapter 4); Table 1.1 maps objectives to research questions, contributions and evidence.

1.7 Research Contributions

The thesis makes six contributions. None of them rests on a claim that PPO is the best pruning method.

Contribution 1. An adaptive layer-wise pruning framework, evaluated against strong baselines. A PPO agent assigns one of six pruning ratios to each of four units per network. It was evaluated against uniform, random, global-magnitude, ERK and LAMP allocations built at exactly the sparsity the agent chose, across two datasets and three architectures. The agent learns non-uniform allocations that strongly outperform uniform and random allocation; the deterministic LAMP method remains stronger before recovery.

Contribution 2. A separation of the exploration effect and the final-model effect of sensitivity guidance. Across three experimental phases, and against several uninformative controls, the analysis shows that sensitivity's main measurable contribution is to the *search*: better-ranked policies earlier and fewer destructive ones. Its contribution to final predictive accuracy was not detectable.

Contribution 3. Identification of a policy-retention problem in RL pruning, and evaluation of archive-based selection. Using exhaustive ground-truth landscapes, the thesis shows that the agent visits near-optimal policies but rarely ends on them. An archive of visited policies, with selection on held-out validation data, restores retention in terms of validation rank. Its effect on test accuracy is mixed.

Contribution 4. Identification of reward misalignment, and evaluation of an accuracy-constrained reward. Improving search and retention exposed that a fixed-rate accuracy–sparsity reward drives fragile networks to over-prune. A constraint-first reward,

with selection on held-out data, substantially reduced the most destructive over-pruning. Validation feasibility, however, did not reliably transfer to the test set.

Contribution 5. A controlled equal-recovery analysis. Every compared model received an identical fixed-mask fine-tuning protocol, with shared seeds and data order, at identical sparsity. This shows how much of the difference between pruning methods remains after recovery: the differences became small. The thesis's main methodological recommendation for pruning comparisons follows from it.

Contribution 6. A deployment analysis that keeps four quantities apart: parameter compression, MAC reduction, measured latency and directly measured operational energy. It comprises:

(a) an exhaustive latency analysis of every structured policy of two networks and two output sizes;
(b) direct CPU package-energy measurement of a structured realisation of the learned allocation;
(c) a deployment-oriented structured PPO that learns physical channel allocations directly;
(d) pre-registered qualification of the energy instrument, reported including its failure.

Novelty. The novelty does not lie in a new pruning criterion, or in a claim that PPO outperforms existing methods. It lies in **treating adaptive pruning as a sequence of connected optimisation and validation problems, rather than evaluating only the final compressed model.** These problems are allocation, exploration, policy retention, reward alignment, recovery, deployment translation and hardware validation. Each is examined with its own experiment and decision criteria fixed before the confirmatory data were collected. The contribution is methodological and analytical: it shows how a weakness at one stage can hide or reverse the effect of improvements at another.

Table 1.1: Research questions, objectives, contributions and evidence.

RQ	Objective	Contribution	Evidence
RQ1	O1	TC1	Chapter 5 (Phase 3; Phases 4–5 LAMP comparisons)
RQ2	O2	TC2	Chapter 5 (Phases 3, 4, 5)
RQ3	O3	TC3	Chapter 5 (Phase 4)
RQ4	O4	TC4	Chapter 5 (Phases 4, 5)
RQ5	O5	TC5	Chapter 5 (Phase 6)
RQ6	O6	TC6	Chapter 6 (Paper 3, Phases 7A, 7B, energy confirmation)

TC1–TC6 denote Contributions 1–6. The labels C0, C1 and C2 used in later chapters denote Phase-5 experimental conditions, not contributions.

Preview of the main findings (reported in full in Chapters 5–7):

(a) PPO's learned allocations substantially outperformed uniform pruning (by up to about 31.4 percentage points of test accuracy) and random pruning in the studied settings, but LAMP remained the stronger method before equal recovery.

(b) Sensitivity guidance improved exploration but not final accuracy. Archive selection restored validation-rank retention; its test effect was mixed.

(c) The accuracy-constrained reward substantially reduced destructive over-pruning. Its validation-certified feasibility, however, held on the test set in only about two thirds of runs.

(d) Equal recovery greatly narrowed the differences between PPO and LAMP. They fell within a pre-registered confidence-interval margin in every setting, although LAMP remained slightly ahead in three.

(e) On the deployment side, parameter reduction was a weak predictor of CPU latency compared with MAC reduction. Physically structured pruning reduced directly measured CPU package energy (Phase 7A), but with a substantial accuracy cost; on one network, a simple uniform allocation was the better choice.

(f) The redesigned agent (Phase 7B) reduced parameter memory by 63.4% and MACs by 36.0% at a validation-accuracy cost of 1.12 percentage points. It missed the pre-registered accuracy criterion, and its direct energy could not be confirmed.

1.8 Scope and Boundaries

Included in the scope:

(a) **Datasets:** CIFAR-10 and CIFAR-100 image classification.

(b) **Networks:** three small convolutional networks (SimpleCNN, a CIFAR adaptation of LeNet-5, and ResNet-8), each with four pruning units and six pruning ratios.

(c) **Method:** PPO; masked (unstructured) pruning in the core framework, with all 1,296 policies per setting evaluated as ground truth; physically structured pruning on a subset of settings in the deployment extension.

(d) **Hardware evaluation:** parameter memory, MACs, batch-1 CPU latency, and CPU package energy measured through the processor's energy counters, on one laptop CPU.

Not established by this thesis:

(a) behaviour at ImageNet scale, or for larger CNNs, transformers or large language models;

(b) performance on GPUs, mobile processors, edge accelerators or cloud inference;

(c) whole-system or wall-plug energy, memory energy, or training energy;

(d) any carbon-footprint, lifecycle or environmental-impact claim.

The energy evaluation concerns operational inference energy; it is not an environmental assessment.

Boundaries that the reader should keep in mind throughout:

(a) PPO is **not** presented as universally better than deterministic pruning. LAMP remains a strong baseline, and was more accurate than PPO before recovery in nearly every setting.

(b) Sensitivity guidance is, on the evidence, primarily an **exploration aid**.

(c) Archive selection is a **validation-retention mechanism**, not a generalisation mechanism.

(d) Validation-based accuracy constraints **do not guarantee** feasibility on the test set.

(e) The ±0.5 percentage-point rule used after equal recovery is a pre-registered confidence-interval decision rule, and **not** a formal statistical equivalence test.

(f) **Phase 7A** directly measured energy reductions, but every pruned model lost more accuracy than the criterion allowed.

(g) **Phase 7B's** direct energy remains **unresolved**, because the measurement setup failed its stability check. No latency or energy reduction is claimed for it.

1.9 Thesis Organisation

Chapter 2 reviews the literature on compression, layer-wise and layer-adaptive allocation, RL for compression, sensitivity, search and policy selection, recovery, hardware-aware compression and energy measurement, and ends with the six research gaps. Chapter 3 presents the framework in the order in which it developed, with the later deployment-oriented structured extension presented separately. Chapter 4 gives the experimental setup: data and validation configurations, architectures, baselines and matching rules, PPO configuration, the conditions of every phase, metrics, statistical decision rules, the latency and energy protocols, and reproducibility. Chapter 5 reports the core-framework results (Phases 3–6, RQ1–RQ5). Chapter 6 reports the deployment analysis: exhaustive latency, the structured transfer with measured energy, the deployment-oriented structured PPO and the energy-confirmation attempt (RQ6). Chapter 7 interprets the findings across phases and states the limitations. Chapter 8 concludes and outlines future work.

1.10 Chapter Summary

This chapter argued that adaptive layer-wise pruning is a chain of connected problems rather than a single choice of ratio, and set out six research questions, objectives and contributions: **RQ1–RQ5** for a PPO-based, sensitivity-aware core framework using masked pruning, and **RQ6** for a structured deployment extension with direct hardware measurement. The novelty is methodological: each stage is evaluated on its own evidence, including its negative results. The thesis does not claim superiority over existing methods, and its scope is limited to small CNNs, CIFAR data and one CPU platform.

Chapter 2 reviews the literature on which these questions build.

CHAPTER 2
LITERATURE REVIEW

This chapter reviews the research on which the thesis builds. Sections 2.1–2.2 cover compression and pruning granularity; Sections 2.3–2.5 the allocation problem; Sections 2.6–2.9 reinforcement learning (RL) for compression, PPO, sensitivity, and search and selection; Section 2.10 recovery; Sections 2.11–2.13 hardware-aware compression, cost metrics and energy measurement. Section 2.14 states the research gap.

2.1 Neural Network Compression

The accuracy gains of deep neural networks have come with growing parameter counts, arithmetic cost and memory traffic, making deployment expensive on servers and difficult on resource-limited devices (Hoefler et al., 2021; Menghani, 2023). Compression reduces these costs while keeping predictive quality close to that of the original model. The main families are:

(a) **pruning**, which removes parameters or structures;
(b) **quantisation**, which lowers numerical precision;
(c) **knowledge distillation**, which trains a smaller model to imitate a larger one;
(d) **efficient architecture design**, sometimes automated through neural architecture search.

Several recent surveys organise this field (Menghani, 2023; Cheng, Zhang and Shi, 2024; He and Xiao, 2024).

Pruning is among the oldest of these ideas. Early work removed weights by a second-order estimate of how much the training loss would increase without them (LeCun, Denker and Solla, 1989). Later work showed that simply removing small-magnitude weights and retraining could reduce the parameter count of large convolutional networks many times over with little accuracy loss (Han et al., 2015). The lottery ticket hypothesis then argued that dense networks contain sparse subnetworks that train to comparable accuracy, which renewed interest in why pruning works (Frankle and Carbin, 2019).

One recent survey counts more than three thousand pruning papers published between 2020 and 2024 (Cheng, Zhang and Shi, 2024). Pruning and quantisation are also often

compared directly. A systematic study found that quantisation usually gives a better accuracy–compression trade-off than pruning, except at very high compression ratios (Kuzmin et al., 2023). This thesis restricts itself to pruning, because its questions concern *where* capacity should be removed; quantisation is complementary and is not studied.

Two observations from this literature shape the thesis.

(i) **"Efficiency" is not a single quantity.** A pruned model may need less storage, fewer arithmetic operations, less time per inference or less energy, and these do not necessarily move together (Hoefler et al., 2021; Dehghani et al., 2022). Sections 2.11–2.13 return to this point.

(ii) **Reported compression results are hard to compare.** Studies differ in architectures, datasets, training budgets and fine-tuning protocols, and these differences can be larger than the differences between pruning methods (Blalock et al., 2020; Liu et al., 2023). This motivates the controlled comparisons in this thesis.

2.2 Unstructured and Structured Pruning

Pruning methods are commonly classified by the *granularity* of what they remove (Table 2.1).

Table 2.1: Pruning granularity.

Granularity	What is removed	Resulting model	Typical deployment benefit
Unstructured (weight-level)	Individual weights	Same tensor shapes, with a sparse mask	Smaller compressed storage; speed-up only with sparse kernels or hardware support
Semi-structured (e.g. N:M)	Fixed patterns of weights within small blocks	Same shapes, regular sparsity pattern	Speed-up on hardware that supports the pattern
Structured (channel, filter, neuron)	Whole channels, filters or neurons	Physically smaller dense tensors	Fewer MACs and smaller dense kernels, usable by standard libraries

Unstructured pruning removes individual weights. Because the choice of which weights to keep is unconstrained, it can reach high sparsity at a given accuracy (Han et al., 2015; Hoefler et al., 2021), but the model keeps its tensor shapes, with zeros stored and computed unless sparse formats and kernels are used. On general-purpose hardware such kernels are effective only at high sparsity and favourable sparsity patterns, so weight-level

sparsity often brings little or no reduction in latency (Hoefler et al., 2021; He and Xiao, 2024).

Structured pruning removes whole structures such as convolutional filters, channels or fully connected neurons (Wen et al., 2016; He, Zhang and Sun, 2017; Li et al., 2017). The pruned network is a smaller dense network: it can be executed by standard dense libraries and directly reduces MACs and memory traffic. A survey of structured pruning describes it as the route to "realistic acceleration" (He and Xiao, 2024). The price is a coarser search space: removing a channel removes all its weights at once, and accuracy typically degrades faster than with unstructured pruning at the same parameter reduction.

Recent work on structured pruning has concentrated on two problems.

(a) **Dependency handling.** In residual and attention architectures, removing a channel in one layer forces matching removals elsewhere. DepGraph models these dependencies as a graph so that coupled parameters are pruned together across arbitrary architectures (Fang et al., 2023). Isomorphic pruning compares the importance of sub-structures only within groups of the same computational topology, because heterogeneous structures have incomparable parameter scales (Fang et al., 2024).

(b) **Pruning during training.** CHEX repeatedly prunes and regrows channels during training instead of pruning a fully trained model once (Hou et al., 2022). OTOv2 trains and compresses a model in a single pass without separate fine-tuning (Chen et al., 2023). Submodular structured pruning offers guarantees in a limited-data setting (El Halabi, Srinivas and Lacoste-Julien, 2022).

Random structure as a baseline. A channel-pruning benchmark found that random search over channel configurations works well compared with other channel-pruning methods, with no clear winner among channel-importance criteria (Li et al., 2022). *How much* to prune in each layer, together with the training protocol, can matter as much as *which* channels are removed.

Relevance to this thesis. The core framework (Chapters 3–5) uses magnitude-based *unstructured* masks within each layer and learns the per-layer ratio, which made exhaustive evaluation of the policy space possible. The deployment extension (Chapter 6) realises per-layer allocations as *physically structured* networks, so that memory, MACs,

latency and energy can be measured; the gap between the two realisations is itself a finding of Chapter 6.

2.3 Layer-Wise Adaptive Pruning

A pruning method must decide not only *which* parameters to remove but also *how many* to remove from each layer: the layer-wise sparsity allocation. The simplest choice applies the same ratio to every layer (uniform pruning). Layers differ, however, in size, position and redundancy. Early layers of a convolutional network are small and often fragile, while large fully connected or late convolutional layers often contain many redundant parameters. A uniform allocation therefore over-prunes some layers and under-prunes others. A sizeable literature shows that the allocation itself is a first-order factor in pruned-model quality (Lee et al., 2021; Liu et al., 2022; Gadhikar, Mukherjee and Burkholz, 2023).

Approaches to the allocation problem fall into four broad classes.

(i) **Rule-based allocations** fix the ratios from layer properties. An example is the Erdős–Rényi-Kernel (ERK) allocation, which scales density with the layer's fan-in and fan-out and was popularised in sparse training (Evci et al., 2020).

(ii) **Global ranking** applies one importance threshold across all layers, so that allocation emerges from where low-importance weights happen to lie. Global magnitude pruning is the simplest such method.

(iii) **Score-normalised global ranking** rescales importance scores so that a single global threshold produces a better allocation. LAMP is the best-known example (Section 2.5) (Lee et al., 2021).

(iv) **Search- or learning-based allocation** treats the per-layer ratios as variables to optimise, using RL (He et al., 2018; Camci et al., 2022), learned global filter ranking (Chin et al., 2020), fast sub-network evaluation (Li et al., 2020) or bi-level optimisation (Zhang et al., 2022).

Recent results have sharpened the importance of allocation. In sparse training, randomly pruned networks can be competitive provided that the per-layer ratios are sensible; the layer-wise ratio, rather than the precise mask, was a key factor (Liu et al., 2022). A theoretical analysis complements this, explaining when random masks with appropriate

layer-wise densities can approximate target networks (Gadhikar, Mukherjee and Burkholz, 2023). The same question has reappeared in large language models. One-shot LLM pruning methods traditionally prune every layer to the same sparsity, but OWL allocates higher sparsity to layers with fewer outlier activations and improves markedly at high sparsity (Yin et al., 2024). BESA learns block-wise sparsity allocations under a global budget (Xu et al., 2024). Other work proposes a sparsity-based compressibility index (PQ Index) to guide adaptive pruning (Diao et al., 2023).

What remains less studied is how a *learned* allocation compares with cheap deterministic allocations once the sparsity level, recovery budget and selection procedure are controlled. Learned allocators are usually reported against published numbers obtained under different protocols, rarely at the exact sparsity the learned method chose and under identical recovery (Research Question 1).

2.4 Magnitude-Based and Deterministic Pruning

Magnitude pruning removes the weights of smallest absolute value. Despite its simplicity it remains one of the strongest and most widely used baselines (Han et al., 2015; Blalock et al., 2020). It requires no data, no gradients and no search, and it combines easily with any allocation rule.

Variants differ in the *scope* over which magnitudes are compared:

(a) **Layer-wise magnitude pruning** removes a fixed fraction within each layer, so it needs an external allocation.

(b) **Global magnitude pruning** compares magnitudes across all layers. Because weight scales differ between layers, it can prune some layers almost completely, which is especially harmful in small layers.

Systematic evaluations repeatedly find that carefully tuned simple methods are hard to beat. ShrinkBench documented that many papers compare against weak or inconsistently configured baselines, and that simple magnitude pruning remains competitive with more complex methods when evaluated consistently (Blalock et al., 2020). Re-examinations of structured pruning found that training a pruned architecture from scratch can match fine-tuning an inherited one, which questions how much the specific inherited weights matter

(Liu et al., 2019). Benchmarks of sparse networks on harder tasks have also shown that apparent gains can vanish under more demanding evaluation (Liu et al., 2023).

Deterministic methods (magnitude pruning, global ranking, ERK, LAMP) are cheap, reproducible and free of search hyperparameters, so any learned method must justify its additional cost against them. The thesis compares PPO with Uniform, Random, global magnitude, ERK and LAMP at the same sparsity as each PPO run, rather than with numbers reported elsewhere.

2.5 LAMP and Related Layer-Adaptive Methods

Layer-Adaptive Magnitude-based Pruning (LAMP) rescales each weight's magnitude by the magnitudes of the weights in the same layer that would survive if it were pruned. This yields a score that reflects the model-level $\ell 2$ distortion caused by pruning (Lee et al., 2021). A single global threshold on the LAMP score then produces a layer-wise allocation automatically, without hyperparameters and with negligible computation. In its original evaluation, LAMP consistently outperformed popular existing schemes for layer-wise sparsity selection under various image-classification setups. It also remained strong under weight rewinding, where the connectivity-oriented allocation, the strongest baseline overall, fell behind even simple global magnitude pruning (Lee et al., 2021).

LAMP is a strong *deterministic* answer to exactly the problem a learned allocator addresses, and it costs almost nothing to compute, whereas an RL search requires thousands of pruned-model evaluations. It is therefore the natural benchmark for the practical value of RL-based allocation.

Related layer-adaptive methods include:

(a) **Learned global ranking**, which learns a single ranking of filters across layers, from which architectures with different accuracy–latency trade-offs follow by pruning the lowest-ranked filters (Chin et al., 2020);
(b) **Outlier-based allocations for LLMs** (Yin et al., 2024);
(c) **Block-wise learned allocations** (Xu et al., 2024).

Limitation shared by these comparisons. These methods, like most allocators, were compared under each paper's own training and fine-tuning protocol. Chapter 5 compares

PPO and LAMP before and after identical recovery, separating the contribution of allocation from that of fine-tuning.

2.6 Reinforcement Learning for Neural Network Compression

Compression can be framed as sequential decisions: an agent visits the layers one by one, chooses a ratio for each, and is rewarded by the accuracy and cost of the compressed model. This connects compression to neural architecture search, where an RL controller generated architectures rewarded by their validation accuracy (Zoph and Le, 2017).

AMC made the formulation concrete for pruning (He et al., 2018). A DDPG agent receives an embedding of each layer and outputs a continuous sparsity ratio. The reward combines accuracy with resource constraints. AMC reported better accuracy than hand-crafted compression at a given FLOPs reduction, and measured speed-ups on a GPU and on mobile hardware (He et al., 2018). Platform-aware methods such as NetAdapt and HAQ extended the idea by placing measured latency or hardware feedback inside the optimisation loop (Yang et al., 2018; Wang et al., 2019).

Recent RL-based pruning research has explored several directions (Table 2.2):

(a) **Q-learning allocation.** QLP uses deep Q-learning to choose layer-wise sparsity ratios. It can revisit layers and prune them gradually instead of making a single first-to-last pass (Camci et al., 2022).

(b) **Lookahead search.** RL guided by a lookahead search has been used to select the filters to remove in channel pruning (Wang and Li, 2022).

(c) **Per-channel agents.** DECORE assigns a lightweight agent to each channel and trains the agents with a policy gradient method, so that channel retention is learned jointly with the network (Alwani, Wang and Madhavan, 2022).

(d) **Topology embeddings.** GNN-RL embeds the network's computational graph with a graph neural network and uses RL to find a compression policy; it reports results on ResNet, VGG, MobileNet and ShuffleNet models (Yu, Mazaheri and Jannesari, 2022).

(e) **Joint training and pruning.** An RL agent sets layer pruning ratios while the network is being trained, rather than after a full pre-training run (Ganjdanesh, Gao and Huang, 2024).

These methods show that RL can find competitive allocations and incorporate non-differentiable objectives. Three features of their evaluation leave questions open, which the thesis addresses (Research Questions 3, 4 and 5):

(i) **Final-policy evaluation**, typically after method-specific fine-tuning, without examining whether the agent *retained* the best policy it encountered during search.

(ii) **Hand-built rewards**, whose coefficients shape pruning aggressiveness in ways that are seldom analysed, including whether better search exploits weaknesses in the reward.

(iii) **Comparisons under different protocols**, against published results with different recovery budgets, which makes it hard to attribute gains to the allocation itself.

Table 2.2: Comparison of representative pruning-allocation and deployment-aware pruning works. "Struct." = structured pruning; "Sens." = an explicit sensitivity or saliency signal drives allocation; "HW metric" = a measured or table-based hardware metric (latency, throughput) enters the method or its evaluation; "Energy" = inference energy measured on hardware. Datasets and models are as stated in each paper's abstract. Limitations are stated relative to the questions of this thesis.

Work	Year	Pruning type	Selection / optimisation	Layer-adaptive	Sens.	RL	Struct.	HW metric	Energy	Datasets / models	Main limitation (for this thesis' questions)
Energy-aware pruning (Yang, Chen and Sze, 2017)	2017	Weight	Energy-model-guided greedy	Yes	No	No	No	Energy model	Estimated (model calibrated on hardware)	ImageNet CNNs (AlexNet, GoogLeNet)	Energy is modelled, not measured per inference
AMC (He et al., 2018)	2018	Weight / channel	DDPG agent per layer	Yes	No	Yes	Both	Latency (GPU, mobile)	No	ImageNet VGG-16, MobileNet	Final-policy evaluation; reward coefficients fixed by hand
NetAdapt (Yang et al., 2018)	2018	Filter	Iterative, measured-latency-driven	Yes	No	No	Yes	Measured latency	No	ImageNet MobileNet V1/V2	Platform-specific measurement loop at each step
LAMP (Lee et al., 2021)	2021	Weight	Rescaled-magnitude global threshold	Yes	Distortion score	No	No	No	No	Image-classification CNNs	Unstructured: no direct latency benefit
QLP (Camci et al., 2022)	2022	Weight	Deep Q-learning, multi-pass	Yes	No	Yes	No	No	No	CIFAR-10 ResNet-32/56; ImageNet ResNet-50, MobileNet-v1	Unstructured masks; deployment effect not measured
Lookahead RL (Wang and Li, 2022)	2022	Channel	RL with lookahead search	Yes	No	Yes	Yes	No	No	MNIST, CIFAR-10, ImageNet	Final-policy evaluation
DECORE (Alwani, Wang and Madhavan, 2022)	2022	Channel	Per-channel policy-gradient agents	Yes	No	Yes	Yes	FLOPs / memory constraints	No	CIFAR-10 ResNet-110, VGG; ImageNet ResNet-50	FLOPs used as cost, not measured latency
GNN-RL (Yu, Mazaheri and Jannesari, 2022)	2022	Channel	GNN embedding + RL	Yes	Topology	Yes	Yes	FLOPs-constrained	No	ResNet, VGG-16, MobileNet-v1/v2, ShuffleNet	Cost measured as FLOPs

Work	Year	Pruning type	Selection / optimisation	Layer-adaptive	Sens.	RL	Struct.	HW metric	Energy	Datasets / models	Main limitation (for this thesis' questions)
HAP (S. Yu et al., 2022)	2022	Structured	Second-order (Hessian) sensitivity	Yes	Yes	No	Yes	No	No	CIFAR-10 PreResNet29; ImageNet ResNet-50	Sensitivity used for selection; no search or retention question
HALP (Shen et al., 2022)	2022	Channel	Latency-constrained knapsack	Yes	Saliency	No	Yes	Latency lookup table	No	ImageNet ResNet-50/101; VOC detection	Needs per-device latency tables; energy not measured
SMCP (Humble et al., 2022)	2022	Channel	Soft masking under a cost constraint	Yes	Saliency	No	Yes	Cost (latency) constraint	No	ImageNet, PASCAL VOC	Energy not measured
CHEX (Hou et al., 2022)	2022	Channel	Prune-and-regrow during training	Yes	No	No	Yes	FLOPs	No	ImageNet ResNet-50; detection, segmentation	FLOPs as cost
DepGraph (Fang et al., 2023)	2023	Structured (any)	Dependency graph + norm criterion	Yes	No	No	Yes	No	No	CNNs, ViT, GNN, LSTM	Grouping, not allocation search
OTOv2 (Chen et al., 2023)	2023	Structured	Train-once optimiser	Yes	No	No	Yes	No	No	VGG, ResNet, ConvNeXt, DenseNet; CIFAR-10/100, ImageNet	Allocation implicit in training
OWL (Yin et al., 2024)	2024	Weight	Outlier-ratio layer allocation	Yes	Outlier statistic	No	No	Inference speed (sparse engine)	No	LLaMA, Vicuna, OPT, Mistral	LLM-specific signal
RL agent + joint training (Ganjdanesh, Gao and Huang, 2024)	2024	Channel	RL agent sets layer ratios during training; accuracy reward	Yes	No	Yes	Yes	No	No	CIFAR-10, ImageNet; ResNets, MobileNets	Final-policy evaluation
This thesis	—	Weight (core); channel (extension)	PPO + centred sensitivity prior + archive + constrained reward	Yes	Yes	Yes	Extension	Measured CPU latency	RAPL (7A); unresolved (7B)	CIFAR-10/100; SimpleCNN, LeNet-5, ResNet-8	Small models; one CPU platform; LAMP remains stronger before recovery

2.7 PPO-Based Optimisation

Proximal Policy Optimisation (PPO) is an on-policy policy-gradient algorithm. It stabilises learning by limiting how far each update can move the policy, using a clipped surrogate objective (Schulman et al., 2017). It is commonly combined with generalised advantage estimation (Schulman et al., 2016). PPO is popular because it is comparatively insensitive to hyperparameters, handles discrete and continuous actions, and has mature implementations such as Stable-Baselines3 (Raffin et al., 2021), which this thesis uses. Large empirical studies have qualified this picture:

(a) **Implementation details.** Code-level choices that are not part of the algorithm's description (for example value-function clipping and reward normalisation) can account for a large share of PPO's reported advantage over related methods (Engstrom et al., 2020).

(b) **Design choices.** A study of more than fifty design choices in on-policy actor-critic methods found that many settings matter considerably and offered practical recommendations (Andrychowicz et al., 2021).

(c) **Multi-agent settings.** PPO proved surprisingly strong in cooperative multi-agent games, where it had been assumed less sample-efficient than off-policy methods (C. Yu et al., 2022).

PPO is therefore a reasonable and well-understood optimiser, but its behaviour must be examined empirically, especially in an unusual setting such as pruning allocation. The pruning task differs from typical RL benchmarks in three ways.

(i) **Short episodes.** An episode consists of a handful of decisions, one per prunable unit, so the problem resembles a contextual combinatorial search more than long-horizon control.

(ii) **Expensive, noisy reward.** The reward requires evaluating a pruned model on held-out data, and its noise comes from a finite validation set.

(iii) **Small, enumerable space in this thesis.** With four units and six ratios there are 1,296 policies, so the full reward landscape can be computed and each PPO run compared against the exact optimum.

The last feature, unusual in the RL-for-compression literature, allows search quality (how highly ranked are the policies PPO visits?) to be measured separately from final-policy quality (how highly ranked is the policy it ends on?).

2.8 Sensitivity-Aware Pruning

Sensitivity, how much a model's loss or accuracy changes when a parameter, structure or layer is pruned, is one of the oldest ideas in pruning:

(a) **Second-order estimates.** Optimal Brain Damage estimated weight saliency from second-order derivatives of the loss (LeCun, Denker and Solla, 1989). Modern second-order structured methods capture correlations across structures with Hessian-vector products (Nonnenmacher et al., 2022). Hessian-aware pruning uses second-order sensitivity to decide which structures to prune and which to replace (S. Yu et al., 2022). Global structural pruning of vision transformers uses a Hessian-aware saliency to redistribute parameters across blocks (Yang et al., 2023).

(b) **First-order (Taylor) estimates.** These approximate the loss change caused by removing a filter or neuron from gradients (Molchanov et al., 2017, 2019).

(c) **Connection sensitivity at initialisation.** SNIP scores connections by their effect on the loss before training (Lee, Ajanthan and Torr, 2019).

(d) **Post-training compression for large models.** Optimal Brain Compression and SparseGPT build on second-order reconstruction for accurate one-shot compression (Frantar and Alistarh, 2022, 2023). Wanda shows that a simple weight-times-activation score is a strong alternative (Sun et al., 2024).

In most of this work, sensitivity is used *directly as the pruning criterion*: it decides which elements to remove or how many. A different use is to provide sensitivity as *guidance to a search procedure* (a feature in the agent's state or a prior over its actions), while the search itself decides the allocation. In that role, sensitivity can help in two distinct ways:

(a) it can make the search more efficient, finding good policies sooner or avoiding destructive ones;

(b) it can lead to a better final model.

These need not coincide: a search may be made faster without being made better at its end point, especially if selection, reward or recovery is the bottleneck. Studies that add sensitivity to a search pipeline usually report only the final accuracy of the combined method. Research Question 2 separates the two effects, measuring exploration quality (rank- and accuracy-based areas under the search trajectory, and the frequency of destructive policies) separately from final accuracy, and comparing a correct sensitivity prior with shuffled and constant controls.

2.9 Search and Policy Selection in Pruning

Any search-based pruning method must decide *which* of the policies encountered during search is returned. The policy an RL agent ends on, the terminal or greedy policy, is not necessarily the best one it saw. The issue is well known in neighbouring fields:

(a) **Neural architecture search.** An evaluation of the NAS search phase found that state-of-the-art NAS algorithms performed, on average, similarly to a random selection policy, and that weight sharing degraded the ranking of candidates. This reveals a gap between the search process and the final selection (Yu et al., 2020).

(b) **Reinforcement learning.** Self-imitation learning reuses the agent's own best past trajectories to steer learning towards them (Oh et al., 2018). Go-Explore keeps an explicit archive of states it has reached and returns to promising ones before exploring further, which overcomes the failure of agents to revisit good states (Ecoffet et al., 2021).

(c) **Pruning.** Evaluating sub-networks is costly, so proxies are needed. EagleEye showed that a fast evaluation step using adaptive batch-normalisation statistics correlates much better with final accuracy than direct evaluation of the pruned weights (Li et al., 2020). In NAS, reduced-training and "zero-cost" proxies are evaluated by how well they preserve the ranking of candidates (Abdelfattah et al., 2021).

Two lessons follow for RL-based pruning: what the agent *retains* can differ substantially from what it *finds*, and the selection step needs validation data separate from the data that drives the search, or it will favour candidates that overfit the search signal. The thesis measures how often the terminal PPO policy is among the top-ranked policies, compared with the best policy PPO visited, introduces archive-based selection with re-ranking on a

held-out selection split (Research Question 3), and documents where this improves validation ranking without improving test accuracy.

2.10 Post-Pruning Recovery and Fine-Tuning

Pruning usually reduces accuracy, which is then partly or wholly recovered by further training. The recovery procedure is not a detail:

(a) **Rewinding.** Weight rewinding and learning-rate rewinding can recover more accuracy than conventional low-learning-rate fine-tuning (Renda, Frankle and Carbin, 2020). Later work traced much of this effect to the learning-rate schedule used during retraining (Le and Hua, 2021).

(b) **Inherited weights.** Some structured-pruning results suggest that the inherited weights matter less than the pruned architecture, since training from scratch can match fine-tuning (Liu et al., 2019).

(c) **Benchmarks.** ShrinkBench identified inconsistent fine-tuning budgets as a major source of incomparable results (Blalock et al., 2020).

(d) **Recovery budgets in large models.** Retraining is costly enough that some methods avoid it entirely (Kwon et al., 2022; Frantar and Alistarh, 2023). Others use a small amount of recovery training (Ma, Fang and Wang, 2023) or substantial continued pre-training after structured pruning (Xia et al., 2024).

(e) **Effects beyond accuracy.** Pruning can affect a model in ways that test accuracy does not reveal: changes in behaviour under distribution shift (Liebenwein et al., 2021), disparate effects across groups (Tran et al., 2022), and transfer performance (Iofinova et al., 2022).

A comparison before recovery measures the immediate damage done by pruning; one after recovery with different budgets partly measures the budgets. Only **identical** recovery isolates the effect of the allocation or criterion. The thesis therefore fixes the recovery protocol across methods (optimiser, learning rate, schedule, epochs, data order) and the pruning mask, and compares methods before and after recovery (Research Question 5).

2.11 Hardware-Aware Neural Network Compression

Hardware-aware methods put a hardware cost, rather than a proxy such as parameter count, into the optimisation.

(a) **Measured or table-based latency.** NetAdapt adapts a network to a target platform using measured latency (Yang et al., 2018). HAQ uses hardware feedback to choose per-layer quantisation precision (Wang et al., 2019). Platform-aware NAS includes measured device latency in the search objective (Tan et al., 2019). Once-for-all trains a single network from which specialised sub-networks are drawn for different hardware constraints (Cai et al., 2020).

(b) **Structured pruning under a latency budget.** HALP formulates pruning as a knapsack problem that maximises retained saliency subject to a latency budget estimated from per-device lookup tables (Shen et al., 2022). SMCP extends this with soft masks that let pruned channels return during training while respecting a cost constraint (Humble et al., 2022).

These methods share a premise, that what is gained on hardware is not captured by parameter counts, and a cost: a hardware model or measurement loop for each target device. Their benefit depends on enough *accuracy-feasible* headroom, meaning cheaper configurations that still meet the accuracy requirement. The thesis treats that headroom as an empirical question to answer before any hardware-aware optimisation (Chapter 6): it measures the latency of every structured configuration in its search space and checks whether accuracy-feasible configurations offer enough room for a hardware-aware RL agent to exploit.

2.12 Parameter Count vs MACs/FLOPs vs Real Latency

Network cost can be described at three related but not interchangeable levels:

(i) **parameter count**, or parameter memory;

(ii) **arithmetic count**, as multiply–accumulate operations (MACs) or floating-point operations (FLOPs);

(iii) **measured runtime** on specific hardware.

(d) **FLOPs are not speed.** ShuffleNet V2 showed early that networks with similar FLOPs can differ markedly in speed, because memory access cost and degree of parallelism also matter (Ma et al., 2018).

(e) **The efficiency misnomer.** Dehghani et al. argued that common cost indicators — parameter count, FLOPs and throughput — are often assumed to be correlated. In fact they can disagree, so reporting only one can give a misleading picture of efficiency (Dehghani et al., 2022).

(f) **Sparsity does not translate into speed.** Unstructured sparsity reduces parameter count but not necessarily runtime, unless the hardware and kernels exploit it (Hoefler et al., 2021). Even structured reductions translate into speed-ups that depend on layer shapes and library kernels (Shen et al., 2022; He and Xiao, 2024).

(g) **Model-level latency studies.** NetAdapt showed that optimising indirect metrics such as MACs or weights need not reduce direct metrics such as latency and energy (Yang et al., 2018). Recent LLM work reports speed-ups only when a sparse inference engine supports the sparsity pattern (Yin et al., 2024).

What is less common is an *exhaustive* measurement within one search space of how well parameter and MAC reductions predict measured latency across *all* configurations a pruning method could choose, showing how often a sparser configuration is actually slower and how much latency headroom exists among configurations that meet an accuracy requirement. Chapter 6 reports such an analysis for all 1,296 structured configurations of two architectures.

2.13 Energy-Efficient and Sustainable Neural Network Inference

The energy cost of machine learning has become a research topic in its own right. Early analyses drew attention to the energy and carbon cost of training large NLP models (Strubell, Ganesh and McCallum, 2019). Researchers were urged to report efficiency alongside accuracy, with the label "Green AI" used for this agenda (Schwartz et al., 2020). Frameworks for the systematic reporting of energy and carbon footprints followed (Henderson et al., 2020). Industrial analyses have discussed the environmental implications of AI across the full model lifecycle (Wu et al., 2022). They have also argued that best practices can substantially reduce training energy and emissions

(Patterson et al., 2022). Carbon estimates for large models depend on lifecycle assumptions as much as on measured energy (Luccioni, Viguier and Ligozat, 2023).

Inference has received growing attention because it is repeated for every query:

(a) Once hardware efficiency gains and consolidated algorithms are taken into account, inference energy has grown much more slowly than parameter-count trends would suggest (Desislavov, Martínez-Plumed and Hernández-Orallo, 2023).

(b) A systematic comparison of inference cost, measured as the energy needed for 1,000 inferences, found multi-purpose generative models to be orders of magnitude more expensive than task-specific models (Luccioni, Jernite and Strubell, 2024).

(c) Benchmarks of LLM inference energy on two GPU generations and multi-GPU configurations show how strongly energy depends on hardware and deployment configuration (Samsi et al., 2023).

(d) Fine-tuning, although cheap per run, contributes to energy use because it is performed so often (Wang et al., 2023).

(e) MLPerf Power defines a methodology for measuring the energy efficiency of ML systems from microwatt devices to data-centre clusters, and emphasises the role of system-level interactions (Tschand et al., 2025).

Energy-aware pruning. Within pruning, energy-aware pruning used an energy model, calibrated from hardware measurements, to guide which layers to prune. It showed that reductions in model size or computation did not necessarily reduce energy (Yang, Chen and Sze, 2017).

Measurement is central and difficult.

(a) **Software power meters.** On CPUs, Intel's Running Average Power Limit (RAPL) interface exposes energy counters for the processor package and its domains without additional hardware. Early experience reports discussed its accuracy and practical pitfalls (Khan et al., 2018). A comparison of software-based power meters distinguished tools that *measure* through hardware interfaces from tools that *estimate* from models such as thermal design power (TDP) (Jay et al., 2023). A recent comparative analysis of RAPL-based measurement mechanisms found that

they differ mainly in usability and robustness, and that RAPL is more accurate than statistical estimation models (Raffin and Trystram, 2025).

(b) **Package scope.** RAPL measures the *package*, not the program. Any other activity in the package (other cores, integrated graphics, uncore components, thermal state) enters the measurement. Stable measurement therefore requires controlled conditions, repeated runs and an explicit stability criterion.

Two distinctions from this literature shape the thesis' treatment of sustainability (Chapter 6):

(a) **Measured versus estimated energy.** The thesis reports only directly measured package energy, never TDP-based or model-based estimates. Where measurement could not be qualified as stable, it reports energy as unresolved.

(b) **Operational energy versus environmental impact.** Inference energy on one processor is only one component of a model's environmental footprint; converting it into carbon or lifecycle impact needs further data and assumptions (Wu et al., 2022; Luccioni, Viguier and Ligozat, 2023). The thesis therefore makes no environmental or carbon claims, and uses "sustainability" only in the narrow, measurable sense of reduced memory, computation and measured operational energy.

2.14 Research Gap

The literature establishes that layer-wise allocation is important, that RL can produce competitive allocations, that sensitivity carries useful information, and that compression metrics do not straightforwardly become deployment efficiency. Six narrower questions remain open.

(i) **Adaptive allocation versus strong deterministic allocation, under controlled conditions.** Layer-specific allocation clearly matters (Lee et al., 2021; Liu et al., 2022; Yin et al., 2024), but learned allocators are usually compared with deterministic ones under different protocols. It is not clear how a learned PPO allocation compares with Uniform, Random, global magnitude, ERK and LAMP when every baseline is built at exactly the sparsity the agent chose. → *Research Question 1.*

(ii) **The separate contributions of sensitivity to exploration and to final accuracy.** Sensitivity is widely used as a pruning criterion (Molchanov et al., 2019; Nonnenmacher et al., 2022; S. Yu et al., 2022). When it is used to *guide* a search, its effect on search efficiency is rarely separated from its effect on final accuracy, or compared with uninformative controls. → *Research Question 2.*

(iii) **Retention of the best policies found during search.** RL-based pruning is usually judged by its final policy (He et al., 2018; Alwani, Wang and Madhavan, 2022; Camci et al., 2022; Ganjdanesh, Gao and Huang, 2024). Whether the optimiser actually retains the best policies it encounters, a failure mode documented in neighbouring fields (Yu et al., 2020; Ecoffet et al., 2021), has not been examined systematically for pruning. → *Research Question 3.*

(iv) **Hidden incentives in pruning rewards.** Pruning rewards combine accuracy and sparsity with hand-chosen weights. RL research shows that more capable optimisers exploit misspecified rewards more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022), and constrained formulations exist for this reason (Achiam et al., 2017; Gu et al., 2024). Whether better pruning search exposes such incentives, and whether an explicit accuracy constraint controls them, has not been tested directly. → *Research Question 4.*

(v) **Comparisons before equal recovery.** Recovery procedures strongly affect pruned accuracy (Blalock et al., 2020; Renda, Frankle and Carbin, 2020; Le and Hua, 2021). Differences between pruning methods measured before recovery, or under different recovery budgets, may therefore overstate the differences that remain after identical recovery. → *Research Question 5.*

(vi) **Compression metrics versus deployment efficiency.** Parameter sparsity, MACs, latency and energy can disagree (Yang, Chen and Sze, 2017; Ma et al., 2018; Yang et al., 2018; Dehghani et al., 2022). How strongly parameter and MAC reductions predict measured CPU latency across a whole pruning search space, how much accuracy-feasible latency headroom exists, and whether structured realisation of a learned allocation reduces *directly measured* energy without material accuracy loss all remain empirical questions. Energy measurement on general-purpose CPUs adds its own reliability problem (Khan et al., 2018; Raffin and Trystram, 2025). → *Research Question 6.*

The thesis addresses these questions in sequence, each answer exposing the next problem. Chapter 3 describes the framework that emerged, indicating which components were part of the original design and which were introduced in response to diagnostic results.

CHAPTER 3
PROPOSED FRAMEWORK

3.1 General

This chapter describes the methodology developed in this thesis: a reinforcement learning (RL) framework that learns layer-wise pruning allocations with Proximal Policy Optimisation (PPO), guided by layer sensitivity.

The framework was built across pre-registered experimental phases, and several components were added because a diagnostic analysis exposed a specific weakness of an earlier version. The chapter therefore presents it **chronologically**, each component with the phase in which it entered and the problem it addressed (Table 3.1; Figure 3.1). It describes *what* each component does and *why*, not how well it performed: outcomes are in Chapters 5 and 6, and datasets, splits, seeds, statistics and measurement procedures in Chapter 4.

Table 3.1: Development of the framework across phases. "Search" components affect which pruning policy is found or selected; "evaluation" components affect how a selected policy is assessed.

Component	Phase introduced	Role	Motivation
Layer-wise pruning units, discrete ratio actions, PPO agent	initial design (frozen in Phase 3)	search	learn a non-uniform allocation instead of one uniform ratio
Accuracy–sparsity reward R0	initial design (frozen in Phase 3)	search	reward retained accuracy and compression jointly
Normalised layer sensitivity	initial design (frozen in Phase 3)	search (guidance)	identify layers whose pruning is most damaging
Soft sensitivity prior on action logits	initial design; evaluated in Phase 3	search (guidance)	bias the agent towards gentler pruning of sensitive layers
Centred sensitivity prior	Phase 4	search (guidance)	remove the shared "prune less" pressure, keeping only *relative* sensitivity
Elite behavioural cloning (Phase 4 only)	Phase 4	search	steer PPO towards the best policies it has visited; dropped from Phase 5 on
Archive of visited policies and held-out selection	Phase 4	search (selection)	the terminal PPO policy may be worse than policies visited during search
Accuracy-constrained reward	Phase 5	search	the accuracy–sparsity trade-off reward encouraged excessive pruning
Constraint-aware archive selection	Phase 5	search (selection)	choose the most compressed candidate that meets the accuracy constraint on held-out data
Equal fixed-mask recovery	Phase 6	evaluation	compare methods after identical fine-tuning
Physical structured realisation; memory, MAC, latency and energy measurement	Paper 3, Phase 7A	evaluation (extension)	test whether compression becomes deployment efficiency
Deployment-oriented structured PPO	Phase 7B	search (extension)	learn directly in the structured deployment space

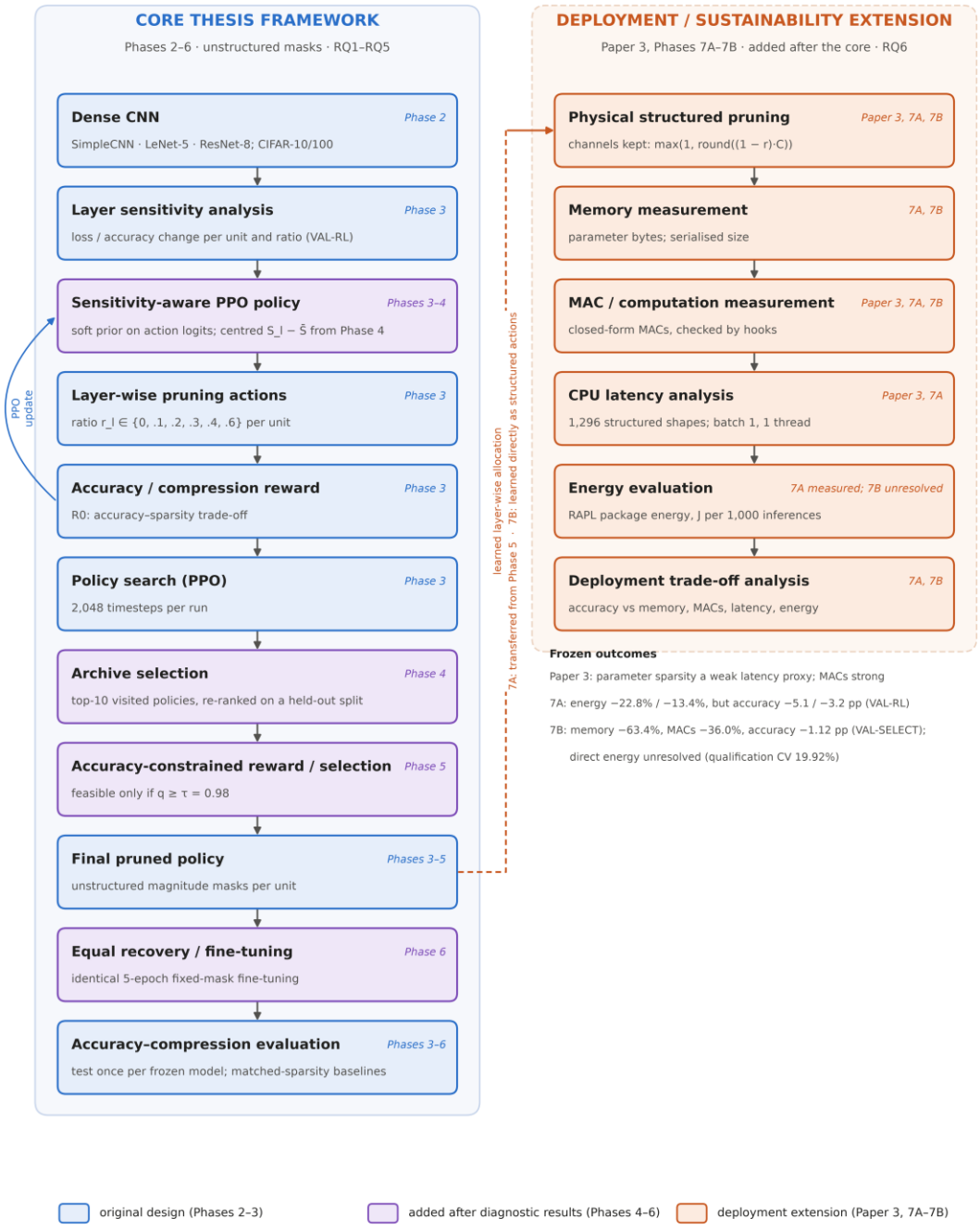


Fig. 3.1: Overall thesis framework. Left: the core framework (Phases 2–6, unstructured masks); each component is tagged with the phase that introduced it (blue: original design; purple: added after diagnostic results). Right: the deployment and sustainability extension (Paper 3, Phases 7A–7B), added later. Dashed arrow: the learned allocation is transferred in Phase 7A and learned directly as structured actions in Phase 7B.

Table 3.2 lists the notation used throughout the thesis.

Table 3.2: Notation.

Symbol	Meaning
M	trained dense network (the reference model)
N_L	number of pruning units ($N_L = 4$ for every architecture in this thesis)
l	index of a pruning unit, $l = 1, \ldots, N_L$
$\boldsymbol{\mathcal{R}}$	ratio set $\{0, 0.1, 0.2, 0.3, 0.4, 0.6\}$
a_l	action at unit l, $a_l \in \{0, \ldots, 5\}$
$r(a)$	pruning ratio of action a: $r(0..5) = 0, 0.1, 0.2, 0.3, 0.4, 0.6$
r_l	pruning ratio assigned to unit l, $r_l = r(a_l)$
$\mathbf{r}$	pruning policy (allocation), $\mathbf{r} = [r_1, \ldots, r_{NL}]$; equivalently the action vector $\mathbf{a}$
$M_{\mathbf{r}}$	network M pruned by policy $\mathbf{r}$
s_l	state observed by the agent before deciding unit l
$\pi_\theta(a \mid s)$	PPO policy with parameters θ
S_l	normalised sensitivity of unit l, $S_l \in [0, 1]$
$\bar{S}$	mean sensitivity over units, $\bar{S} = (1/N_L)\ \Sigma_l\ S_l$
$\tilde{S}_l$	centred sensitivity, $\tilde{S}_l = S_l - \bar{S}$
c_l	prior coefficient of unit l (P1: $c_l = 0.5 \cdot S_l$; centred prior: $c_l = 1.0 \cdot \tilde{S}_l$)
A_b	accuracy of the dense reference on the reward split (VAL-RL)
A_p	accuracy of the pruned network $M_{\mathbf{r}}$ on the same split
q	retained accuracy, $q = A_p / A_b$
ρ	sparsity: percentage of zero weights over all convolutional and fully connected weights
D	number of distinct actions in $\mathbf{a}$
R	episode reward
τ	retained-accuracy threshold of the constrained reward ($\tau = 0.98$ in Phase 5)
$\mathcal{A}$	archive of unique visited policies
C_l	number of output channels (or neurons) of unit l before structured pruning
k_l	number of channels kept by structured pruning of unit l
MAC($\mathbf{r}$)	multiply–accumulate operations per image of the pruned network
T($\mathbf{r}$)	measured batch-1 CPU latency per inference (ms)
E($\mathbf{r}$)	measured CPU package energy per 1,000 inferences (J)
$\hat{E}(\mathbf{r})$	surrogate (predicted) energy per 1,000 inferences, used only inside the Phase-7B search

The letter L is deliberately not used on its own. N_L is the number of pruning units and T is latency.

3.2 Problem Formulation

Let M be a trained convolutional network whose prunable convolutional and fully connected layers are grouped into N_L **pruning units**. A pruning policy assigns each unit a pruning ratio:

$$\mathbf{r} = [r_1, r_2, \dots, r_{N_L}], r_l \in \mathcal{R} = \{0,0.1,0.2,0.3,0.4,0.6\}. \quad (3.1)$$

Applying **r** to M yields the pruned network $M_{\mathbf{r}}$. The uniform policy, $r_1 = \dots = r_{NL}$, is one point in this space. The framework searches for **non-uniform** allocations, because units differ in weight share, computation, sensitivity to the same pruning fraction, and downstream effect on later units.

Compression is measured by the sparsity ρ: the percentage of zero-valued weights over **all** convolutional and fully connected weight tensors, including the output classifier, which is never pruned:

$$\rho(M_{\mathbf{r}}) = 100 \cdot \frac{\sum_t \#\{w \in t : w = 0\}}{\sum_t |t|}, \quad (3.2)$$

where t ranges over every Conv2d and Linear weight tensor. Biases and batch-normalisation parameters are not counted and are never pruned.

The goal is conceptual, not a guaranteed optimum: a layer-wise policy **r** that balances predictive performance and compression as specified by the reward and constraints in force. Formally, the agent maximises

$$\max_\theta \mathbb{E}_{\mathbf{a}\sim\pi_\theta}[R(M_{\mathbf{r(a)}})], \quad (3.3)$$

where R encodes the trade-off (Sections 3.7 and 3.13). Equation (3.3) defines what PPO optimises; it does not claim that PPO finds the best policy. The space is searched stochastically, the returned policy is chosen by a separate selection rule (Sections 3.11–3.13), and enumeration (Section 3.6) shows how close it comes to the best one.

3.3 Layer-Wise Pruning Representation

Pruning units. Each architecture registers exactly four pruning units (Table 3.3), so every setting has the same action space. A unit consists of one or more weight tensors, and an action applies the same ratio to every tensor in the unit. The output classifier is excluded in every architecture.

Table 3.3: Pruning units of the core framework (unstructured).

Architecture	Unit 1	Unit 2	Unit 3	Unit 4	Never pruned
SimpleCNN	conv1 (`features.0`)	conv2 (`features.3`)	conv3 (`features.6`)	fc1 (`classifier.1`)	output layer (`classifier.3`)
LeNet-5	conv1	conv2	fc1	fc2	output layer
ResNet-8	stem convolution	stage 1 (both block convolutions)	stage 2 (both block convolutions and the 1×1 shortcut)	stage 3 (both block convolutions and the 1×1 shortcut)	output layer

Within-unit pruning (core framework). In Phases 3–6, a ratio r_l is applied as **L1 unstructured** pruning to every tensor of unit l: the round($r_l \cdot |t|$) smallest-magnitude weights of each tensor t are masked to zero, using PyTorch's pruning reparameterisation (Paszke et al., 2019). Shapes are unchanged and the mask can be frozen during fine-tuning (Section 3.15), so every pruned network is a masked copy of the same dense network and the whole policy space can be evaluated once and cached for the exhaustive diagnostics.

Same actions, different structural meaning. In SimpleCNN a unit is a single layer; in ResNet-8 it is a residual stage, so one action prunes several tensors at once. When the deployment extension realises policies as physically smaller networks (Section 3.16), the ResNet-8 units are redefined around the residual coupling, so the two definitions are not interchangeable.

3.4 Reinforcement Learning Formulation

Choosing **r** is represented as a **finite-horizon sequential decision process** with one decision per pruning unit. An episode visits the units in their fixed order $l = 1, \ldots, N_L$ (Algorithm 3.1). At step l:

(i) the agent observes a state s_l describing unit l and the decisions taken so far (Section 3.5);

(ii) it samples an action $a_l \sim \pi_\theta(\cdot \mid s_l)$ from the discrete action set (Section 3.6);

(iii) the environment applies ratio $r(a_l)$ to unit l of the working copy of the network;

(iv) the next state s_{l+1} is formed from unit l + 1 and the updated history.

After the last unit ($N_L = 4$ steps), the pruned network M_r is evaluated on the reward split and the episode reward R is paid. All intermediate rewards are zero, and each episode starts again from an unpruned copy of M.

Transitions are deterministic given the actions, and the terminal reward depends only on the complete action vector. The state summarises the current unit and the cumulative pruning, not the full history; the thesis does not claim it is a sufficient statistic, so the process is not claimed to be strictly Markov in the state.

Why RL rather than enumeration? The small space is enumerated only to *evaluate* the search (Section 3.6). The same agent and reward apply unchanged to spaces far too large to enumerate and to non-differentiable objectives such as measured hardware cost; the small settings are a laboratory in which the search can be measured against ground truth.

3.5 State Representation

Core framework (Phases 3–6). The state is the seven-feature vector of the historical environment (`src/rl`, `unit_state`), computed for the current unit l of N_L units:

$$s_l = [\frac{l-1}{N_L - 1}, \log(1 + n_l), \frac{n_l}{\max_j n_j}, |w_l|, \sigma(w_l), \sum_{j<l} r_j, \sigma_l^{\text{slot}}], \quad (3.4)$$

where n_l is the number of weights in unit l, mean $|w_l|$ and $\sigma(w_l)$ are the mean absolute value and the standard deviation of its current weights, $\Sigma_{j<l}\, r_j$ is the cumulative pruning ratio chosen so far, and σ^slot_l is a sensitivity slot.

The sensitivity slot was set to 0 in every condition of Phases 3, 4 and 5, as pre-registered, so in the core framework sensitivity never entered PPO through its *observation*, only through the soft action prior (Sections 3.9–3.10). After the last unit the state is a zero vector.

Deployment extension (Phase 7B). The deployment-oriented structured PPO of Phase 7B (Section 3.16.3) uses a different, eight-feature state, which does include centred sensitivity. The two state definitions are listed separately in Table 3.4 and must not be mixed.

Table 3.4: State features by phase.

#	State feature	Meaning	Phases used
1	$(l-1)/(N_L-1)$	position of the current unit	3, 4, 5
2	$\log(1+n_l)$	log weight count of the unit	3, 4, 5
3	$n_l / \max_j\, n_j$	relative weight count	3, 4, 5
4	mean $\lvert w_l \rvert$	mean weight magnitude (current, masked weights)	3, 4, 5
5	$\sigma(w_l)$	weight standard deviation	3, 4, 5
6	$\Sigma_{j<l}\, r_j$	cumulative pruning ratio so far	3, 4, 5
7	sensitivity slot	**set to 0 in all Phase 3–5 conditions**	3, 4, 5 (constant 0)
1′	$(l-1)/(N_L-1)$	position of the current unit	7B
2′	$\log(1+C_l)$	log channel count of the unit	7B
3′	unit share of dense parameters	parameter contribution	7B
4′	unit share of dense MACs	computational contribution	7B
5′	$\tilde{S}_l$	centred structured sensitivity	7B
6′	r_{l-1}	ratio chosen for the previous unit	7B
7′	$\text{mean}_{j<l}\, r_j$	mean ratio chosen so far	7B
8′	$\hat{E}$(partial policy) / $\hat{E}$(dense)	predicted energy of the policy so far (remaining units at ratio 0), relative to dense	7B

3.6 Action Space

At each unit the agent chooses one of six actions, which map to pruning ratios:

$$r(a) \in \{0, 0.1, 0.2, 0.3, 0.4, 0.6\}, a \in \{0,1,2,3,4,5\}. \quad (3.5)$$

That is, 0%, 10%, 20%, 30%, 40% or 60% of the unit's weights (core framework) or channels (structured extension). All actions are always available; no action is masked. A policy is the sequence of the four unit actions, so the number of distinct policies is

$$|\mathcal{R}|^{N_L} = 6^4 = 1{,}296 \quad (3.6)$$

in every setting. The highest single ratio is 0.6, so even the most aggressive policy [5, 5, 5, 5] leaves 40% of every pruned tensor. The excluded classifier is never pruned, which caps the achievable total sparsity ρ of each architecture below 60%.

The small space is deliberate: all 1,296 policies can be evaluated once per setting, giving an exact reward landscape that shows how good the policies PPO *visits* and *returns* are, and where the reward's own optimum lies.

The same count of 1,296 holds for every architecture, but the policies do not correspond across architectures. In SimpleCNN, an action on unit 2 masks the selected fraction of the weights of a single convolution. For ResNet-8, an action is applied to a residual-stage weight group, where the selected pruning ratio masks the corresponding fraction of weights across the convolutional layers associated with that group; the residual stage itself is not structurally removed (Table 3.3).

3.7 Initial Reward Function

The initial reward R0 was inherited from the original framework and frozen in Phase 3. It is paid at the end of each episode:

$$R_0 = 1.5 \cdot \frac{A_p}{A_b} + \lambda_s \cdot \rho + 0.05 \cdot D, \lambda_s = 0.01. \qquad (3.7)$$

Here A_p is the accuracy (%) of M_r on the reward split (VAL-RL); A_b is the accuracy (%) of the dense reference on the same split, fixed per setting, so A_p/A_b is the retained accuracy q; ρ is the sparsity in **percent** as defined in Equation (3.2) (global, parameter-based, over all Conv2d/Linear weights including the classifier); and D is the number of distinct actions in the episode's action vector (1 to 4). The weights are 1.5 on retained accuracy, $\lambda_s = 0.01$ per percentage point of sparsity, and a small bonus of 0.05 per distinct action. The coefficients are fixed and were not tuned in any phase.

The diversity term, part of the original design, is a mild incentive against trivially uniform policies. For reproducibility: the original notebook used $\lambda_s = 0.04$, which the library default still carries; every frozen phase (Phases 3, 4 and the R0 control in Phase 5) uses $\lambda_s = 0.01$, as recorded in the Phase-4 pre-registration.

Equation (3.7) trades accuracy against sparsity at a fixed exchange rate, with no threshold below which accuracy loss becomes unacceptable. Section 3.13 returns to this point.

3.8 Layer Sensitivity Analysis

The sensitivity of a unit measures how much the network's loss increases when that unit alone is pruned. It is computed once per setting on the dense reference model, using only the reward split (VAL-RL), never the selection split or the test set. The procedure (`src/sensitivity`, `loss_sensitivity`) is:

(i) **Baseline loss.** Evaluate the dense model's mean cross-entropy loss L_0 on VAL-RL.

(ii) **Single-unit probes.** For each unit l and each probe ratio r ∈ {0.1, 0.2, 0.4, 0.6}, prune *only* unit l by r (L1 unstructured, as in Section 3.3), leaving every other unit dense. Evaluate the loss $L_{l,r}$.

(iii) **Raw sensitivity.** Average the relative loss increases over the probe ratios:

$$S_l^{\text{raw}} = \frac{1}{4} \sum_{r \in \{0.1,0.2,0.4,0.6\}} \frac{L_{l,r} - L_0}{L_0}. \tag{3.8}$$

(iv) **Normalisation.** Min–max normalise across the units of the network:

$$S_l = \frac{S_l^{\text{raw}} - \min_j S_j^{\text{raw}}}{\max_j S_j^{\text{raw}} - \min_j S_j^{\text{raw}}} \in [0,1]. \tag{3.9}$$

The most sensitive unit has $S_l = 1$ and the least sensitive $S_l = 0$; the normalisation makes sensitivities comparable across architectures but discards their absolute scale. Loss rather than accuracy registers small degradations that do not yet change predictions.

The vector was frozen on the configuration-I VAL-RL split for Phases 3–4, recomputed on the configuration-II split in Phase 5, and computed with each unit pruned *structurally* (whole channels removed; Section 3.16) in Phase 7B (Section 4.10). In every case only VAL-RL data are used. Because the probes measure each unit in isolation and miss interactions between units, sensitivity is treated as *guidance* for the search, not as a substitute for it.

3.9 Sensitivity-Guided Action Prior

In the initial design, sensitivity guides the agent through a **soft prior on the action logits**. Let $z_l(a)$ be the logit that the PPO actor assigns to action a at unit l. Before the action distribution is formed, the logits are shifted:

$$\tilde{z}_l(a) = z_l(a) - c_l \cdot \frac{r(a)}{0.6}, \pi_\theta(a \mid s_l) = \mathrm{softmax}_a(\tilde{z}_l(a)). \quad (3.10)$$

Here c_l is the prior coefficient of unit l, and r(a)/0.6 rescales the ratio so that the most aggressive action has weight 1. In Phase 3, $c_l = \beta \cdot S_l$ with $\beta = 0.5$ (condition P1, "correct prior"). The implementation (`src/rl`, `SoftPriorPolicy`) subtracts the pre-computed matrix $B[l, a] = c_l \cdot r(a)/0.6$ from the logits in every forward pass, both when actions are sampled and when log-probabilities are evaluated during the PPO update.

The penalty grows with the unit's sensitivity and the action's aggressiveness, and action 0 (no pruning) is never penalised. The prior **shifts preferences but does not replace the learned policy**: every action keeps a non-zero probability, the actor can learn to override it, and with $c_l = 0$ the policy is exactly standard PPO.

Phase 3 tested this prior against three controls of the same strength $\beta = 0.5$: P0, no prior; P2, a shuffled prior in which each unit receives another unit's sensitivity; and P3, a constant prior in which every unit receives the mean sensitivity. The controls separate the effect of *layer-specific* information from that of a generic bias towards lighter pruning.

3.10 Centred Sensitivity Prior

Since every $S_l \geq 0$, the total penalty $\Sigma_l c_l = \beta \cdot \Sigma_l S_l$ of the Section 3.9 prior is positive, so the correct, shuffled and constant priors carry the **same total conservatism**. Any benefit they share could come from this common "prune less" pressure rather than from information about *which* units are sensitive.

Phase 4 therefore introduced a **centred** prior that keeps only relative sensitivity:

$$\tilde{S}_l = S_l - \overline{S}, \overline{S} = \frac{1}{N_L}\sum_{j=1}^{N_L} S_j\,, c_l = 1.0 \cdot \tilde{S}_l, \qquad (3.11)$$

used in Equation (3.10) in place of $c_l = 0.5 \cdot S_l$. Its coefficients sum to zero ($\Sigma_l\, c_l = 0$), so the prior carries no net conservatism: aggressive actions are penalised on units more sensitive than average ($\tilde{S}_l > 0$) and favoured on less sensitive ones ($\tilde{S}_l < 0$), while action 0 is unchanged. Pruning is redistributed from sensitive to insensitive units instead of being reduced everywhere. This centred prior (condition "P2" in the Phase-4 and Phase-5 records) became the standard prior of the core framework from Phase 4 onward.

Phase 4 evaluated it against the Phase-3 prior, a centred prior permuted across units, a magnitude-matched pure-conservatism prior (c_l = mean_j $|\tilde{S}_j|$ for every unit) and a sign-reversed prior ($c_l = -\tilde{S}_l$) (Section 4.10). The sensitivity state slot remained 0 (Section 3.5), so sensitivity acts on the search only through this prior.

3.11 Policy-Retention Problem

The initial design returns the agent's **terminal policy**: after training, the deterministic policy a_l = argmax_a $\pi_\theta(a \mid s_l)$ is rolled out once, and the resulting action vector is the selected pruning policy.

During training PPO samples hundreds of complete policies, but the distribution it converges to need not favour the best of them: a stochastic policy-gradient method updates on noisy reward estimates and can sample an excellent policy early, then drift towards a region of high but not maximal average reward. Returning only the terminal policy discards evaluations that have already been paid for. With exact landscapes (Section 3.6), the problem is the gap between the rank of the **best policy sampled** and the rank of the **policy finally returned**; when the first is consistently better, the limitation lies in *retention and selection*, not exploration. Phase 4 identified this gap and introduced archive-based selection to address it.

3.12 Archive-Based Policy Selection

3.12.1 Archive (Phase 4)

During each PPO run, an **archive** $\mathcal{A}$ records every unique policy sampled in a training episode, with its action vector **a**, its reward on the reward split (VAL-RL), which is the reward the agent was trained on, and the episode in which it was first sampled. A policy sampled again keeps its first entry. Recording is read-only: it does not change the training process and draws no random numbers.

3.12.2 Selection rule (Phase 4)

At the end of training, the archive is reduced to a candidate set and one candidate is selected on held-out data:

$$\mathcal{C} = \text{top}_{10}\{\mathbf{a} \in \mathcal{A} \text{ by } R_{\text{VAL-RL}}(\mathbf{a})\}, \mathbf{a}^{\star} = \text{argmax}_{\mathbf{a}\in\mathcal{C}} R_{\text{VAL-SELECT}}(\mathbf{a}). \quad (3.12)$$

$\mathcal{C}$ holds the 10 unique archived policies with the highest VAL-RL reward (ties: earlier discovery), and $\mathbf{a}\star$ is the candidate scoring highest under the same reward on the **selection split** (VAL-SELECT; ties: higher VAL-RL reward, then earlier discovery). VAL-SELECT is disjoint from the reward split and never used during training (Chapter 4).

The two-stage rule avoids relying only on the final policy (Section 3.11) and only on the best VAL-RL reward, which would favour candidates whose reward is inflated by noise in the reward split.

Archive selection is a retention and selection mechanism based on validation evidence. It decides which visited policy is returned. It does not change what the agent learns, and its correctness is defined relative to the validation splits, not the test set.

3.12.3 Elite behavioural cloning (Phase 4 only)

Phase 4 also tested a training-time complement to the archive, using the best policies seen so far as additional supervised targets for the PPO actor:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \eta\mathcal{L}_{\text{elite}}, \mathcal{L}_{\text{elite}} = -\frac{1}{4K}\sum_{e=1}^{K}\sum_{l=1}^{4}\log\pi_\theta(a_l^{(e)} \mid s_l^{(e)}), \quad (3.13)$$

where (s^(e)l, a^(e)l) are the state–action pairs of the K = 5 best unique VAL-RL-reward policies seen so far, η = 0.1, and the term is active once at least five unique policies have been seen. The Phase-4 final pipeline combined it with the archive rule (condition S2A); Phase 5 removed it and kept the archive, so the mature core framework does not include Equation (3.13).

3.12.4 Constraint-aware selection (Phase 5)

With the accuracy-constrained reward (Section 3.13) the selection rule changed with it. The candidates are still the top 10 archived policies by VAL-RL reward, and the archive of all runs is frozen and hashed *before* any VAL-SELECT evaluation. The selected policy is

$$\mathbf{a}^\star = \text{argmax}_{\mathbf{a}\in\mathcal{C}}\{\rho(\mathbf{a}): q_{\text{VAL-SELECT}}(\mathbf{a}) \geq \tau\}, \quad (3.14)$$

that is, the **most compressed candidate whose retained accuracy on the selection split meets the threshold**. Ties go to higher VAL-SELECT accuracy, then higher VAL-RL reward, then earlier discovery. If no candidate meets the threshold, the candidate with the highest $q_{\text{VAL-SELECT}}$ is chosen and the fallback is reported. Training rewards compression inside the feasible region; selection then confirms feasibility on held-out data.

3.13 Reward Misalignment and Accuracy-Constrained Reward

3.13.1 Why R0 can encourage excessive pruning

Equation (3.7) converts accuracy and sparsity into one scalar at a fixed rate: an extra percentage point of sparsity is worth 0.01, and an extra percentage point of retained accuracy 0.015. A policy that gives up some accuracy for enough sparsity can therefore score *higher* than a gentler policy, however damaging the accuracy loss is in practice. R0 has no region in which accuracy loss becomes unacceptable, so in fragile architectures the reward-optimal policy can be one a practitioner would reject.

A weak optimiser hides this problem; once search and retention improve (Sections 3.10–3.12), the agent finds exactly what the reward specifies, a pruning instance of the observation that more capable optimisers exploit misspecified rewards more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022). Phase 4 also tested alternative trade-off rewards, with sparsity targets and normalised axes (Section 4.12). None met its pre-registered criterion, so R0 was retained for Phase 4, and Phase 5 redesigned the reward around an explicit constraint.

3.13.2 Accuracy-constrained reward (Phase 5)

The Phase-5 reward is constraint-first. With q = A_p/A_b on VAL-RL and the sparsity written as a fraction (ρ/100):

$$R_c = \begin{cases} 1 + \dfrac{\rho}{100} + \varepsilon q, & q \geq \tau, \\ -\kappa(\tau - q), & q < \tau, \end{cases} \quad \tau = 0.98, \varepsilon = 0.01, \kappa = 10. \tag{3.15}$$

Here τ = 0.98 is the retained-accuracy threshold: a policy is *feasible* if it keeps at least 98% of the dense VAL-RL accuracy. τ was chosen before any PPO run, by a pre-registered rule on the VAL-RL landscapes (Section 4.9.3); the test set was not used. The offset 1 makes every feasible reward larger than every infeasible one; ρ/100 is the compression reward, paid only inside the feasible region; ε·q with ε = 0.01 is a small tie-breaker that prefers higher accuracy among equally sparse feasible policies; and κ = 10 is

the penalty slope for infeasible policies, proportional to the shortfall below τ. There is no diversity term.

Compression is rewarded only inside the acceptable-accuracy region. Any feasible policy (reward ≥ 1) is worth more than any infeasible one (reward < 0); among infeasible ones, smaller shortfalls are penalised less, which keeps a learning signal pointing back towards feasibility. The reward-optimal policy is the most compressed feasible policy on VAL-RL, matching a stated tolerance of 2% relative accuracy loss on VAL-RL. Two limitations are inherent and examined in Chapter 5: feasibility on VAL-RL does not imply feasibility on unseen data, and τ is a *relative* threshold, not an absolute accuracy bound. The centred prior was kept in Phase 5 as an exploration mechanism, with an otherwise identical prior-free condition to measure its contribution.

3.14 Final Core Framework

Table 3.5 lists the components that survived the diagnostic sequence, separating **search** components, which determine the selected policy, from **evaluation** components, which assess it.

Table 3.5: Mature core framework (as frozen after Phase 5, evaluated in Phase 6).

Stage	Component	Type	Section
1	Dense reference model M, trained once per setting	input	3.2
2	Normalised loss sensitivity S_l on VAL-RL	search (guidance)	3.8
3	PPO over four units with six ratio actions; seven-feature state (sensitivity slot 0)	search	3.4–3.6
4	Centred sensitivity prior, $c_l = 1.0 \cdot \tilde{S}_l$, on the action logits	search (guidance)	3.10
5	Accuracy-constrained reward R_c with $\tau = 0.98$ on VAL-RL	search	3.13
6	Archive of unique visited policies; top-10 candidates by VAL-RL reward	search (retention)	3.12
7	Constraint-aware selection on VAL-SELECT	search (selection)	3.12.4
8	Selected pruning policy $\mathbf{a}^\star$, applied as unstructured masks	output	3.3
9	Equal fixed-mask recovery (5 epochs, identical for every method)	**evaluation**	3.15
10	Test evaluation, once per frozen model; matched-sparsity baselines	**evaluation**	Chapter 4

Relative to the initial version, the prior is centred, the returned policy comes from an archive on held-out data, the reward is constraint-first, and selection enforces the same constraint on held-out data; behavioural cloning was dropped, and the state is unchanged.

Equal recovery (stage 9) is **not part of PPO's decision making**. The agent's reward is always computed on the pruned network *without* fine-tuning. Recovery is applied only after a policy has been selected, identically to every compared method, so that the final comparison reflects the allocation rather than differences in recovery budgets.

3.15 Equal Recovery and Fine-Tuning Protocol

Pruning methods differ in how much accuracy they lose *immediately*, but in deployment a pruned network is normally fine-tuned before use, and recovery can reduce or reverse those differences (Renda, Frankle and Carbin, 2020; Le and Hua, 2021). To compare allocations fairly after recovery, Phase 6 applied one fixed protocol to every compared model (`experiments/phase6/phase6_lib.py`, `PROTOCOL` and `finetune`): Adam at learning rate 1×10^{-4} with cosine annealing to zero over E epochs, stepped once per epoch,

$$\eta_e = \frac{1}{2}\eta_0(1+\cos(\pi e/E)), \eta_0 = 10^{-4}, e = 0, \ldots, E-1, \quad (3.16)$$

with E = 5 epochs fixed in advance by a timing rule. The remaining settings (optimiser constants, batch size, augmentation) are listed in Section 4.13. Four design rules define the protocol:

(a) **Frozen mask.** The pruning mask is frozen with PyTorch's pruning reparameterisation, so masked weights stay exactly zero; no pruned weight may regrow and no new weight may be pruned, which is checked after every epoch.

(b) **Batch normalisation.** Training mode during fine-tuning, so running statistics are updated identically for every method; evaluation mode for every evaluation.

(c) **Shared seeds.** The fine-tuning seed is the policy seed + 100, and the PPO, LAMP and global-magnitude models that form one comparison triple share it, and hence the same data order.

(d) **No checkpoint selection.** The model after the final epoch is used.

The protocol is deliberately modest: it does not maximise any method's recovered accuracy, which would need method-specific tuning, but gives every allocation the same opportunity to recover. The extension reuses it on physically compacted networks, which have no masks to freeze: 5 epochs with seed 600 and one shared data stream in Phase 7A, and 5 epochs for Phase-7B final models, whose accuracy proxy (Section 3.16.3) shortens it to 2 epochs.

3.16 Deployment-Oriented Structured Extension

> **This section describes a later extension**, developed after the core framework (Phases 3–6) was complete, to test whether layer-wise compression becomes real deployment efficiency (Research Question 6). The deployment-oriented structured PPO of Section 3.16.3 is a separate variant evaluated on its own terms, not the final form of the core method.

3.16.1 From masks to physical structure

The core framework's **unstructured** masks keep the dense tensor shapes. On general-purpose CPUs the zeros are still stored and multiplied unless sparse kernels are used, so parameter sparsity need not reduce latency or energy (Hoefler et al., 2021; He and Xiao, 2024).

The extension therefore realises a layer-wise allocation as a **physically smaller dense network**, using structured channel (filter) and neuron removal (He, Zhang and Sun, 2017; Li et al., 2017). For a unit with C_l output channels or neurons and ratio r_l, the number of channels kept is

$$k_l = \max(1, \text{round}((1 - r_l)C_l)), \tag{3.17}$$

and the k_l channels with the largest L1 filter norms are kept (ties go to the lower index). Importance is always computed on the dense model, so the result does not depend on the order in which units are compacted (`src/pruning/structured_units.py`). Removing an output channel also removes the corresponding input of every layer that reads it; the output layer itself is never pruned.

Residual coupling (ResNet-8). In a residual network every layer that writes to or reads from the same residual stream must keep the same channel set. The structured ResNet-8 units therefore differ from the unstructured units of Table 3.3: the stem unit cuts the stage-1 residual width on one shared channel set, and each stage unit removes only block-internal channels (exact definitions in Section 4.6.2). The same action index thus has a different structural meaning in ResNet-8 than in SimpleCNN, and a policy learned for unstructured ResNet-8 units maps onto the structured units only approximately.

3.16.2 Measurement: three levels of deployment efficiency

A compacted network can be assessed at three related but non-interchangeable levels (Table 3.6), used throughout Chapter 6. A result at one level does not establish a result at the next.

Table 3.6: Three levels of deployment efficiency.

Level	Quantity	Definition in this thesis
1. Memory / compression	parameter memory	Σ over parameters of (number of elements × element size) in bytes; serialised checkpoint size as a secondary measure
2. Computation	MACs; latency T(**r**)	multiply–accumulate operations per image of all convolutional and linear layers, from a closed form checked against forward hooks; measured batch-1 CPU latency
3. Operational energy	E(**r**)	CPU package energy per 1,000 inferences, read from the Intel RAPL counters through the Windows Energy Meter Interface; no TDP-based or modelled estimate

The measurement protocols are specified in Sections 4.17–4.18.

3.16.3 Two uses of the extension

(a) Structured transfer of the learned allocation (Phase 7A). The most frequently selected Phase-5 policy has its action indices interpreted as structured ratios (Equation 3.17); the compacted network is recovered with the equal protocol of Section 3.15 and measured (memory, MACs, latency and energy) against a dense model and structured comparators recovered identically. The agent is not retrained, so Phase 7A tests the transfer of an allocation from *masked* to *structured* pruning, not a new search.

(b) Deployment-oriented structured PPO (Phase 7B). Phase 7B redesigns the agent to search directly in the structured space of one architecture (SimpleCNN). Its search reward uses a *surrogate (predicted)* energy saving, not measured energy, so the name does not imply any measured energy result for its models:

(a) **Actions.** Structured ratios per unit (Equation 3.17), so every action vector is a physical channel-width configuration.

(b) **State.** The eight features of Table 3.4 (rows 1′–8′), including centred structured sensitivity $\tilde{S}_l$.

(c) **Prior.** The centred prior of Equation (3.10), with $c_l = 1.0 \cdot \tilde{S}_l$, computed from structured sensitivity.

(d) **Accuracy proxy.** Full recovery of every candidate is too expensive, so feasibility is judged by $\hat{q}$ = A_2/A_2,dense, the VAL-RL accuracy after a 2-epoch version of the recovery protocol relative to the dense model's. The proxy and its threshold $\hat{\tau}$ were validated against 5-epoch recovery before the search (Chapter 4).

(e) **Energy surrogate.** Energy cannot be measured inside the search, so the reward uses a surrogate fitted beforehand to directly measured energies of sampled policies, linear in the policy's measured batch-1 latency T_{table} (Paper 3 latency tables):

$$\widehat{E}(\mathbf{r}) = a + b \cdot T_{\text{table}}(\mathbf{r}), a = -0.117, b = 13.762 (\text{J per 1,000 inferences; } T \text{ in ms}). \quad (3.18)$$

The surrogate makes the search computationally feasible. **It is not a measurement**, and a predicted saving is never reported as an energy result. Any energy claim about a final model requires direct hardware measurement under a qualified protocol (Chapter 6).

(a) **Reward.** Constraint-first, as in Equation (3.15), with the compression term replaced by the predicted energy saving:

$$R_{7B} = \begin{cases} 1 + \alpha \dfrac{\hat{s}(\mathbf{r})}{s_{\max}} + \varepsilon \hat{q}, & \hat{q} \geq \hat{\tau}, \\ -\kappa(\hat{\tau} - \hat{q}), & \hat{q} < \hat{\tau}, \end{cases} \quad \hat{s}(\mathbf{r}) = 1 - \frac{\widehat{E}(\mathbf{r})}{\widehat{E}(\text{dense})}, \quad (3.19)$$

where s_{max} is the largest predicted saving over the 1,296 policies, $\alpha = 4$ was fixed on development seeds before the confirmatory runs, and $\varepsilon = 0.01$ and $\kappa = 10$, as in Phase 5.

(a) **Feasible archive and selection.** Every unique *feasible* policy visited is archived, and the policy with the lowest predicted energy is selected (tie rules in Section 4.11). Confirmatory runs used 20 pre-registered seeds; the primary policy is the one selected most often across seeds.

Phase 7B thus carries sensitivity guidance, a constraint-first reward, archive selection and equal recovery into the deployment setting, adding a proxy and a surrogate only to make the search affordable. Its evaluation criteria are defined in Chapter 4 and its outcomes reported in Chapter 6.

3.17 Overall Algorithm

Algorithm 3.1 summarises the mature core framework and Algorithm 3.2 the deployment extension's evaluation workflow. They are kept separate so that the core framework is not confused with the extension.

Algorithm 3.1 — Sensitivity-aware adaptive pruning with archive selection and accuracy constraint (core framework).

```
Input:  trained dense network M with pruning units 1..N_L; ratio set R = {0, .1, .2, .3,
.4, .6};
        reward split VAL-RL; selection split VAL-SELECT; threshold tau = 0.98;
        PPO hyper-parameters (Table 3.7); episodes E_ep = 512; archive size K_c = 10
Output: selected pruning policy a*; pruned network M_a*

 1  A_b <- accuracy of M on VAL-RL
 2  for each unit l: S_raw[l] <- mean_{r in {.1,.2,.4,.6}} (Loss(M with only unit l
pruned by r) - Loss(M)) / Loss(M)    (3.8)
 3  S <- min-max normalise S_raw; c[l] <- 1.0 * (S[l] - mean(S))
(3.9),(3.11)
 4  initialise PPO actor-critic pi_theta, V_phi; archive Arch <- empty map
 5  for episode = 1 .. E_ep:
 6      M' <- copy of M; actions <- []
 7      for l = 1 .. N_L:
 8          s_l <- state of unit l (seven features; sensitivity slot = 0)
(3.4)
 9          logits <- actor(s_l) - c[l] * r(a)/0.6 for every action a
(3.10)
10          a_l ~ softmax(logits); prune every tensor of unit l of M' by r(a_l) (L1
unstructured); append a_l
11      A_p <- accuracy of M' on VAL-RL; rho <- sparsity of M' (all conv/linear weights)
(3.2)
12      R <- R_c(A_p / A_b, rho; tau)
(3.15)
13      if actions not in Arch: Arch[actions] <- (R, episode)
14      store transitions (rewards 0 at l < N_L, R at l = N_L); every 64 steps: PPO
update (clipped objective)            (3.20)
15  C <- the 10 entries of Arch with the highest R (ties: earlier episode)
(3.12)
16  freeze C; evaluate every candidate on VAL-SELECT
```

```
17  a* <- argmax rho over {a in C : q_VAL-SELECT(a) >= tau}; ties: higher VAL-SELECT
accuracy, higher R, earlier;
        if the set is empty: a* <- argmax q_VAL-SELECT (reported fallback)
(3.14)
18  M_a* <- M pruned by a*
19  [evaluation] recover M_a* with the equal fixed-mask protocol (5 epochs; Section
3.15), identically for every method
20  [evaluation] evaluate on the test set once, after the model is frozen
```

Steps 1–18 are search; steps 19–20 are evaluation and do not influence the selected policy. The earlier versions are obtained by substitution: Phase 3 used $c_l = 0.5 \cdot S_l$, R0 (Equation 3.7) and the terminal deterministic policy (steps 15–17 replaced); Phase 4 used $c_l = 1.0 \cdot \tilde{S}_l$, R0 and Equation (3.12), with behavioural cloning (Equation 3.13) added to the PPO loss.

Algorithm 3.2 — Deployment-oriented structured evaluation (extension; Phases 7A–7B).

```
Input:  dense network M; a structured policy a (7A: transferred from the core framework;
        7B: selected by the deployment-oriented structured PPO); comparator policies;
recovery protocol
Output: accuracy, parameter memory, MACs, latency and measured energy of each model

 1  for each model (dense, a, comparators):
 2      k_l <- max(1, round((1 - r(a_l)) * C_l)) for every unit; keep the k_l channels
with largest L1 norm          (3.17)
 3      build the physically compacted network; propagate removed channels to the layers
that read them
        (ResNet-8: shared channel set across the residual stream)
 4      recover with the equal protocol (5 epochs, identical seed and data stream);
record VAL-RL / VAL-SELECT accuracy
 5      Level 1: parameter memory (bytes) and serialised size
 6      Level 2: MACs (closed form, hook-checked); batch-1 CPU latency
 7  qualify the energy instrument on the dense model (pre-registered stability
criterion)
 8  if qualified: Level 3: paired sessions of package energy per 1,000 inferences, dense
anchor in every session
    else: report energy as unresolved (no estimate is substituted)
```

Table 3.7: PPO configuration (identical in Phases 3, 4, 5 and 7B).

Setting	Value
Implementation	Stable-Baselines3 PPO (Raffin et al., 2021)
Actor and critic	MLP with two hidden layers of 64 units, tanh activations, orthogonal initialisation
Optimiser	Adam, learning rate 3×10^{-4}
Rollout length / minibatch / epochs per update	n_{steps} = 64, batch size 32, 10 epochs
Discount γ / GAE λ	0.99 / 0.95
Clipping range ϵ	0.2
Entropy coefficient / value coefficient / gradient clipping	0.01 / 0.5 / 0.5
Training budget	2,048 timesteps = 512 episodes of 4 steps per run
Threads	1 per run

PPO objective. PPO (Schulman et al., 2017) updates the actor by maximising the clipped surrogate objective, with advantages estimated by generalised advantage estimation (Schulman et al., 2016):

$$\mathcal{L}^{\mathrm{PPO}}(\theta) = \mathbb{E}_t[\min(\kappa_t(\theta)\widehat{A}_t, \mathrm{clip}(\kappa_t(\theta),1-\epsilon,1+\epsilon)\widehat{A}_t)] - c_v\mathbb{E}_t[(V_\phi(s_t)-\widehat{R}_t)^2] + c_e\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot\mid s_t))], \tag{3.20}$$

where $\kappa_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_\theta\mathrm{old}(a_t \mid s_t)$ is the probability ratio, $\hat{A}_t$ the advantage estimate, $\hat{R}_t$ the return target, $\boldsymbol{\mathcal{H}}$ the policy entropy, and $\epsilon = 0.2$, $c_v = 0.5$ and $c_e = 0.01$ (Table 3.7). The sensitivity prior enters through π_θ, since the shifted logits of Equation (3.10) define the distribution in both κ_t and $\boldsymbol{\mathcal{H}}$. The behavioural-cloning term of Equation (3.13) is added to the loss only in the Phase-4 S2 runs.

3.18 Computational Workflow and Reproducibility

Three design choices make the framework measurable and reproducible (implementation in Chapter 4):

(i) **Exhaustive landscapes.** All 1,296 policies of every setting are evaluated once on the reward and selection splits. PPO's terminal evaluations are read from this cache, which returns exactly the live value and consumes the random stream as an

uncached run does, so each run is an *auditable search* in which every visited or returned policy has a known rank.

(ii) **Separated data roles.** Training data update weights only; VAL-RL drives sensitivity, reward and training; VAL-SELECT only chooses among frozen archive candidates; the test set is read once per frozen, hashed model. The archive is frozen before any selection-split evaluation, and the final policy list before any test evaluation.

(iii) **Pre-registered configurations.** Reward constants, prior coefficients, thresholds, seeds, selection rules, the recovery protocol and decision criteria were fixed before each phase's confirmatory runs. No constant was tuned on test data, no threshold was changed after results were seen, and the library was verified to reproduce the historical environment bit for bit (Phase 2).

Computational cost. A run needs 512 episodes (each one cached pruned-network evaluation), 64-step PPO updates and, in the core framework, one VAL-SELECT evaluation of 10 candidates; equal recovery costs 5 epochs per model. The extension adds structured compaction, hardware measurement and, in Phase 7B, a 2-epoch proxy landscape of all 1,296 structured policies. This cost, compared with deterministic methods that need no search, is discussed in Chapter 7.

3.19 Chapter Summary

A PPO agent assigns one of six pruning ratios to each of four units, with sensitivity entering only as a soft prior on the action logits, never as an active state feature. Phase 4 centred the prior and added archive selection on held-out data; Phase 5 replaced the fixed-rate reward R0 with a constraint-first reward that pays for compression only when at least 98% of the dense accuracy is retained; Phase 6 fixed an equal, mask-preserving recovery protocol. Separately and later, the deployment extension realises allocations as physically smaller networks evaluated for memory, computation and measured energy: Phase 7A transfers the core allocation, and Phase 7B's deployment-oriented structured PPO searches the structured space directly, using a validated accuracy proxy and a latency-based energy surrogate, neither of which is treated as a measurement.

Chapter 4 specifies the experimental setup used to evaluate each component, and Chapters 5 and 6 report the results.

CHAPTER 4
EXPERIMENTAL SETUP

4.1 Experimental Design Overview

This chapter specifies how the framework of Chapter 3 was evaluated, in enough detail for every experiment to be reproduced and audited; it reports no results. Every phase committed its conditions, seeds, metrics, tests and decision criteria before any confirmatory run, treating earlier phases as frozen evidence verified by hash. Phase 1 audited an earlier pipeline whose test set had been used for reward, sensitivity and epoch selection, and labelled it superseded; all reported experiments follow the leakage-guarded protocol adopted after it (Section 4.3). Phase 2 fixed the splits, trained the dense references and verified the shared library. The **core framework** (Phases 3–6) uses mask-based pruning on six architecture–dataset settings on a desktop CPU; the **deployment extension** (Paper 3, Phases 7A and 7B, an energy confirmation study) uses physically compacted networks measured on a laptop CPU.

4.2 Datasets

The experiments use CIFAR-10 and CIFAR-100 (Krizhevsky, 2009) (Table 4.1), scaled to [0, 1] and normalised per channel with constants from the official 50,000-image training file: means (R, G, B) 0.4914, 0.4822, 0.4465 for CIFAR-10 and 0.5071, 0.4865, 0.4409 for CIFAR-100, and standard deviations 0.2470, 0.2435, 0.2616 and 0.2673, 0.2564, 0.2762. All training uses random 32×32 crops with 4-pixel zero padding and random horizontal flips; evaluation is never augmented. CIFAR-100 is used only in Phases 2–6 and the Paper 3 latency tables.

4.3 Dataset Partitioning and Validation Protocol

4.3.1 Training, validation and test

For both datasets, a fixed permutation of the 50,000 training indices (`torch.randperm`, seed 42) assigns 45,000 images to **training** and the last 5,000 to **validation**; the official 10,000 images are the **test** set. Both datasets use identical index lists, recorded with hashes in split manifests.

4.3.2 Two validation configurations

Validation is split into two class-stratified, disjoint parts: **VAL-RL** (reward split) drives sensitivity, rewards and PPO training, and **VAL-SELECT** (selection split) only chooses among already-produced candidates. The sizes changed once (Table 4.1), as declared before any Phase-5 data existed: **configuration I** (Phases 3–4, 3,000 / 2,000; about 60% of each class to VAL-RL, largest-remainder rounding, per-class permutation with seed 20260928; recorded as V_{RL} and V_{SELECT}) and **configuration II** (Phase 5 onward, 2,500 / 2,500; seed 20260930; per class the halves differ by at most one image).

The change removed a Phase-4 bias: the final Phase-4 search variant had been chosen by the VAL-SELECT rank of policies that the archive had itself selected on VAL-SELECT. Phase 5 therefore used a new balanced split and conditions fixed in advance, and drew no inference about conditions from VAL-SELECT metrics of VAL-SELECT-selected policies. Of the 3,000 configuration-I VAL-RL images, 1,521 (CIFAR-10) and 1,515 (CIFAR-100) fall into the new VAL-SELECT, so sensitivity was recomputed. VAL-RL and VAL-SELECT name *roles*; the configuration is stated with every accuracy, and accuracies from different configurations or splits are never compared.

Table 4.1: Datasets and split configurations.

Item	CIFAR-10	CIFAR-100
Image size / classes	32×32 RGB / 10	32×32 RGB / 100 (fine)
Training / full validation	45,000 / 5,000	45,000 / 5,000
Configuration I (Phases 3–4): VAL-RL / VAL-SELECT	3,000 / 2,000 (seed 20260928)	3,000 / 2,000 (seed 20260928)
Configuration II (Phase 5 on): VAL-RL / VAL-SELECT	2,500 / 2,500 (seed 20260930)	2,500 / 2,500 (seed 20260930)
Test	official 10,000	official 10,000

4.3.3 Use of each split by phase

In every phase (Table 4.2) the reward split never selects the returned policy, and the test set never informs a search, selection or design choice.

Table 4.2: Data-split usage by phase.

Phase	Training	VAL-RL	VAL-SELECT	Test
2 Dense references	weight updates	— (full validation selects the epoch)	—	once per hashed reference
3	not used	I: sensitivity, reward, landscape, exploration metrics	not used	once per frozen final policy and matched baseline; confirmatory
4	not used	I: reward, training, search metrics, Stage-A/B selections	I: archive re-ranking, held-out ranks, Stage-C selection	locked until all selections and final policies were frozen; confirmatory
5	not used	II: sensitivity, choice of τ, reward, training, archive candidates	II: selection among frozen candidates only	after the final-policy list was committed; confirmatory
6	fine-tuning	II: recorded per epoch; no selection	II: recorded per epoch; no selection	after the 480 final checkpoints were committed; confirmatory
Paper 3	recovery	II: recovered accuracy, proxy validation, threshold calibration	II: stored, not analysed	not read
7A	recovery	II: **primary** accuracy criterion	II: secondary	not used
7B	proxy and final recovery	II: sensitivity, proxy, reward, archive	II: held out; **final** accuracy criterion	never read
7B energy confirmation	—	—	7B values, read only	—

The Phase-7A policy was trained on VAL-RL and selected on VAL-SELECT in Phase 5, so its validation accuracy may be optimistically biased relative to the baselines (recorded in the Phase-7A pre-registration). Without a frozen test protocol for the structured models, Phases 7A and 7B report validation accuracy only.

4.4 Neural Network Architectures

The networks are defined in `src/models`:

(a) **SimpleCNN** (the historical network): three 3×3 convolutions (padding 1, with bias) with 32, 64 and 128 filters, each with ReLU and 2×2 max-pooling; flattening to

2,048 features; a 256-unit hidden layer with ReLU; an output layer of C units. No batch normalisation or dropout; module names match the historical checkpoint.

(b) **LeNet-5, adapted to CIFAR** (LeCun et al., 1998): two 5×5 valid convolutions with 6 and 16 filters, each with ReLU and 2×2 max-pooling (replacing the original sigmoidal units and trainable subsampling); fully connected layers 400 → 120 → 84 with ReLU; an output layer of C units.

(c) **ResNet-8** (He et al., 2016) (n = 1, 6n + 2 = 8 weight layers): a 16-channel 3×3 stem convolution with batch normalisation; three single-BasicBlock stages with 16, 32 and 64 channels and strides 1, 2 and 2, each with two 3×3 convolutions and batch normalisation; identity shortcuts in stage 1 and 1×1 projections with batch normalisation in stages 2 and 3; global average pooling and an output layer of C units. Convolutions have no bias and Kaiming-normal (fan-out) initialisation. A pre-registered fallback for an over-budget ResNet-8 was not needed.

Table 4.3: Network architectures and pruning units (core framework). Counts are CIFAR-10 / CIFAR-100. "Weights" are the Conv2d and Linear weights, the denominator of sparsity (Equation 3.2).

Architecture	Datasets	Pruning units (share of prunable weights)	Dense parameters	Weights (prunable)	Main role
SimpleCNN	C10, C100	conv1 0.14%, conv2 2.99%, conv3 11.94%, fc1 84.93%	620,362 / 643,492	619,872 / 642,912 (617,312)	all phases; the only network of Phase 7B
LeNet-5	C10, C100	conv1 0.74%, conv2 3.94%, fc1 78.78%, fc2 16.54%	62,006 / 69,656	61,770 / 69,330 (60,930)	core phases 3–6 only
ResNet-8	C10, C100	stem 0.56%, stage 1 6.01%, stage 2 18.69%, stage 3 74.74%	78,042 / 83,892	77,360 / 83,120 (76,720)	core phases; Paper 3; Phase 7A

The output layer, biases and batch-normalisation parameters are never pruned.

4.5 Dense Reference Models

Each setting has one dense **reference model**, the starting point of every pruning method, trained in Phase 2 under a protocol committed before any training run: Adam, learning rate 10^{-3}, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, no weight decay; cosine annealing to zero over 30 epochs, stepped per epoch; batch 128, cross-entropy loss and the augmentation of Section 4.2; seeds 0, 1 and 2, where the seed sets the initial weights and a generator with seed +

10^6 sets data order and augmentation. The epoch with the highest full-validation accuracy is kept and hashed, and only then is the test set read, once.

The seed-0 model, fixed by rule, is the reference (seeds 1 and 2 only measure variability), except the disclosed CIFAR-10 SimpleCNN historical checkpoint (Adam at 10^{-3} for 10 epochs, final epoch). Epoch selection used the full validation set, later split into VAL-RL and VAL-SELECT, which affects every method of a setting identically. Later phases assert the reference hash (Table 4.4), and Phases 6, 7A and 7B add a dense model fine-tuned with the pruned models' protocol, separating recovery from extra training.

Table 4.4: Dense reference models (accuracy, %). The VAL-RL columns are the denominators A_b of retained accuracy in Phases 3–4 (configuration I) and Phase 5 (configuration II).

Setting	Checkpoint	SHA-256	Full validation	Test	VAL-RL (I)	VAL-RL (II)
CIFAR-10 SimpleCNN	historical, 10 epochs	`ca28f834`	77.32	77.03	76.9667	76.88
CIFAR-10 LeNet-5	seed 0	`5ca6e0fc`	64.32	64.96	64.3000	64.20
CIFAR-10 ResNet-8	seed 0	`41fc5ee5`	78.72	78.69	78.5333	78.92
CIFAR-100 SimpleCNN	seed 0	`d75dd19c`	49.08	50.50	50.3667	48.96
CIFAR-100 LeNet-5	seed 0	`8e1b1aee`	32.58	32.28	33.3000	32.76
CIFAR-100 ResNet-8	seed 0	`8b84bfad`	43.48	43.99	43.7667	43.76

The extension starts from the CIFAR-10 SimpleCNN and ResNet-8 references.

4.6 Pruning Units and Policy Spaces

4.6.1 Masked pruning units (core framework)

The four units per architecture are those of Tables 3.3 and 4.3, with ResNet-8's three **residual-stage weight groups** (stage 1: the two block convolutions; stages 2 and 3: plus the 1×1 projection). An action applies its ratio as **L1 unstructured** pruning to *each* tensor of the unit (Section 3.3); a ResNet-8 stage action removes no stage, block or channel. For the exploration endpoints, a policy is **destructive** if it prunes the setting's most sensitive unit (highest raw sensitivity, Equation 3.8; ties to the earliest unit) by 40% or more, a rule fixed before any PPO run.

4.6.2 Physically compacted structured units (deployment extension)

A ratio removes whole channels or neurons: a unit keeps the $k = \max(1, \text{round}((1 - r)\cdot C))$ channels with the largest L1 norm on the dense model (Equation 3.17), and the network is rebuilt with smaller layers.

In **SimpleCNN** (C = 32, 64, 128, 256) conv1–conv3 remove output filters and the next layer loses the matching inputs (16 flattened columns per channel of the first fully connected layer after conv3); fc1 removes hidden neurons and the output layer the matching inputs. In **ResNet-8** (C = 16, 16, 32, 64) the **stem is a coupled residual pruning group** setting the stage-1 residual width: one shared channel set, ranked by the summed L1 norms of the two writing filters, cuts the writers (the stem convolution and the second stage-1 convolution, each with its batch normalisation), the stage-1 identity shortcut, and the readers (the inputs of the first stage-1 convolution, the first stage-2 convolution and the stage-2 projection). **Stages 1–3 are block-internal channel groups** (the outputs of the block's first convolution and its batch normalisation, and the matching inputs of its second convolution); the stage-2 and stage-3 output widths and the output layer are not pruned.

The masked and structured ResNet-8 units are different objects (the masked stage-2 unit covers three convolutions, the structured one only block-internal channels), so a masked ResNet-8 policy maps onto structured units only approximately, a recorded Phase-7A limitation; for SimpleCNN the mapping is direct. Every compacted model is checked: no masks remain, parameter and MAC counts equal the analytic values, logits agree with the masked-equivalent model within 10^{-4}, and outputs are identical after reloading.

4.6.3 Policy spaces and enumeration

Every setting has $6^4 = 1{,}296$ policies. A PPO run samples 512 episodes, so it evaluates at most 512 policies (about 40% of the space). **Exhaustive diagnostic landscapes** of masked policies without fine-tuning exist on configuration-I VAL-RL (Phase 3), configuration-I VAL-SELECT (Phase 4) and both configuration-II splits (Phase 5); they are an exact cache for PPO's terminal evaluations, define each policy's rank, and are never computed on the test set. **Structured enumerations:** Paper 3 timed all 1,296 structured

shapes for SimpleCNN and ResNet-8, each with 10 and 100 output classes (four tables, 5,184 shapes), and Phase 7B recovered all 1,296 structured SimpleCNN policies with a 2-epoch protocol as a proxy cache. A planned exhaustive 5-epoch Paper 3 landscape (5,184 policies) was stopped for compute after 126 CIFAR-10 SimpleCNN policies, used only as pilot data, so fully recovered accuracy is known only for sampled or selected policies; energy was measured only for the models named in Section 4.18.

4.7 Baseline Pruning Methods

4.7.1 Core-framework baselines

The baselines are one-shot methods from `src/pruning`, each calibrated to an exact number K of zeroed weights from the prunable tensors only (the output layer stays dense), with largest-remainder rounding of per-tensor counts. All are deterministic except Random:

(a) **Uniform:** every prunable tensor loses the fraction K/N of its smallest-magnitude weights (N prunable weights).

(b) **Random:** uniform allocation with weights removed at random inside each tensor; ten mask seeds (1000–1009), summarised by their mean accuracy.

(c) **Global magnitude:** the K smallest-magnitude weights over all prunable tensors, a standard reference point in pruning evaluations (Blalock et al., 2020).

(d) **ERK** (Evci et al., 2020): tensor t receives density d_t = min(1, ε·(sum of its dimensions)/n_t), with ε solved so that N − K weights are kept (saturated tensors fixed dense, ε re-solved), removing the smallest-magnitude weights within each tensor.

(e) **LAMP** (Lee et al., 2021): each weight receives the score $w_(u)^2 / \Sigma_{v \geq u} w_(v)^2$ within its magnitude-sorted tensor, and the K smallest scores network-wide are removed; every tensor keeps its largest weight.

LAMP and ERK are layer-adaptive allocation rules, uniform and random fixed allocations, and global magnitude a single network-wide threshold; each is referred to by name, not as a "heuristic".

4.7.2 Deployment-extension comparators

Uniform-Structured uses one common channel ratio for all four units and **LAMP-Structured** a LAMP allocation translated into per-unit structured ratios. In Phase 7B, **old PPO** is the frozen Phase-7A transfer of the Phase-5 policy and **new PPO** the policy learned by the deployment-oriented structured PPO.

4.7.3 Matching rules

In the core framework, "matched" always means an **identical zero count**, i.e. identical sparsity ρ. In the extension, MACs or parameter counts are matched approximately, because channel counts are integers (Table 4.5).

Table 4.5: Baseline methods and matching rules.

Comparison	Methods	Matching rule	Exactness
Phase 3	P0 vs Uniform, Random, Global, ERK, LAMP	zero count of each PPO run's final model; PPO chooses its own sparsity, so there is no common target	exact, asserted
Phase 3, descriptive	the five baselines	fixed sparsity grid 10–60%	curves only; never treated as matched
Phase 4	F, K vs LAMP, Global; F, K vs Uniform, ERK, Random	zero count of each distinct selected model	exact
Phase 5	C0, C1, C2 vs LAMP, Global	zero count of each selected policy	exact (mismatch 0)
Phase 6	PPO vs LAMP, Global, dense control	zero count per (setting, policy seed)	exact (mismatch 0, 120/120 triples)
Phase 7A	PPO vs Uniform-Structured	**MACs**: common ratio on a 0.001 grid minimising |MACs − MACs_PPO| (ties: smaller ratio)	approximate; residual gap recorded
Phase 7A	PPO vs LAMP-Structured	Phase-6 LAMP mask at PPO's zero count; per-unit sparsity realised as structured ratios	approximate; not MAC-matched
Phase 7B	new PPO vs Uniform-Structured	**MACs**, 0.001 grid over [0, 0.9]	approximate
Phase 7B	new PPO vs LAMP-Structured	**parameter count**: LAMP sparsity on a 0.1% grid over [0, 90]%, realised structurally; closest parameter count (ties: smaller sparsity)	approximate
Phase 7B	new PPO vs old PPO	none (frozen Phase-7A checkpoint)	not matched

4.8 PPO Training Configuration

Every agent uses Stable-Baselines3 2.8.0 (Raffin et al., 2021), running PPO (Schulman et al., 2017) with generalised advantage estimation (Schulman et al., 2016) and the hyper-parameters of Table 3.7; Table 4.6 shows what differs by phase (Phases 3–6 spread the single-thread runs over four worker processes). The evaluation data loader is still created at each cached terminal step, so the random stream matches an uncached run.

Table 4.6: PPO configuration by phase.

Item	Phase 3	Phase 4	Phase 5	Phase 7B
Network; lr; n_{steps}; batch; epochs	MLP [64, 64] tanh; 3×10^{-4}; 64; 32; 10	same	same	same
γ; GAE λ; clip; entropy; value; grad norm	0.99; 0.95; 0.2; 0.01; 0.5; 0.5	same	same	same
Budget; threads	2,048 steps = 512 episodes; 1	same	same	same
Environment; state	masked; 7 features, sensitivity slot 0	same	same	structured SimpleCNN; 8 features with centred sensitivity
Prior	P0–P3, uncentred, β = 0.5	P0–P5; P2 centred, β = 1.0	P2 (C0, C1); none (C2)	centred structured, β = 1.0
Reward	R0	R0, R1a, R1b, R2	R0 (C0); R_c, τ = 0.98 (C1, C2)	R_{7B}, α = 4
Evaluation cache	VAL-RL (I) landscape	same	VAL-RL (II) landscape	2-epoch proxy landscape
Extra loss	none	elite behavioural cloning in S2 (K = 5, η = 0.1)	none	none
Returned policy	terminal	terminal (S0, S2) or archive (S1, S2A)	archive	feasible archive
Seeds	300–319	400–419	500–519	900–904 (development); 700–719 (confirmatory)

The Phase 3–5 seeds were verified unused by any earlier PPO run, except an unrecorded 64-timestep smoke test with seed 300 in the Phase-2 checks. Seeds are shared across conditions and settings within a phase, so comparisons are paired; no seed was removed, replaced or added. Elite behavioural cloning (Phase 4, S2) adds $\eta\cdot\mathcal{L}_{\text{elite}}$ (Equation 3.13) to a copy of the SB3 PPO update, bit-identical to standard PPO at η = 0.

4.9 Experimental Conditions by Phase

Table 4.7 summarises each phase.

Table 4.7: Phase-by-phase experimental design.

Phase	Research purpose	Datasets	Architectures	Conditions	Seeds	Runs	Primary output
3	PPO vs baselines; sensitivity prior	C10, C100	all three	P0, P1, P2, P3	300–319	480	test accuracy vs matched baselines; exploration endpoints
4	reward, centred prior, retention	C10, C100	all three	Stage A: 4 rewards; B: 6 priors; C: 4 search variants	400–419	1,200	stage selections; search quality; final pipeline vs control and LAMP
5	accuracy-constrained reward	C10, C100	all three	C0, C1, C2	500–519	360	constraint, search and accuracy endpoints
6	equal fixed-mask recovery	C10, C100	all three	PPO, LAMP, global, dense control	fine-tuning 600–619	480 (+6 repeats)	accuracy before and after equal fine-tuning
Paper 3	structured CPU latency	C10, C100 (latency); C10 (accuracy)	SimpleCNN, ResNet-8	1,296 shapes per table	recovery seed 600	5,184 shapes	latency tables; GO/NO-GO for hardware-aware PPO
7A	structured transfer, measured energy	C10	SimpleCNN, ResNet-8	Dense, PPO-, Uniform-, LAMP-Structured	recovery seed 600	8 models; 5 sessions per architecture	accuracy, memory, MACs, latency, energy
7B	deployment-oriented structured PPO	C10	SimpleCNN	new PPO; Dense, Uniform-, LAMP-Structured, old PPO	900–904; 700–719	20 searches; 11 recovered models	accuracy, memory, MACs; planned latency and energy
7B energy confirmation	direct energy of frozen 7B models	C10	SimpleCNN	Dense vs new PPO	—	6 qualification blocks; 10 sessions planned	qualification; planned paired energy

4.9.1 Phase 3

Phase 3 ran the four conditions of Section 3.9 (P0, no prior; P1, $c_l = 0.5 \cdot S_l$; P2, S permuted per seed so that every unit receives another unit's value; P3, the mean of S for every unit) across six settings with 20 seeds: 480 runs, returning the terminal deterministic policy, pruned without fine-tuning, with baselines built after all PPO runs. Four separate questions were posed: (A) PPO vs uniform; (B) PPO vs global magnitude, LAMP, ERK and random; (C) whether the correct prior improves exploration (primary endpoint: VAL-RL accuracy AUC); and (D) whether it improves the final model (test accuracy in excess of matched global magnitude, P1 − P0). "P2" is the *shuffled* prior in Phase 3 and the *centred* prior in Phases 4–5; the phase is always stated.

4.9.2 Phase 4

Phase 4 used a staged design with mechanical, validation-only selections, on seeds 400–419:

(a) **Stage A, reward.** R0, R1a, R1b and R2 (Section 4.12), with P0 and the terminal policy: 480 runs. A redesigned reward qualified only if it at least halved the cross-setting spread of final sparsity (with a seed-bootstrap upper bound below 1) **and** kept VAL-RL retention ≥ 0.95 in at least five settings; otherwise R0 was kept.

(b) **Stage B, prior.** 600 new runs under the Stage-A reward with six priors: P0, none; P1, $c_l = 0.5 \cdot S_l$; P2, $c_l = 1.0 \cdot (S_l - \bar{S})$ (centred); P3, P2 permuted across units; P4, $c = 1.0 \cdot \text{mean_1}\,|S_l - \bar{S}|$ for every unit (magnitude-matched pure conservatism); P5, $c_l = -1.0 \cdot (S_l - \bar{S})$ (sign-reversed). P1 and P2 were each tested against P0 on reward-rank AUC; a prior supported in at least three settings with none opposite was eligible, the eligible prior supported in more settings was carried forward (tie: larger mean d_z), and otherwise P0 was.

(c) **Stage C, search.** 120 new runs: **S0**, terminal policy; **S1**, archive rule on the S0 runs; **S2** and **S2A**, elite behavioural cloning with the terminal policy or the archive rule. The winner had the lowest mean, over settings, of the median VAL-SELECT rank of the selected policy.

The selections kept R0 (no redesigned reward qualified), chose prior P2 and chose search variant S2A, defining the final pipeline **F = (R0, P2, S2A)** and the control **K = (R0, P0, S0)**, the Phase-3 configuration on fresh seeds.

4.9.3 Phase 5

Phase 5 used configuration II and three conditions fixed in advance: **C0**, R0 + P2 + archive, selecting by the highest R0 score on VAL-SELECT (the Phase-4 pipeline without behavioural cloning); **C1 (proposed)**, R_c with $\tau = 0.98$, P2 and constraint-aware selection (Equation 3.14); and **C2**, as C1 without a prior. With seeds 500–519 the design has 360 runs; C1 − C2 is the sensitivity ablation and C1 − C0 the reward comparison. τ was chosen before any run, on the VAL-RL landscapes only, as the largest value in {0.95, 0.97, 0.98, 0.99} giving every setting at least 20 feasible policies, including one with ≥ 10% sparsity: $\tau = 0.98$.

4.9.4 Phase 6

Phase 6 tests whether accuracy differences persist under the same fine-tuning budget at exactly the same sparsity. The methods are **PPO** (the frozen Phase-5 C1 policy of each setting and policy seed), **LAMP** and **global magnitude** at PPO's exact zero count, and a **dense control**. **Phase 6 contains no uniform, ERK or random method, and no Phase-5 C0 or C2 policy**, which the pre-registration excluded to fit the fixed compute budget. There are 360 pruned runs plus 120 dense controls; the four runs of each (setting, policy seed) share the fine-tuning seed (policy seed + 100), minibatch order and augmentation, checked by a data-stream hash, and six reproduction runs repeat seed 500's PPO run in each setting (Section 4.13). The unit of analysis is the policy seed (n = 20 per setting); because seeds often selected the same policy, a cluster-level sensitivity analysis uses one value per distinct policy where there are at least six.

4.9.5 Paper 3

Paper 3 tested whether structured compression becomes CPU latency, through pre-registered decisions: **H0**, a calibration selecting the primary timing configuration (Section 4.17); **H1**, 50 checks of structured compaction, MAC and FLOP counters and structured sensitivity, with the Phase-2 library checks rerun on the laptop; and **H2-A**, the four

latency tables (5,184 shapes). For CIFAR-10 recovered accuracy, a 1-epoch recovery proxy was validated against 5-epoch recovery on frozen samples (156 SimpleCNN and 31 ResNet-8 policy pairs), and an architecture could proceed only if the Spearman correlation was ≥ 0.90. The protocol's Youden-J (Youden, 1950) threshold rule needed at least three truly feasible validation pairs; ResNet-8 had fewer, triggering the pre-registered fallback to a decision by the principal investigator, and a pre-committed calibration on 48 frozen ResNet-8 policies set the threshold by a recall-first rule with a 0.005 margin. A latency-room rule then required at least 20 proxy-feasible policies and one with a speed-up ≥ 1.10×.

Stopping outcomes (values in Chapter 6). SimpleCNN did not meet the proxy-validity criterion; ResNet-8 met it and passed calibration, but not the latency-room rule. Hardware-aware PPO was therefore stopped before any run, and **no hardware-aware reinforcement learning was run**; Paper 3 contributes the latency tables and their cost-metric analysis.

4.9.6 Phase 7A

Phase 7A uses CIFAR-10 SimpleCNN and ResNet-8 with four methods each: **Dense**; **PPO-Structured**, the most frequent Phase-5 C1 selected policy over seeds 500–519 (ties: smaller action vector), read as structured ratios; **Uniform-Structured**, MAC-matched to PPO; and **LAMP-Structured**, from the Phase-6 LAMP mask. All receive the equal 5-epoch recovery with seed 600 and five energy sessions (Section 4.18). The fixed criteria are **accuracy**, $|A_{method} - A_{dense,recovered}| \leq 0.5$ pp on VAL-RL, and **consistent energy reduction**, lower energy than the same-session dense model in 5 of 5 sessions **and** a median saving above a noise threshold of twice the dense model's between-session coefficient of variation. **Case A** (both hold) is the primary success; **B** is consistent reduction with more than 0.5 pp loss; **C** accuracy within 0.5 pp without consistent reduction; **D** a baseline with the better trade-off, i.e. accuracy within 0.5 pp of the other method and lower energy in 5 of 5 sessions, or accuracy higher by more than 0.5 pp with median energy not higher.

4.9.7 Phase 7B

Phase 7B uses SimpleCNN on CIFAR-10 only. Its agent, the deployment-oriented structured PPO (Section 3.16.3), was rewarded during the search with the *surrogate (predicted)* energy saving of Equation (3.18); no measured energy entered the search, and the name does not imply a measured energy result.

Stage A (development), before the final pre-registration, validated the proxy and surrogate (Section 4.18.3), built the 2-epoch proxy landscape and tuned $\alpha \in \{0.5, 1, 2, 4\}$ on seeds 900–904 by the lowest median predicted energy of the selected policies (ties: smaller α), giving $\alpha = 4$; $\beta = 1.0$, $\kappa = 10$, $\varepsilon = 0.01$, the 1% tie band and the selection rule were fixed, not tuned. **Stage B (confirmatory)** ran 20 seeds (700–719), all reported, with structured sensitivity on VAL-RL (probes at $\{0.1, 0.2, 0.4, 0.6\}$) and $\hat{\tau} = 0.98849$, whose target was 5-epoch VAL-RL accuracy within 0.5 pp of the recovered dense model. All final models (dense, the primary new-PPO policy, the other distinct per-seed selections, Uniform- and LAMP-Structured) receive the equal 5-epoch recovery; old PPO is reused read only.

Joint deployment-efficiency criterion (named *GREEN-SUCCESS* in the pre-registration). A model succeeds only if all five conditions hold at once: (1) VAL-SELECT accuracy at least the recovered dense VAL-SELECT accuracy minus 0.5 pp; (2) parameter-memory reduction > 0; (3) MAC reduction > 0; (4) latency reduced, with the median below dense and the model faster in 5 of 5 sessions; and (5) measured energy reduced beyond noise, lower in 5 of 5 sessions with a median saving above twice the dense model's between-session coefficient of variation. There is no override, an empty archive counts as failure, and the success frequency over the 20 seeds is also defined.

4.9.8 Phase 7B energy confirmation

This protocol, written after Phase 7B had closed, does not alter Phase 7B or its verdicts. After hash verification and qualification of the instrument on the dense model (Section 4.18.5), ten paired Dense and new-PPO sessions would run, five in each order (seeded permutation). Confirmation requires that the setup qualified, the PPO model used less energy in at least 8 of the 10 sessions, the median saving exceeded the noise threshold, and a leave-one-session-out check held for every omitted session.

4.10 Sensitivity Conditions and Controls

Sensitivity enters the core framework only through the prior (state slot 0; Section 3.5). **Layer-specific information** is tested by priors that scramble the unit assignment of the same pressure (shuffled P2 in Phase 3; permuted P3 in Phase 4); **generic conservatism** by pressure without sensitivity information (constant P3 in Phase 3; magnitude-matched P4 in Phase 4); and **direction** by the sign-reversed P5 (Phase 4). The centred P2 of Phase 4 removes net conservatism by construction, and the sensitivity-specific statement held in a setting only if P2 beat **both** P3 and P4 there. Phase 5 compares C1 with C2; Phase 7B ran no prior-free control. The vectors are computed on configuration-I VAL-RL (Phases 3–4), configuration-II VAL-RL (Phase 5) and, with structured probes, configuration-II VAL-RL (Phase 7B); none uses VAL-SELECT or test data.

4.11 Archive and Selection Protocols

Phase 3 returns the terminal deterministic policy without an archive. In **Phase 4**, S0 and S2 return the terminal policy and S1 and S2A apply Equation (3.12) to the archive (Section 3.12).

Phase 5 follows a fixed order:

(i) The archive of all 360 runs was frozen and hashed before any VAL-SELECT evaluation.

(ii) The candidates were evaluated on VAL-SELECT, and each result had to equal the VAL-SELECT landscape.

(iii) C1 and C2 selected by Equation (3.14), with its tie-break and reported fallback; C0 by the highest R0 score on VAL-SELECT.

(iv) The final-policy list was committed before the test set was read.

In **Phase 7B** the archive holds every distinct proxy-feasible policy visited. Per seed, the lowest predicted energy is selected; policies within 1% count as tied, and ties go to higher proxy accuracy, then fewer MACs, then lower parameter memory, then the smaller action vector. The primary new-PPO model is the most frequent selection over the 20 seeds. The same rule applied to every proxy-feasible landscape policy gives an enumeration reference, reported but not a method. VAL-SELECT plays no part in Phase-7B selection.

4.12 Reward Configurations

All rewards are computed on VAL-RL from the pruned network without fine-tuning; Phase 7B uses its 2-epoch proxy.

Table 4.8: Reward configurations.

Reward	Definition	Used in
R0	$1.5 \cdot A_p/A_b + 0.01 \cdot \rho(\%) + 0.05 \cdot D$ (Equation 3.7)	Phases 3 and 4; Phase 5 C0
R1a	$1.5 \cdot A_p/A_b - \kappa_a \cdot \lvert\rho - 50\rvert + 0.05 \cdot D$	Phase 4 Stage A
R1b	$1.5 \cdot A_p/A_b - \kappa_b \cdot \lvert\rho - \rho^*_s\rvert + 0.05 \cdot D$, with ρ^*_s the largest sparsity retaining ≥ 95% of VAL-RL accuracy	Phase 4 Stage A
R2	$1.5 \cdot (A_p - A_60)/(A_b - A_{60}) + 1.5 \cdot \rho/\rho_60 + 0.05 \cdot D$, anchored on the maximal policy [5,5,5,5]	Phase 4 Stage A
R_c	constraint-first, $\tau = 0.98$ (Equation 3.15)	Phase 5 C1, C2
R_{7B}	constraint-first on the proxy, predicted energy saving, $\alpha = 4$ (Equation 3.19)	Phase 7B

κ_a and κ_b were each set before any run as the smallest value in {0.005, 0.01, 0.02, 0.04, 0.08} whose VAL-RL reward optimum lies within ±5 pp of its target in all six settings. Only R0 and R_c enter the mature core framework.

4.13 Recovery and Fine-Tuning Protocol

The Phase-6 protocol (Section 3.15) is used unchanged in every later phase: Adam at 10^{-4}, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, weight decay 0, a new optimiser per run; cosine annealing to 0, stepped per epoch; batch 128 on the 45,000 training images, with crop and flip; one thread; no checkpoint selection. **E = 5** was fixed before any confirmatory run by a timing rule (E = 5 if the projected Phase-6 wall time was at most 30 hours, otherwise E = 3); the projection, from a pilot that used no accuracy, was 29.44 hours, and no 10-epoch schedule was run.

In Phase 6 the mask is fixed by PyTorch's pruning reparameterisation (Paszke et al., 2019), with the mask hash, the absence of non-zero masked weights and per-layer zero counts asserted after every epoch. Compacted models (Paper 3, 7A, 7B) must keep their tensor shapes through recovery and all use recovery seed 600 with data generator $600 + 10^6$, so all models of a phase see the same minibatch sequence. The Paper-3 1-epoch proxy is exactly the first epoch of the 5-epoch protocol; the Phase-7B 2-epoch proxy is a

separate two-epoch cosine run on the same seed and data stream. Final accuracy claims always use 5-epoch models.

4.14 Evaluation Metrics

Table 4.9 lists the metrics; accuracy always names its split, and every reduction its reference. Masked sparsity ρ, the channel removal ratio r_l (Equation 3.17), physical parameter reduction over all parameters including biases and batch normalisation, and MAC reduction are kept apart; per-unit sparsity is used only to translate LAMP allocations into structured ratios.

Table 4.9: Evaluation metrics. ↑ higher is better; ↓ lower is better.

Group	Metric (symbol)	Definition	Phases	Better
Predictive	accuracy (A)	top-1 accuracy (%) on a named split	all	↑
	retained accuracy (q)	A / A_{ref}, with A_{ref} the dense or recovered dense model on the same split	3–7B	↑
	accuracy difference (Δ)	A − A_{ref}, or method minus baseline (pp)	all	↑
	recovery; gap change ΔG	$A_{post} - A_{pre}$; ($A_{PPO} - A_{base}$) after minus before fine-tuning	6	↑
	retention gap	VAL-SELECT retention minus test retention	6	↓
	proxy accuracy ($\hat{q}$)	A_E / $A_{E,dense}$ on VAL-RL after an E-epoch recovery (E = 1 in Paper 3, 2 in 7B)	Paper 3, 7B	↑
Compression	masked sparsity (ρ)	Equation 3.2	3–6	context
	parameter reduction	1 − P/P_dense	Paper 3, 7A, 7B	↑
	parameter memory	Σ numel × element size (bytes, float32)	7B	↓
	serialised size	saved state-dict bytes	6, 7A, 7B	↓
Computation	MACs; FLOPs	Conv2d/Linear multiply–accumulates per image, closed form checked against hooks; FLOPs = 2 × MACs	Paper 3, 7A, 7B	↓
Deployment	latency (T)	median per-inference batch-1 time (ms)	Paper 3, 7A, 7B-EC	↓
	speed-up; latency gain (g_T)	T_{ref} / T; 1 − T/T_identity within one table	Paper 3, 7A	↑
Energy	E	package J / N × 1,000 for a block of N inferences	7A, 7B-EC	↓

Group	Metric (symbol)	Definition	Phases	Better
	energy per inference	package J / N	7A, 7B-EC	↓
	energy saving	100·(1 − E/E_dense), same-session dense model	7A, 7B-EC	↑
	EDP	(J/inference × latency) normalised to the same-session dense model; not a carbon metric	7A, 7B-EC	↓
	predicted energy (Ê)	Equation 3.18; inside the search only, never a result	7B	—
Search	reward rank; rank score	rank among the 1,296 landscape policies (1 = best; ties get the minimum rank); (1,296 − rank)/1,295	3–5	↓; ↑
	constrained rank	rank under R_c on the VAL-RL landscape (common yardstick for C0–C2)	5	↓
	best-seen and selected rank	minimum rank over the 512 sampled policies; rank of the returned policy	4, 5	↓
	top-k reach; first entry	best-seen or selected policy within the top k (100, 50, 20, 10, 5, optimum); first episode reaching it (censored at 513)	4, 5	↑; ↓
	trajectory AUC	trapezoidal integral over episodes of the sampled policy's rank score, VAL-RL accuracy or utility, normalised by the episode span	3–5	↑
	destructive frequency	share of sampled policies that are destructive (Section 4.6.1)	3–5	↓
	frontier regret; feasible-sample rate	best accuracy at equal or higher sparsity minus own accuracy; share of samples with $q \geq \tau$	4; 5	↓; ↑

4.15 Statistical Analysis and Decision Criteria

4.15.1 Paired tests (Phases 3–6)

Comparisons are paired by seed (n = 20 per setting) with the **exact two-sided sign-flip test** over all 2^{20} sign patterns as primary test, reporting the mean, standard deviation and median, the 95% t-interval (df 19), a bootstrap 95% interval (10,000 resamples in Phase 3), d_z, the exact Wilcoxon signed-rank p (Wilcoxon, 1945) (Phases 3 and 4), and wins, ties and losses. Holm's step-down correction (Holm, 1979) is applied within each pre-registered family. A comparison is **supported** with the hypothesised direction, a Holm-adjusted $p < 0.05$ **and** a 95% t-interval excluding 0; **opposite** under the same criteria in reverse; otherwise **inconclusive**. Across settings (Phases 3–4) a statement is **STRONG** if

supported in ≥ 5 of 6 settings and opposite in none, **MODERATE** if supported in 3–4 and opposite in none, **LIMITED** if supported in 1–2 (or ≥ 3 with one opposite), and **NOT SUPPORTED** otherwise; opposite in ≥ 3 settings is also labelled REVERSED. No subset result is described as general.

Families. Phase 3: A (6), B (24), C (6), C-specificity (12), one per C-secondary endpoint, and D (6), with McNemar counts supportive only. Phase 4: reward, prior, search and final-model families, a seed bootstrap for Stage A, and a **one-sided non-inferiority test** of F against LAMP with a 0.5 pp margin (a Holm-corrected sign-flip test on d + 0.5, plus a lower t-bound above −0.5). Phase 5: C1 − C0 on test accuracy (6); C1 − C2 on four primary exploration endpoints (24: reward-rank AUC, VAL-RL accuracy AUC, first episodes in the top 20 and top 10) and on secondary endpoints (18); C1 − LAMP and its gap change (6 each); C1 − global magnitude **descriptive**. Phase 6: recovery, dense-control and pruning-specific recovery families; PPO vs LAMP and PPO vs global (12 each); the retention gap; and, because architecture dependence compares different experiments, an unpaired permutation test (100,000 permutations) with a Welch interval, plus a seed-paired sensitivity version.

4.15.2 The ±0.5 pp rule of Phase 6

Phase 6 reports statistical support and practical size separately, with a fixed 0.5 pp margin:

(a) a difference is *practically relevant* only if |mean| ≥ 0.5 pp;

(b) two methods lie **within the ±0.5 pp margin** if the 95% t-interval of their paired difference lies inside (−0.5, +0.5) pp;

(c) a supported difference below 0.5 pp is *statistically detectable but small*.

This is a **pre-registered confidence-interval inclusion rule**, not a formal statistical equivalence test. A formal test, such as two one-sided tests with a justified margin and stated error rate, was not pre-registered. The pre-registration calls the rule "practically equivalent"; the thesis writes "within the ±0.5 pp margin" so that it is not read as a formal equivalence claim.

4.15.3 Deployment-extension statistics

Paper 3 is descriptive and rank-based: Spearman correlations of latency gain with parameter and MAC reduction over all 1,296 shapes, the fraction of policy pairs in which the policy with more parameter reduction is slower, and the H0 signal-to-noise rule; proxy validity uses Spearman correlation, error, bias, top-10 overlap and a confusion matrix. **Phases 7A and 7B** summarise sessions by median, mean, standard deviation and interquartile range, and test each method against dense with an **exact one-sided Wilcoxon signed-rank test** on five paired sessions, whose minimum attainable p of 0.031 makes p-values secondary to the 5-of-5 rule and the noise threshold; two-sided tests between methods are descriptive. The **energy confirmation** pre-registered an exact Wilcoxon and a sign test on ten pairs. No other significance test was used.

4.15.4 Pre-registered decision rules

Table 4.10: Pre-registered decision criteria. The commit is the one that first added the governing file.

Experiment	Frozen rule	Purpose	Source (commit)
Phase 2	seed-0 reference; best full-validation epoch; test once after hashing	dense references	`cpu_dense_training_protocol.md` (`366d21ae`)
Phase 3	outcome and class rules; four separate questions	PPO vs baselines; prior	`phase3_preregistration.md` (`2dbcc770`)
Phase 4	mechanical Stage A–C selections; criteria A–E (reward, specific prior, search, final model, LAMP)	reward, prior, retention	`phase4_preregistration.md` (`f52e87d9`)
Phase 5	τ rule; criteria A–F (accuracy safety, compression, search, sensitivity, final model, LAMP)	constrained reward	`phase5_preregistration.md` (`3cbaec1a`)
Phase 6	E timing rule; criteria A–F (fairness, recovery, competitiveness, generalisation, sparsity preservation, reproducibility); ±0.5 pp rule	equal recovery	`phase6_preregistration.md` (`b1b5396f`)
Paper 3 H0	SNR ≥ 5, reduction ≥ 10%, positive in every session; order A, C, B	primary latency configuration	`h0_protocol.md` (`c2dcb22a`)
Paper 3 H2, proxy	table protocol; proxy GO iff Spearman ≥ 0.90; Youden threshold with fallback	latency tables; proxy	`paper3_preregistration.md` (`45936846`); `paper3_proxy_protocol.md` (`ec7a7f27`)
Paper 3 ResNet-8	≥ 5 feasible calibration policies; threshold rule; latency room (≥ 20 feasible, one ≥ 1.10×)	GO/NO-GO for hardware-aware PPO	`resnet8_threshold_calibration_protocol.md` (`5b127804`)
Phase 7A	±0.5 pp on VAL-RL; consistent energy reduction; Cases A–D; pilot CV ≤ 3%	transfer and energy	`preregistration.json` (`b8c27665`)
Phase 7B Stage A	proxy acceptance; surrogate acceptance; calibration pilot CV ≤ 4%; α rule	development	`phase7b_design.md` (`f5abe039`)
Phase 7B Stage B	joint deployment-efficiency criterion; comparison rule; pilot CV ≤ 3%, N doubled once, then STOP	confirmatory	`phase7b_final_preregistration.json` (`0e101d55`)
7B energy confirmation	qualification CV ≤ 3%, one attempt; energy confirmation rule	direct energy	`protocol.json` (`df233be0`)

4.16 Hardware and Software Environment

All experiments ran on CPUs with CPU-only PyTorch builds; no GPU was used (Table 4.11).

Table 4.11: Hardware and software environments.

Item	Core framework (Phases 2–6)	Extension (Paper 3, 7A, 7B, energy confirmation)
CPU	Intel Core i5-6400 @ 2.70 GHz, 4 logical CPUs (desktop)	Intel Core i7-1260P (laptop): 4 performance cores × 2 threads (logical 0–7), 8 efficiency cores (8–15)
Memory	not recorded	15.7 GB
OS	Windows 11, build 22621	Windows 11 Home, build 26200
Python; PyTorch	3.11.9; 2.7.1+cpu (torchvision 0.22.1+cpu)	3.11.9; 2.7.1+cpu with oneDNN 3.7.1 and oneMKL 2025.1
Other libraries	Stable-Baselines3 2.8.0, sb3-contrib 2.8.0, Gymnasium 1.2.3, NumPy 2.2.2, pandas 2.2.3, ptflops 0.7.5, psutil 7.2.2	Stable-Baselines3 2.8.0 in the pinned environment; the 7B records do not store package versions separately
Threads	1 per run (dense training: 4)	1 intra-op, 1 inter-op
Power	mains	AC required; Balanced plan

Package versions are pinned in `requirements_locked.txt`, and every phase records its environment and commit, verified by the integrity checks. **Results do not transfer between machines:** no desktop latency or energy value is used.

4.17 Structured Pruning and Latency Protocol

Measurement core (Paper 3, reused in Phases 7A and 7B): batch 1, float32, NCHW, `eval()` and `inference_mode()` on one preallocated, seeded input tensor; one intra-op and one inter-op thread, with affinity and OpenMP/MKL thread variables set before PyTorch is imported and the process pinned to logical CPU 2, a performance-core thread other than core 0, avoiding hybrid-scheduling noise; each pass timed with `time.perf_counter_ns()` with garbage collection disabled, never timing model construction, loading or data handling; AC power required, system load ≤ 15% over 5 s before and after each session, and no other experiment running.

H0 calibration. Six SimpleCNN shapes were timed under configurations **A** (batch 1, 1 thread on CPU 2, 50 / 300 warm-up / timed passes), **B** (batch 1, 4 threads, one per performance core, 50 / 300) and **C** (batch 128, 4 threads, 10 / 50), in five rounds opening

with a dense anchor block over five fresh-process sessions; the rule selected configuration A.

H2-A latency tables. Configuration A in five fresh-process sessions per table, each timing all 1,296 shapes once in a seeded random order, with a dense anchor block before the first shape and after every 25. A session whose anchors drift by more than 10% from their median is repeated once; four attempts were repeated, and all 20 accepted sessions met the criterion. A shape's latency is the median over sessions of its per-session median (the 95th percentile, interquartile range, throughput, speed-up, latency gain against the identity shape and an anchor-normalised latency are also stored), and the tables were hashed and committed before use. The absolute dense latency on the table days was about 16% higher than on the H0 calibration day, a cross-occasion check kept as a recorded failure; within each table the identity shape agrees closely with the dense anchors, so relative within-table latency is analysed, later sessions compare against a same-session dense model, and the Phase-7B surrogate fits table latency to energies measured on the same machine. **Phase 7A latency** is the median per-inference time over the energy block's N inferences, with speed-up computed within a session.

4.18 Energy Measurement Protocol

4.18.1 Backend

Energy is read from the **Intel RAPL package counter** (Khan et al., 2018) through the **Windows Energy Meter Interface** (counter `\Energy Meter(RAPL_Package0_PKG)\Energy`): raw PDH values in picowatt-hours (1 pWh = 3.6×10^{-9} J), refreshed about every 1 ms, read immediately before and after each block. The core (PP0) domain is secondary; the DRAM domain reads zero on this machine and is not used. No TDP model, software estimate or surrogate replaces a measurement, and no external meter was available. Software energy counters need careful protocol design and stability checks (Jay et al., 2023; Raffin and Trystram, 2025), hence the gates and qualification rules below.

4.18.2 Phase 7A

Per model, 1,000 warm-up inferences precede one block of N batch-1 inferences (E = package J / N × 1,000), in five fresh-process sessions per architecture, each with the four models in a seeded random order after 30 s of idle-power measurement, under the gates of Section 4.17 and an identical power plan. A dense-only pilot of 2 processes × 3 blocks set N: N = 100,000 if the coefficient of variation of the six values was ≤ 3% for both architectures, otherwise N was doubled once and then the phase stopped. The rule was met at N = 100,000.

4.18.3 Phase 7B energy surrogate

The surrogate's target was **directly measured RAPL energy** of 200 structured policies fixed with seed 20261028: dense and the H0 policies (7), 160 policies stratified over eight MAC-reduction bins, and similar-MAC pairs (MACs within 2%, widths far apart) separating MACs from shape. They were measured in three fresh-process sessions on CPU 2 (200 warm-up inferences, N = 20,000 per policy, a dense anchor every 25 policies, sessions drifting by more than 10% repeated), taking the three-session median; a dense pilot with a 4% limit met the limit at N = 20,000. Candidate predictors were tried in order, the first to pass being accepted: (1) MACs; (2) table latency T_{table}; (3) MACs, parameters and T_{table}; (4) a degree-2 ridge model in the normalised widths plus MACs, parameters and T_{table}. Acceptance required, on 5-fold cross-validated (seed 20261029) out-of-fold predictions, Spearman ≥ 0.95, a median absolute percentage error ≤ 5%, and the same error ≤ 5% within the lowest-energy quartile. Candidate 1 failed; candidate 2 was accepted, giving Equation (3.18) (statistics in Chapter 6). **Predicted energy is never reported as measured energy.**

4.18.4 Phase 7B confirmatory measurement and its stopping rule

Five fresh-process sessions were planned (30 s idle, 1,000 warm-up inferences, N = 100,000 per block, 5 s pauses; dense three times per session, every other model once, in a seeded order), preceded by the Phase-7A pilot rule with a 3% limit (N doubled once, then STOP). The pilot gave a coefficient of variation of 11.36% at N = 100,000; the first doubled-N attempt was halted by the load gate during foreground activity and, by recorded user decision, declared void (as an earlier AC-gate refusal had been); the

permitted rerun at N = 200,000 with the laptop idle gave 4.01%, above the 3% limit. Under the frozen rule, hardware validation **stopped**: no confirmatory session was run, so the latency and energy conditions of the joint criterion could not be assessed. **No Phase-7B energy reduction is reported.**

4.18.5 Energy confirmation study

A new protocol, committed before any qualification data, used a 180 s thermal soak, 2,000 warm-up inferences and N = 200,000 per block with 5 s pauses, a fresh child process per run, and all models hash-checked before the soak, recording package and core energy, latency, system load, CPU-2 processor performance, thermal-zone temperature and AC status per block. A run is void, kept and repeated if load exceeds 15% during a block or the laptop leaves AC (never on energy or latency values); more than three voids in a stage is a stop.

Qualification used the dense model only (3 fresh runs × 2 blocks; coefficient of variation of the six values ≤ 3%; one attempt, no relaxation). The coefficient of variation was 19.92%, so the setup was **not qualified**, and the study stopped **before any paired Dense-versus-PPO measurement**: energy confirmation is NO. The stop reflects the instability of the measurement setup; no comparison of the two models was made.

4.19 Reproducibility and Pre-Registration

Each protocol was committed before its confirmatory data (Table 4.10) and never edited (necessary changes were separate amendments committed before the affected runs). There was no interim peeking and no result-based exclusion; infrastructure failures were repeated with the same seed and logged, and resumption was verified to be bit-identical. Implementation checks used only seed 42 and synthetic inputs; integrity checks before any statistic covered complete run matrices, unchanged hashes, no test access in training, the freeze order of archives, final lists and test evaluations, exact matching, and thread counts and environments. Splits, references, final-model lists, checkpoints and latency tables are stored with SHA-256 hashes; six Phase-6 reproduction runs test bit-identical reproduction, and Paper 3 verified that recovery does not depend on the core that runs it.

Test-set governance. Code enforces Table 4.2: a guard raises on any test access during training, or before a phase's final-model list is frozen. In Phases 3–5 the test set was read once per distinct frozen model and matched baseline; in Phase 6 only for the 480 frozen checkpoints and the pre-fine-tuning masks, each pre-fine-tuning accuracy having to equal the Phase-5 record; in Paper 3, 7A and 7B never. No design choice used test accuracy.

Two disclosures. (1) **Phase 6 is not a fully independent confirmation**, because the test sets had been read in Phases 2–5 for earlier frozen models. (2) **Validation-to-test transfer is measured, not assumed:** Phase 5 records each selected policy's feasibility separately on VAL-RL, VAL-SELECT and test, with q_{test} = test / dense test, and Phase 6 tests whether fine-tuning changes the gap between VAL-SELECT and test retention.

4.20 Chapter Summary

The design fixes two validation configurations, a test set read only for frozen models, exact zero-count matching, one PPO configuration and one 5-epoch recovery protocol, pre-registered tests including a ±0.5 pp confidence-interval rule that is not a formal equivalence test, and measurement gates that stopped the Phase-7B measurements before any paired comparison.

CHAPTER 5
RESULTS OF THE CORE FRAMEWORK

5.1 General

This chapter reports the results of the core framework, Phases 3–6, and answers Research Questions 1–5; the deployment extension is reported in Chapter 6. Each section gives the question, the experiment (designed in Chapter 4), the numerical and statistical result, a short interpretation and a claim boundary stating what the result does *not* show. Broader interpretation across phases is left to Chapter 7.

Reporting conventions (from Chapter 4):

(a) **Accuracy and differences.** Accuracies are percentages, and differences are percentage points (pp). Every accuracy names its split:
   (i) "test" is the official 10,000-image test set;
   (ii) "VAL-RL (I)" and "VAL-SELECT (I)" are the 3,000 / 2,000 validation configuration of Phases 3–4;
   (iii) "VAL-RL" and "VAL-SELECT" without a numeral are the 2,500 / 2,500 configuration of Phases 5–6.

(b) **Statistics.** Comparisons are paired by seed (n = 20 per setting) and use Holm-adjusted exact two-sided sign-flip p-values. A result is **supported** when its direction matches the hypothesis, the Holm $p < 0.05$ and the 95% t-interval excludes 0; **opposite**, **inconclusive** and the cross-setting classes STRONG, MODERATE, LIMITED, NOT SUPPORTED and REVERSED are defined in Section 4.15.1.

(c) **Matching.** Baselines are built at the identical zero count of the PPO model they are compared with, that is, at each PPO run's *own* sparsity.

(d) **Sparsity and settings.** Sparsity ρ is masked weight sparsity (Equation 3.2). Settings are written C10 and C100 for CIFAR-10 and CIFAR-100.

5.2 Dense Reference Performance

Table 5.1 gives the six dense references against which every pruned model is compared.

Table 5.1: Dense reference models (accuracy %, seed-0 reference).

Dataset	Architecture	Full validation	Test	Test, 3 seeds (mean ± SD)	VAL-RL (I)	VAL-RL (II)
CIFAR-10	SimpleCNN†	77.32	77.03	— (1 model)	76.97	76.88
CIFAR-10	LeNet-5	64.32	64.96	66.21 ± 1.51	64.30	64.20
CIFAR-10	ResNet-8	78.72	78.69	78.69 ± 0.51	78.53	78.92
CIFAR-100	SimpleCNN	49.08	50.50	49.84 ± 0.65	50.37	48.96
CIFAR-100	LeNet-5	32.58	32.28	31.42 ± 0.75	33.30	32.76
CIFAR-100	ResNet-8	43.48	43.99	43.58 ± 0.53	43.77	43.76

† The historical 10-epoch checkpoint, not the 30-epoch Phase-2 recipe (Section 4.5).

Across the three training seeds, test accuracy varies by at most about 1.5 pp (SD). The fixed seed-0 reference of C10 LeNet-5 lies below its three-seed mean; it was kept because the reference rule is fixed in advance. The two VAL-RL columns are the denominators A_b of retained accuracy q in Phases 3–4 and in Phase 5.

5.3 Phase 3 — PPO versus Baselines

Question (RQ1). Does a PPO agent learn layer-wise allocations that are more accurate than uniform allocation and other one-shot baselines at the same sparsity?

Evidence. Phase 3 ran 480 PPO runs: 6 settings × 4 conditions × seeds 300–319. The P0 (no-prior) final policy of each run was compared on test accuracy with Uniform, Random, Global magnitude, ERK (Evci et al., 2020) and LAMP (Lee et al., 2021) (Section 4.7), each built at the run's exact zero count. No model was fine-tuned.

Sparsity context. PPO chose its own sparsity. The mean final sparsity of P0 was 55.06% (C10 SimpleCNN), 51.54% (C10 LeNet-5), 28.35% (C10 ResNet-8), 53.28% (C100 SimpleCNN), 34.59% (C100 LeNet-5) and 9.46% (C100 ResNet-8). Phase 3 is therefore not a single fixed-sparsity comparison. The range is 9.5–55.1%, and the SimpleCNN networks were pruned roughly six times as heavily as CIFAR-100 ResNet-8.

Table 5.2: Phase 3: P0 minus baseline, test accuracy (pp, mean of 20 paired runs). "+" = supported, "−" = opposite, "·" = inconclusive.

Setting (P0 test)	Uniform	Random	Global	ERK	LAMP
C10 SimpleCNN (76.44)	+31.37 +	+59.60 +	−0.07 ·	+0.75 +	−0.55 −
C10 LeNet-5 (61.67)	+16.29 +	+48.69 +	+2.74 +	+1.93 +	−1.64 −
C10 ResNet-8 (76.95)	+14.17 +	+64.36 +	+7.38 +	−0.06 ·	+0.15 ·
C100 SimpleCNN (49.66)	+29.69 +	+47.29 +	−0.01 ·	+4.35 +	−0.65 −
C100 LeNet-5 (31.18)	+3.67 +	+27.89 +	−0.13 ·	+0.30 ·	−0.68 −
C100 ResNet-8 (43.01)	+0.23 ·	+38.76 +	+0.28 +	−0.80 −	−0.75 −
Supported / opposite (of 6)	**5 / 0**	**6 / 0**	**3 / 0**	**3 / 1**	**0 / 5**
Class	**STRONG**	**STRONG**	**MODERATE**	**LIMITED**	**NOT SUPPORTED, REVERSED**

Uniform and Random.

(a) **Uniform: STRONG** (5 of 6 settings). Every supported setting has Holm $p = 1.1 \times 10^{-5}$, and PPO won in 20 of 20 seeds in each.

  (i) The largest gain, **+31.37 pp** (exact mean 31.365 pp, ≈ +31.4 pp; 95% CI 26.57–36.16), is on **CIFAR-10 SimpleCNN, test accuracy**. There, at 55.06% mean sparsity, uniform pruning fell to 45.08% while PPO kept 76.44%.

  (ii) This maximum is specific to that setting (Table 5.2).

(b) **C100 ResNet-8 is inconclusive** (+0.23 pp). PPO chose the uniform policy [1, 1, 1, 1] itself in 19 of 20 runs, so 19 pairs are ties.

(c) **Random: STRONG** (6 of 6), from +27.89 to +64.36 pp (Holm $p = 4.6 \times 10^{-5}$ in every setting).

Global magnitude and ERK.

(a) **Global: MODERATE.** PPO is better in 3 settings (Holm $p \leq 0.0015$) and within ±0.13 pp in the other three (inconclusive).

(b) **ERK: LIMITED.** PPO is better in 3 settings (Holm $p \leq 0.0087$), inconclusive in 2, and worse in C100 ResNet-8 (−0.80 pp, Holm $p = 5.3 \times 10^{-5}$).

LAMP. LAMP is significantly more accurate than PPO in **5 of 6 settings**, by 0.55–1.64 pp (Holm $p \leq 0.002$; LAMP won 19–20 of 20 seeds in four of them). C10 ResNet-8 is

inconclusive (+0.15 pp). The statement "PPO > LAMP" is NOT SUPPORTED and, being opposite in five settings, REVERSED.

Allocations. PPO's final policies are non-uniform (Figure 5.2):

(a) **SimpleCNN:** it prunes conv3 and the large hidden layer at 40–60% and keeps the first convolution at 0–20%.

(b) **LeNet-5:** the two fully connected layers absorb most of the pruning.

(c) **C10 ResNet-8:** stage 3 is pruned most.

Uniform, by contrast, must apply the same ratio to the small, sensitive first layers.

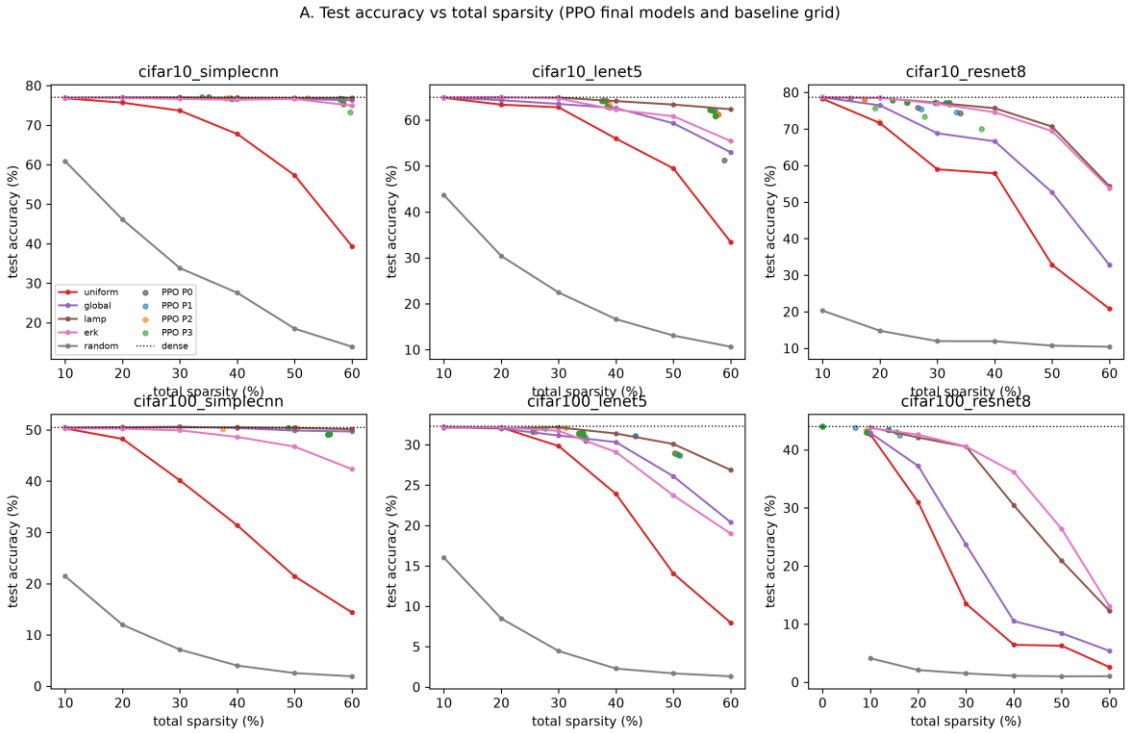


Fig. 5.1: Test accuracy against total sparsity (Phase 3). Lines: the five baselines on a fixed descriptive sparsity grid (10–60%); points: the final models of the 20 PPO runs per condition (P0–P3); dotted line: dense test accuracy. The statistical comparison in Table 5.2 uses baselines built at each PPO run's own zero count.

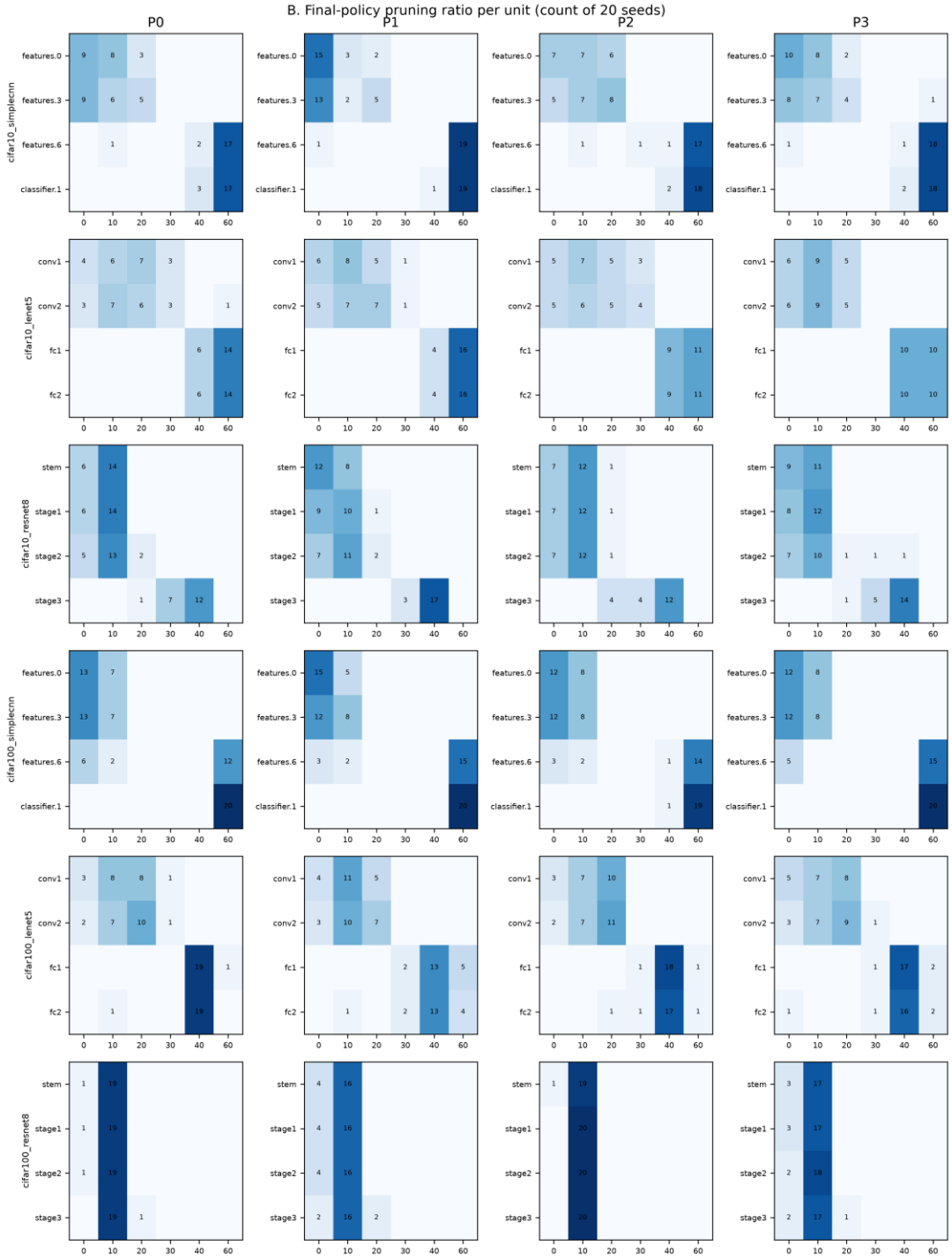


Fig. 5.2: Final-policy pruning ratio per unit (Phase 3). Each cell is the count of the 20 seeds that chose that ratio for that unit, by setting (rows) and condition (columns P0–P3). In Phase 3, P2 is the *shuffled* prior.

Interpretation. The result supports the first part of RQ1: PPO's non-uniform allocations are far more accurate than uniform or random allocation at the same sparsity. Against the stronger magnitude-based baselines the advantage is mixed; against LAMP it is reversed.

Claim boundary.

(a) The comparison is before any fine-tuning, at the sparsity PPO chose.

(b) It does not show that PPO beats one-shot magnitude-based allocation in general.

(c) It does not show that PPO matches LAMP.

(d) It says nothing about uniform pruning after fine-tuning: Phase 6 did not include Uniform.

5.4 Phase 3 — Effect of Sensitivity Guidance

Question (RQ2, first test). Does the sensitivity prior improve PPO's search and the final model?

Evidence. Four conditions were compared on the configuration-I VAL-RL landscape and on test: P0 (no prior), P1 (the sensitivity prior), P2 (the shuffled prior) and P3 (the constant prior). The primary exploration endpoint is the VAL-RL (I) accuracy AUC over the 512 sampled policies. The final-model endpoint is the test accuracy in excess of matched global magnitude, compared as P1 − P0.

Results (Table 5.11, Phase-3 rows):

(a) **Primary exploration endpoint (P1 − P0, VAL-RL (I) accuracy AUC): MODERATE**, supported in 3 of 6 settings: C10 SimpleCNN (+0.46, Holm p = 0.002), C100 SimpleCNN (+0.35, Holm p = 0.0098) and C100 ResNet-8 (+0.54, Holm p = 5.7×10^{-5}). The other three settings are inconclusive, and none is opposite.

(b) **Secondary exploration endpoints: STRONG in 6 of 6** settings for each of:

  (i) reward-rank AUC: +0.013 to +0.032 in rank score, Holm $p \leq 5.5 \times 10^{-4}$;

  (ii) utility AUC: Holm $p \leq 3.6 \times 10^{-4}$;

  (iii) destructive-policy frequency: lower under P1, by 0.9–3.8 percentage points of sampled policies, Holm $p \leq 0.0022$.

(c) **Layer specificity** (P1 better than both the shuffled and the constant prior): **LIMITED**, supported only in C10 SimpleCNN.

(d) **Final model: NOT SUPPORTED**, 0 of 6 supported and 0 opposite. The P1 − P0 differences in excess over global magnitude range from −0.10 to +0.86 pp, and every Holm $p \geq 0.094$. The raw test-accuracy differences are also all inconclusive.

Control insight. On LeNet-5 the shuffled and constant priors did at least as well as the correct prior:

(a) **C10 LeNet-5, VAL-RL (I) accuracy AUC:** P1 − P2 = −0.21 (Holm p = 0.27); P1 − P3 = −0.21 (Holm p = 0.31).

(b) **Descriptive mean test accuracy:** C10 LeNet-5: P1 62.17%, P2 62.54%, P3 62.83%; C100 LeNet-5: P1 30.81%, P2 31.15%, P3 31.05%.

All three Phase-3 priors carry the same total pressure towards lighter pruning, $\Sigma c_l = 0.5 \cdot \Sigma S_l$ (Section 3.10). The pattern therefore raises the possibility that part of the exploration effect comes from a generic conservative bias rather than from layer-specific information. The Phase-3 data cannot separate the two, and they do not prove either. This possibility motivated the centred prior of Phase 4.

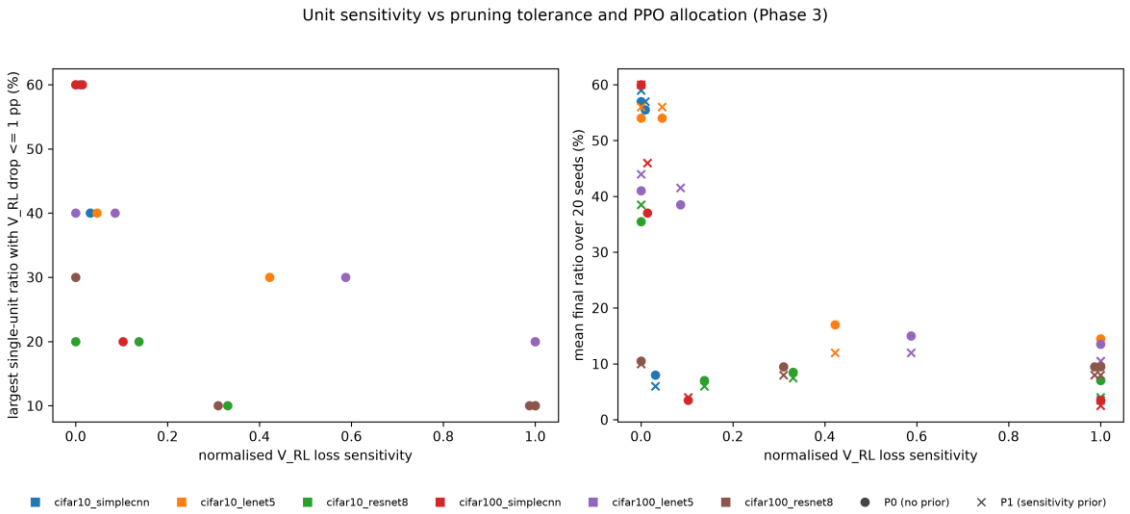


Fig. 5.3: Unit sensitivity, pruning tolerance and PPO allocation (Phase 3). Left: for each unit, the largest single-unit ratio whose VAL-RL (I) accuracy drop is ≤ 1 pp, against normalised loss sensitivity S_l. Right: mean final ratio over 20 seeds of P0 (circles) and P1 (crosses) against S_l. Each setting has one colour in both panels. Highly sensitive units tolerate only 10–20% pruning and receive low ratios under both conditions.

Interpretation. The sensitivity prior changes *how* PPO searches (better-ranked policies sooner, fewer destructive ones), not *what* it returns, in a way detectable on test accuracy.

Claim boundary. "Sensitivity improves pruned-model accuracy" is not supported. The layer-specific part of the effect is not established by Phase 3.

5.5 Phase 4 — Centred Sensitivity and Exploration

Question (RQ2, second test). Without net conservatism, does *relative* sensitivity still help the search?

Evidence. Phase 4 Stage B ran 600 runs under R0 with seeds 400–419. It compared the centred prior P2, $c_l = 1.0 \cdot (S_l - \bar{S})$, whose coefficients sum to zero, against P3 (the centred values permuted across units), P4 (a magnitude-matched pure-conservatism prior), P5 (the sign-reversed prior) and P0 (no prior). The primary endpoint was the reward-rank AUC on the configuration-I VAL-RL landscape.

Results (Table 5.11, Phase-4 rows):

(a) **Sensitivity-specific statement** (P2 better than both P3 and P4): **STRONG**, supported in 5 of 6 settings and opposite in none.

(i) P2 − P3 ranges from +0.018 to +0.097 in rank-score AUC (Holm $p \leq 3.4 \times 10^{-4}$, all 6 supported).

(ii) P2 − P4 ranges from +0.028 to +0.070 in the five supported settings.

(iii) In C100 ResNet-8, P2 − P4 = −0.002, which is inconclusive.

(b) **Against the sign-reversed prior** (P2 − P5): supported in **6 of 6** settings, by +0.021 to +0.145 (Holm $p = 1.1 \times 10^{-5}$).

(c) **Against no prior:** P1 − P0 and P2 − P0 were each supported in 6 of 6.

(d) **One exploration measure was reversed.** On the VAL-RL (I) accuracy AUC, the statement "P2 better than P3 and P4" is NOT SUPPORTED and REVERSED. P2 was significantly *below* the magnitude-matched conservative prior P4 in 4 of 6 settings (C10 LeNet-5, C10 ResNet-8, C100 LeNet-5 and C100 ResNet-8; mean −0.26 to −0.76). The centred prior pushes aggressive actions onto the insensitive units, so the policies it samples are better ranked under R0 but less accurate on average. Reward rank and accuracy are different quantities under a trade-off reward, and the improvement holds for the former only.

(e) **Other secondary endpoints:** destructive frequency MODERATE (4 of 6); utility AUC LIMITED (5 supported, 1 opposite); best-seen rank LIMITED (1 of 6); final-selected rank NOT SUPPORTED (0 of 6).

Interpretation. With the shared conservatism removed, correctly assigned relative sensitivity still makes the explored policies better ranked under the reward, which is stronger evidence for a layer-specific effect than Phase 3 provided. The benefit concerns the *average* explored policy, not the best policy found or the policy returned.

Claim boundary.

(a) The centred prior is not shown to improve accuracy-based exploration in Phase 4 (that endpoint is reversed).
(b) It is not shown to improve the final model.

5.6 Phase 4 — Policy Retention and Archive Selection

Question (RQ3). Does PPO end on the good policies it visits, and does archive-based selection change this?

Evidence. Phase 4 Stage C compared four search variants under R0 and P2, each with 20 seeds per setting:

(a) **S0:** the terminal deterministic policy;
(b) **S1:** archive selection over the S0 runs;
(c) **S2:** elite behavioural cloning ($K = 5$, $\eta = 0.1$) with the terminal policy;
(d) **S2A:** elite behavioural cloning with archive selection.

The archive rule takes the top 10 visited policies by VAL-RL (I) reward and returns the one with the highest VAL-SELECT (I) score. Ranks are positions among all 1,296 policies on the VAL-RL (I) landscape of R0 (1 = optimum).

Table 5.3: Retention: the best policy encountered, the terminal policy (S0) and the archive-selected policy (S2A). Reach is the share of the 20 runs whose policy is in the top-k of the 1,296-policy VAL-RL (I) landscape. Ranks are medians over runs. Δ test is S2A − S0 test accuracy (pp, mean).

Setting	A: best seen, top-10 reach (S0)	B: terminal (S0), top-10 reach	B: median rank	C: archive (S2A), top-10 reach	C: top-20 reach	C: median rank	Δ test (S2A − S0)
C10 SimpleCNN	100%	0%	92	100%	100%	3	+0.14
C10 LeNet-5	100%	0%	69	100%	100%	6	**+0.89**
C10 ResNet-8	95%	0%	96	100%	100%	1	**−5.66**
C100 SimpleCNN	100%	0%	65	100%	100%	6	+0.22
C100 LeNet-5	100%	5%	173	85%	100%	7	+0.45
C100 ResNet-8	95%	0%	114	80%	100%	10	**−4.00**

Results.

(a) **A. PPO visits top policies.** The best policy seen has a median rank of 1–2.5 and is in the top 10 in 95–100% of runs (Phase 3: median best-seen rank 1–4.5).

(b) **B. PPO does not end on them.** The terminal policy reaches the top 10 in **0% of runs in five settings and 5% in the sixth** (C100 LeNet-5). Its median rank is 65–173, and it reaches the top 20 in at most 5% of runs. In Phase 3, the terminal P0 policy reached the top 10 in 0% of runs in all six settings.

(c) **C. The archive retains them.** With S2A, the selected policy reaches the top 10 in **80–100%** of runs and the top 20 in 100%, with a median rank of 1–10.

   (i) **Statistics:** "S2A better than S0 on final-selected VAL-RL (I) rank" is STRONG, supported in 6 of 6 settings. The mean rank improves by 66–150 positions (Holm $p = 3.4 \times 10^{-5}$). The same holds on VAL-SELECT (I) rank, 6 of 6.

   (ii) **Archive alone (S1):** the archive rule without behavioural cloning gives 75–100% top-10 reach and is likewise supported in 6 of 6.

Contribution of elite behavioural cloning. Behavioural cloning alone (S2) improved the final-selected rank in only 3 of 6 settings (MODERATE) and the best-seen rank in 1 of 6 (LIMITED). The retention gain is therefore mainly a *selection* effect of the archive, not an effect of cloning or of the sensitivity prior.

Test accuracy is mixed. The archive improves the *validation rank* of the returned policy, but its test-accuracy effect (Table 5.3, last column) varies: small gains in four settings

(+0.14, +0.89, +0.22 and +0.45 pp) and **large losses on ResNet-8: −5.66 pp (C10) and −4.00 pp (C100).** The archive-only variant S1 shows the same pattern. On ResNet-8, the better-ranked policies are much sparser than the terminal ones: 32.2% → 47.8% (C10) and 8.5% → 27.7% (C100) mean sparsity for S0 → S2A.

The pre-registered Stage-C criterion, the VAL-SELECT (I) rank of the selected policy, favours the archive variants by construction, because they select on VAL-SELECT (I). The test comparison does not share this bias.

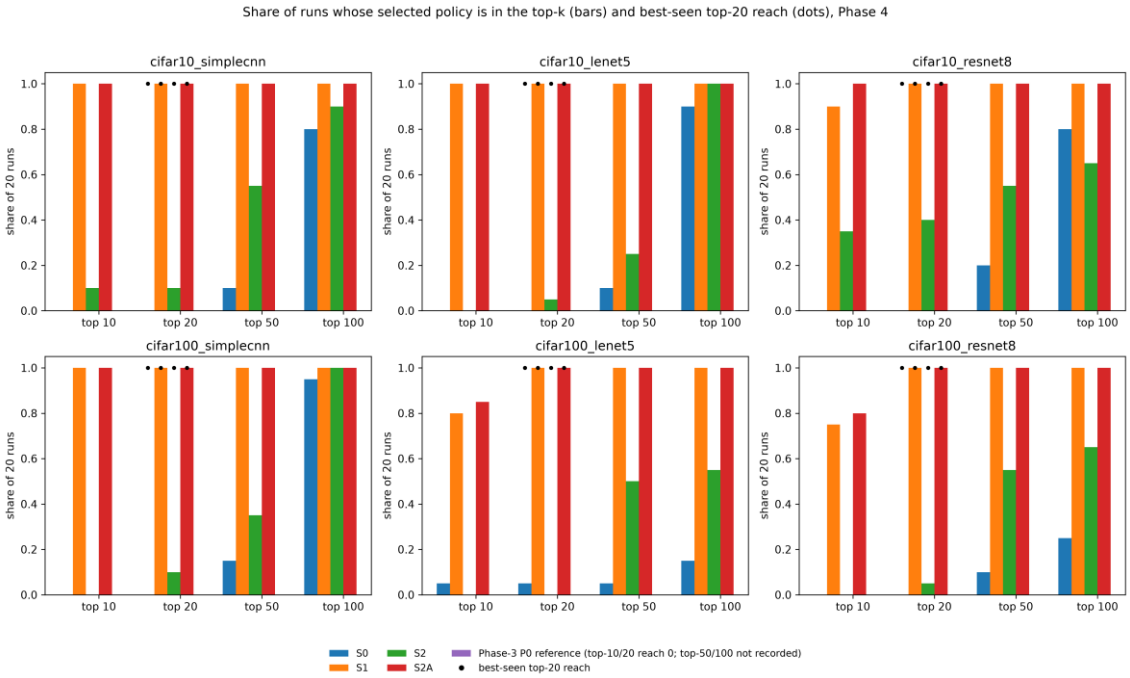


Fig. 5.4: Share of runs whose selected policy lies in the top-k of the VAL-RL (I) landscape (Phase 4, Stage C). Bars: S0, S1, S2 and S2A. Dots: best-seen top-20 reach, which is 100% for every variant. The Phase-3 P0 reference series shows no bars: its top-10 and top-20 reach is 0, and top-50/100 reach was not recorded.

Interpretation. Exploration is not the bottleneck; retention is. PPO finds near-optimal policies early but does not end on them, and an archive of visited policies recovers them on validation ranking.

Claim boundary.

(a) Archive selection does **not** consistently improve test accuracy. It lost 4–6 pp on ResNet-8, because the policies it retains are those the reward ranks highest, and that reward is misaligned (Section 5.7).

(b) The retention result is a validation-ranking result.

5.7 Phase 4 — Reward Misalignment

Question (RQ4, diagnosis). Once search and retention work, does the reward point PPO towards acceptable models?

Evidence. Stage A tested three redesigned trade-off rewards against R0. The final pipeline F = (R0, P2, S2A), fixed by the pre-registered stage selections, was compared with the control K = (R0, P0, S0), the Phase-3 configuration on fresh seeds.

Redesigned rewards (criterion A: NOT MET). No redesigned reward met both pre-registered conditions:

(a) **R1a (common 50% target):** reduced the cross-setting spread of final sparsity to 0.29× that of R0. However, it kept ≥ 95% VAL-RL (I) retention in only 2 of 6 settings; C100 ResNet-8 retained 67.5%.

(b) **R1b (95%-retention targets):** kept retention but did not reduce the spread (1.05×).

(c) **R2 (anchor-normalised):** reached 0.66× and 4 of 6.

R0 was therefore carried forward unchanged.

R0's own optimum. On the VAL-RL (I) landscapes, R0's optimum lies at **23.5–56.4% sparsity**, depending on the setting. On ResNet-8 that optimum is an accuracy-losing policy: 48.8% sparsity retaining 89.7% of dense VAL-RL (I) accuracy (C10), and 23.5% sparsity retaining 93.4% (C100).

Better search finds that optimum (Table 5.5, Phase-4 rows). On ResNet-8, F's archive selection reached R0's optimum region, with median selected VAL-RL (I) rank 1 (C10) and 10 (C100) and mean sparsity rising from 29.0% to 47.8% (C10) and from 8.3% to 27.7% (C100) relative to K. Test accuracy fell accordingly: **C10:** 76.82% → 70.83%, a **7.86 pp** drop below dense; **C100:** 43.11% → 39.22%, a **4.77 pp** drop.

Final pipeline against the control and LAMP.

(a) **Criterion D, F > K on test excess over matched LAMP: NOT MET (LIMITED).** Supported in C10 SimpleCNN (+0.50), C10 LeNet-5 (+0.95) and C100 LeNet-5 (+0.67 pp); **opposite on both ResNet-8 settings** (−0.95 and −0.92 pp); inconclusive in C100 SimpleCNN.

(b) **Criterion E, LAMP competitiveness: met only as non-inferiority.**

(i) LAMP is significantly better than F in **6 of 6** settings: F − LAMP = −0.11, −0.11, −0.68, −0.48, −0.08 and −1.62 pp (Holm $p \leq 0.012$).

(ii) The pre-registered one-sided non-inferiority test with a 0.5 pp margin is supported in 3 settings: C10 SimpleCNN, C10 LeNet-5 and C100 LeNet-5 (Holm $p = 5.7 \times 10^{-6}$).

(c) F remains far ahead of Uniform: +12.05 to +36.04 pp at matched zero count.

Table 5.4: Phase 4 pre-registered criteria.

Criterion	Status
A. Reward	**NOT MET**
B. Sensitivity-specific prior	**MET** (STRONG)
C. Search	**MET** (STRONG for S1 and S2A)
D. Final model vs control	**NOT MET** (LIMITED)
E. LAMP competitiveness	**MET only through non-inferiority** in 3 settings; "F > LAMP" NOT SUPPORTED

By its own rule, Phase 4 counted as successful if at least one criterion held. That rule is satisfied, but the phase is not a success on its two model-level criteria (A and D).

Interpretation. The optimiser did what the reward specified. R0 trades accuracy for sparsity at a fixed rate (Section 3.13.1); once search and retention improved, PPO found R0's optimum, which on fragile networks is an over-pruned model.

Claim boundary. This is evidence of reward misalignment, not a failure of PPO alone. It concerns R0 and these landscapes. It does not show that every trade-off reward over-prunes.

5.8 Phase 5 — Accuracy-Constrained Reward

Question (RQ4, remedy; RQ2, third test). Does an accuracy-constrained, validation-aware reward remove the over-pruning? Does sensitivity guidance still help under it?

Evidence. Phase 5 ran 360 runs on configuration II with seeds 500–519, under three conditions:

(a) **C0:** R0 + centred prior + R0-score archive selection (the Phase-4 pipeline without behavioural cloning);

(b) **C1:** constrained reward R_c with $\tau = 0.98$ + centred prior + constraint-aware selection;

(c) **C2:** as C1 without the prior.

All three were evaluated once on test, after the policy list was frozen.

Table 5.5: ResNet-8 reward alignment across Phases 4 and 5 (test accuracy %, mean of 20 runs; sparsity = mean selected masked sparsity).

Setting	Configuration	Sparsity	Test	Drop vs dense (pp)
C10 ResNet-8 (dense 78.69)	Phase-4 K (R0, no prior, terminal)	29.0%	76.82	1.87
	Phase-4 F (R0, P2, S2A)	47.8%	70.83	7.86
	Phase-5 C0 (R0, P2, archive)	46.2%	71.37	**7.32**
	Phase-5 C1 (R_c, P2, constrained selection)	24.5%	77.51	**1.18**
	Phase-5 C2 (R_c, no prior)	23.9%	77.57	1.12
C100 ResNet-8 (dense 43.99)	Phase-4 K	8.3%	43.11	0.88
	Phase-4 F	27.7%	39.22	4.77
	Phase-5 C0	25.9%	39.94	4.05
	Phase-5 C1	15.6%	42.83	1.16
	Phase-5 C2	15.5%	42.90	1.09

The Phase-4 and Phase-5 rows use different validation configurations but share the test set and dense reference.

Over-pruning on ResNet-8. On **CIFAR-10 ResNet-8**, the constrained reward reduced the mean test-accuracy loss from **7.32 pp (C0) to 1.18 pp (C1)**, and on **CIFAR-100 ResNet-8** from 4.05 to 1.16 pp, at much lower sparsity (Table 5.5). On test, C1 − C0 was +6.14 pp (95% CI 4.92–7.36; 20 of 20 seeds favouring C1; Holm $p = 1.1 \times 10^{-5}$) on C10 ResNet-8 and +2.89 pp (95% CI 2.30–3.48; 20 of 20; Holm $p = 1.1 \times 10^{-5}$) on C100 ResNet-8. C1's selected policies lie at or next to the constrained optimum: median constrained rank 3 (C10) and 1 (C100).

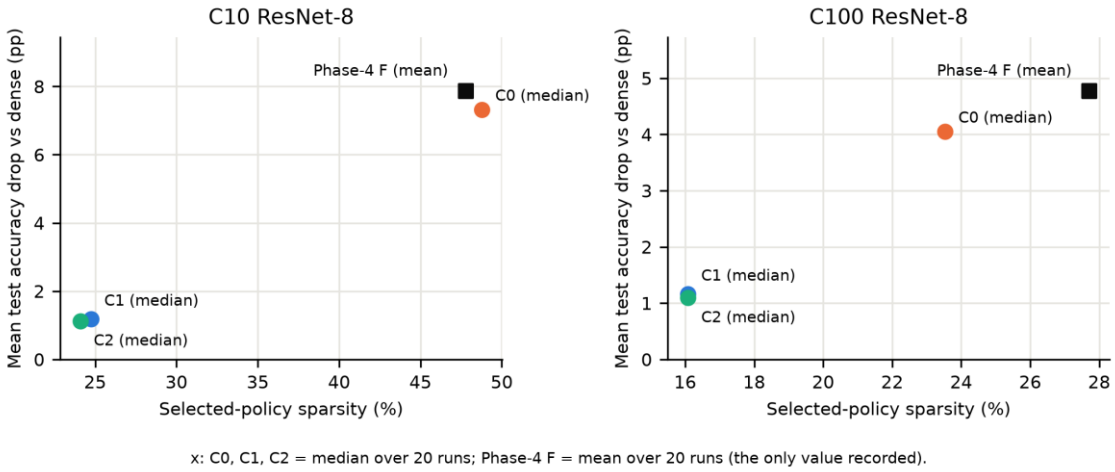


Fig. 5.5: ResNet-8 reward alignment: test-accuracy drop below dense against selected-policy sparsity.

(a) C0, C1 and C2 are plotted at their **median** sparsity (48.8%, 24.7% and 24.1% for C10; 23.5%, 16.1% and 16.1% for C100).

(b) The Phase-4 F reference is plotted at its **mean** sparsity (47.8% and 27.7%), the only value recorded; each label states its statistic.

Effect across all six settings (Table 5.6).

(a) **C1 − C0 on test accuracy:** supported in 4 settings, LeNet-5 (+0.70 and +1.12 pp) and ResNet-8; **opposite in 2:** C10 SimpleCNN (−0.31 pp, Holm $p = 4.6 \times 10^{-5}$) and C100 SimpleCNN (−0.21 pp, Holm $p = 0.0022$).

(b) Where C1 is more accurate, it prunes 10–22 pp less than C0. On SimpleCNN, C0 already chose near-safe policies, and C1 spends the 2% accuracy allowance on slightly more sparsity.

(c) Safe sparsity is architecture-dependent: C1's mean sparsity runs from 15.6% (C100 ResNet-8) to 56.9% (C10 SimpleCNN).

Table 5.6: Phase 5 by setting (test accuracy %, mean of 20 runs; differences in pp).

Setting	C0 test (sparsity)	C1 test (sparsity)	C2 test	C1 − C0	C1 − LAMP	C1 − Global
C10 SimpleCNN	76.75 (56.1%)	76.45 (56.9%)	76.50	−0.31 −	−0.49 −	−0.09 ·
C10 LeNet-5	63.14 (53.9%)	63.83 (38.6%)	63.99	+0.70 +	−0.32 −	+1.42 +
C10 ResNet-8	71.37 (46.2%)	77.51 (24.5%)	77.57	+6.14 +	−0.53 −	+6.51 +
C100 SimpleCNN	50.28 (52.9%)	50.07 (53.1%)	50.13	−0.21 −	−0.29 −	+0.32 +
C100 LeNet-5	30.67 (46.3%)	31.79 (30.6%)	31.86	+1.12 +	−0.26 −	+0.36 +
C100 ResNet-8	39.94 (25.9%)	42.83 (15.6%)	42.90	+2.89 +	−0.51 −	+3.48 +

Comparison with global magnitude (descriptive by pre-registration). C1 is more accurate than global magnitude at the identical zero count in **5 of 6** settings, by +0.32 to +6.51 pp on test (sign-flip Holm $p \leq 0.0011$). C10 SimpleCNN is inconclusive (−0.09 pp). The comparison is not confirmatory.

Comparison with LAMP.

(a) LAMP is significantly more accurate than C1 in **all 6** settings, by **0.26–0.53 pp** on test (Holm $p \leq 5.9 \times 10^{-5}$; LAMP won 16–20 of 20 seeds).

(b) The gap is within 0.5 pp in four settings. In both ResNet-8 settings it is 0.51–0.53 pp.

(c) Compared with C0, the gap to LAMP narrowed in C100 ResNet-8 (+0.66 pp, supported), widened on both SimpleCNN settings (opposite), and was unchanged elsewhere.

Sensitivity under the corrected reward (C1 − C2; Table 5.11, Phase-5 rows):

(a) **Primary family** (24 tests): **0 opposite outcomes.**

  (i) Reward-rank AUC: supported in 5 of 6 settings (Holm $p \leq 0.019$; C10 ResNet-8 inconclusive).

  (ii) VAL-RL accuracy AUC: supported in 5 of 6 (Holm $p \leq 0.012$; C100 ResNet-8 inconclusive).

  (iii) Episodes to the first top-20 and top-10 policy: inconclusive in all six settings, although every point estimate favours C1.

(b) **Secondary:** destructive-action rate lower in 6 of 6 (Holm $p \leq 1.5 \times 10^{-4}$); selected rank better in 3 of 6; best-seen rank better in 1 of 6.

(c) **Final test accuracy** (descriptive): C1 is **0.05–0.16 pp below** C2 in all six settings. The C1 − C2 test accuracy comparison was not pre-registered as a test.

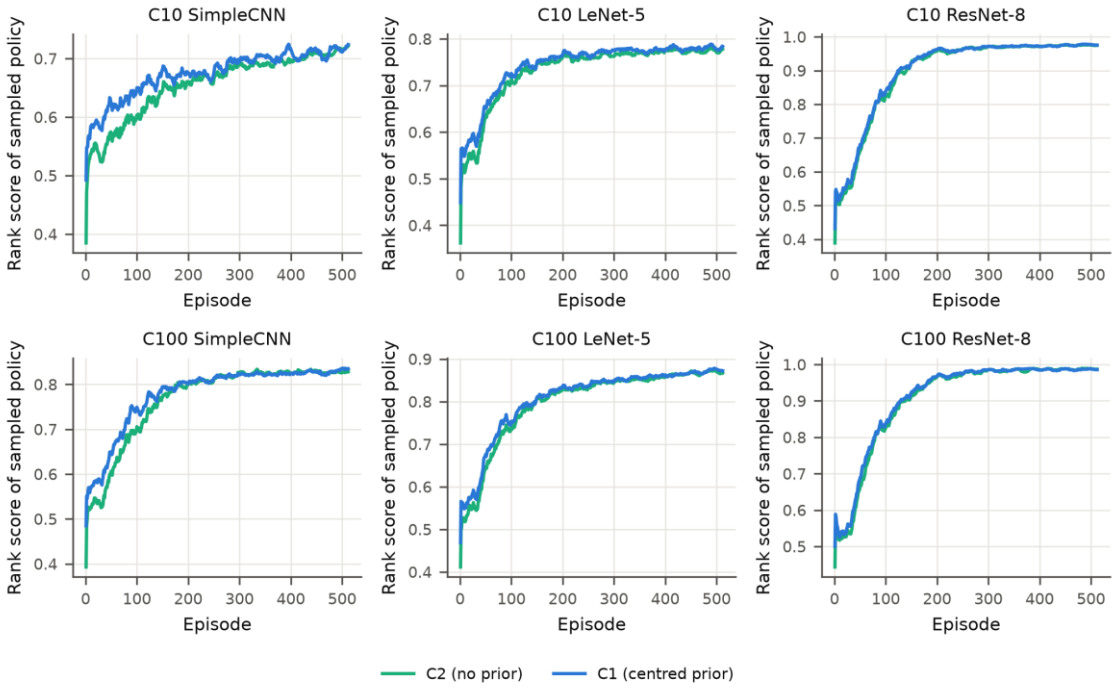


Fig. 5.6: Rank score of the sampled policy over 512 episodes, C1 (centred prior) vs C2 (no prior) (Phase 5, VAL-RL). The mean over 20 seeds, smoothed with a 16-episode rolling mean. The prior's advantage is concentrated early in training; in most settings the curves meet before the end.

Table 5.7: Phase 5 pre-registered criteria.

Criterion	Status	Reason
A. Accuracy safety	**PARTIALLY MET**	Supported in both ResNet-8 settings, but opposite in 2 other settings (at most 1 allowed)
B. Non-trivial compression	**MET**	C1 median sparsity ≥ 10% in 6 of 6 and ≥ 20% in 5 (4 required)
C. Search quality	**PARTIALLY MET**	Median selected constrained rank ≤ 20 in 5 of 6, but only 76.7% of runs selected a top-20 policy (≥ 80% required); 90.8% visited one
D. Sensitivity contribution	**MET**	A primary exploration endpoint supported in every setting, 0 of 24 opposite
E. Final model improvement	**PARTIALLY MET**	Supported in 4 settings, opposite in 2 (no degradation allowed)
F-strong. LAMP	**NOT MET**	C1 never beats LAMP; LAMP better in 6 of 6
F-weak. LAMP	**MET**	Within 0.5 pp in 4 settings, none worse than 1.0 pp

Interpretation. Making accuracy a constraint rather than a traded quantity largely removed the ResNet-8 over-pruning; the remaining effects are small and architecture-dependent. Sensitivity guidance still improves exploration and still does not improve the final model.

Claim boundary.

(a) The constrained reward does not "guarantee" accuracy (Section 5.9).

(b) It does not eliminate over-pruning in every setting.

(c) It does not make PPO better than LAMP.

(d) The comparison with global magnitude is descriptive.

5.9 Phase 5 — Validation-to-Test Transfer

Question. Does a policy feasible on validation data remain feasible on test?

Evidence. For every selected policy, Phase 5 recorded whether $q \geq 0.98$ held on three splits: on VAL-RL, where the reward enforced it; on VAL-SELECT, where the selection rule enforced it; and on test, measured as test accuracy divided by dense test accuracy.

Table 5.8: Validation-to-test feasibility of the selected Phase-5 policies ($q \geq 0.98$; runs out of 120).

Condition	Feasible on VAL-RL	Feasible on VAL-SELECT	Feasible on test
C0	40 / 120	44 / 120	40 / 120 (33.3%)
C1	**120 / 120**	**119 / 120 (99.2%)**	**81 / 120 (67.5%)**
C2	120 / 120	119 / 120	90 / 120 (75.0%)

(a) **Selection fallback.** The single C1 policy infeasible on VAL-SELECT is the one selection-rule fallback (C100 LeNet-5).

(b) **Transfer failures.** Of the 119 C1 policies feasible on VAL-SELECT, **39 were infeasible on test.**

(c) **C1 test feasibility by setting:** 100% in both SimpleCNN settings and in C10 ResNet-8; 60% in C100 LeNet-5; 45% in C10 LeNet-5; **0% in C100 ResNet-8**, where the mean test retention was 0.974, just below the 0.98 threshold.

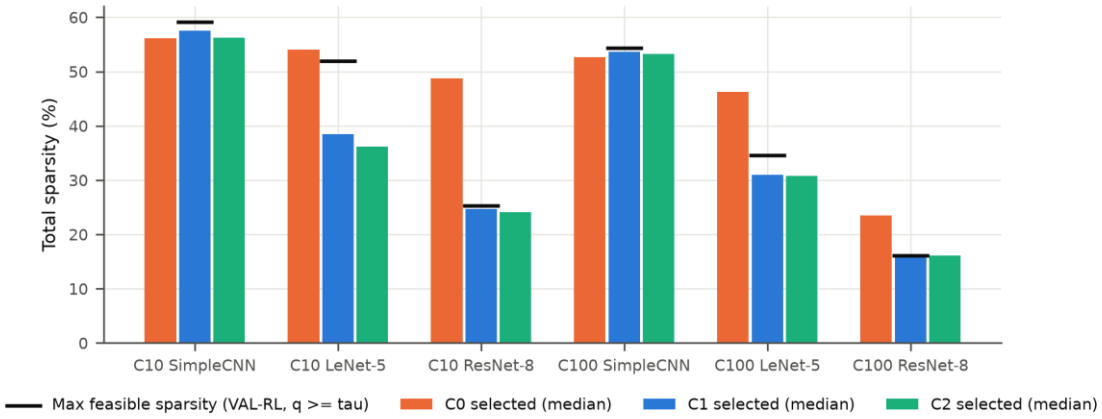


Fig. 5.7: Median selected sparsity of C0, C1 and C2 against the maximum feasible sparsity on VAL-RL ($q \geq \tau$), by setting (Phase 5). C0 selects policies beyond the feasible maximum in the LeNet-5 and ResNet-8 settings. C1 and C2 select at or just below it; these boundary policies are where test feasibility is most fragile.

Interpretation. Validation feasibility is evidence about test feasibility, not a guarantee. The failures occur where C1 selects close to the VAL-RL feasibility boundary (Figure 5.7). The frozen Phase-5 analysis notes that two 2,500-image splits carry roughly ±1 pp of sampling noise, comparable to the 2% relative margin in these settings.

Claim boundary. No claim of the form "the validation constraint guarantees generalisation" is made. Phase 6 shows that equal fine-tuning removes most of this gap (Section 5.10).

5.10 Phase 6 — Equal Fine-Tuning

Question (RQ5). Do the differences between PPO and the strongest deterministic methods persist when every pruned model receives the same fine-tuning?

Evidence.

(a) **Methods.** Phase 6 fine-tuned three pruned models in each of 120 (setting, policy seed) pairs: **PPO**, the frozen Phase-5 C1 policy; and **LAMP and global magnitude**, at exactly PPO's zero count (mismatch 0 in 120 of 120 triples). A dense control was fine-tuned alongside them. **Phase 6 contains no Uniform baseline.**

(b) **Protocol.** Every model received 5 epochs of fixed-mask Adam fine-tuning at learning rate 10^{-4} with shared seed and data order (Section 4.13); the model after epoch 5 was evaluated once on test. There were 480 runs, plus 6 reproduction runs.

Table 5.9: Phase 6: test accuracy before → after equal fine-tuning, and the PPO–LAMP and PPO–Global gaps (pp; mean of 20 paired runs; 95% t-CI).

Setting (sparsity)	PPO	LAMP	Global	PPO − LAMP before	PPO − LAMP after [95% CI]	PPO − Global after [95% CI]
C10 SimpleCNN (56.9%)	76.45 → 79.73	76.94 → 79.95	76.54 → 79.89	−0.49	−0.22 [−0.28, −0.15]	−0.16 [−0.21, −0.11]
C10 LeNet-5 (38.6%)	63.83 → 64.88	64.15 → 65.06	62.41 → 64.83	−0.32	−0.18 [−0.28, −0.09]	+0.05 [−0.05, 0.15]
C10 ResNet-8 (24.5%)	77.51 → 78.74	78.03 → 78.73	71.00 → 78.67	−0.53	+0.00 [−0.02, 0.03]	+0.07 [0.03, 0.11]
C100 SimpleCNN (53.1%)	50.07 → 50.53	50.36 → 50.58	49.75 → 50.51	−0.29	−0.05 [−0.09, −0.01]	+0.03 [−0.02, 0.07]
C100 LeNet-5 (30.6%)	31.79 → 32.36	32.05 → 32.49	31.43 → 32.24	−0.26	−0.14 [−0.19, −0.08]	+0.12 [0.06, 0.18]
C100 ResNet-8 (15.6%)	42.83 → 44.02	43.34 → 44.06	39.35 → 44.04	−0.51	−0.04 [−0.07, −0.01]	−0.02 [−0.07, 0.03]
Mean of 6 settings	**57.08 → 58.38**	**57.48 → 58.48**	**55.08 → 58.36**			

The dense control moved from 57.91 to 58.51 (mean over settings).

Recovery.

(a) **Every method recovered in every setting.** "Fine-tuning improves accuracy over the same mask before fine-tuning" is supported in 18 of 18 method × setting tests (Holm $p < 10^{-4}$).

(b) **Recovery beyond the dense control's own gain** is supported in 17 of 18. The exception is LAMP on C10 SimpleCNN, which gained 0.04 pp less than the dense control.

(c) **Mean recovery per setting:** PPO: 0.46–3.28 pp; LAMP: 0.22–3.01 pp; global magnitude: 0.76–7.67 pp. The ordering, global > PPO > LAMP in all six settings, is the inverse of the pre-fine-tuning accuracy: the more a mask had lost, the more it recovered.

(d) **Shortfall after fine-tuning.** Every method ended within 0.14 pp of the dense reference on average. On C10 SimpleCNN, every model, dense control included, ended about 3 pp *above* the historical reference: the dense control itself gained +3.05 pp from the 5 extra epochs (Table 5.1, note †).

PPO versus LAMP.

(a) **Before fine-tuning,** LAMP led by 0.26–0.53 pp in every setting.

(b) **After fine-tuning,** the absolute PPO–LAMP difference was **0.00–0.22 pp.**

(c) **Change in the gap (ΔG).** The gap moved in PPO's favour, a supported result, in **5 of 6** settings (Holm $p \leq 0.027$). C10 LeNet-5 was inconclusive (Holm $p = 0.055$), and no setting was opposite.

(d) **Who is ahead after fine-tuning.** LAMP was still significantly ahead in **3 of 6** settings: C10 SimpleCNN, −0.22 pp (Holm $p = 3.8 \times 10^{-5}$); C10 LeNet-5, −0.18 pp (Holm $p = 0.0026$); C100 LeNet-5, −0.14 pp (Holm $p = 4.6 \times 10^{-4}$). The other three were inconclusive. PPO was significantly ahead in none.

(e) **The ±0.5 pp rule. The post-recovery PPO–LAMP differences fell within the pre-registered ±0.5 pp decision margin in all six settings.** The rule is that the 95% t-interval lies inside (−0.5, +0.5) pp. It is a confidence-interval decision rule, not a formal statistical equivalence test (Section 4.15.2). Three of the six settings were already inside the margin before fine-tuning (C10 LeNet-5, C100 SimpleCNN and C100 LeNet-5).

PPO versus global magnitude.

(a) **Before fine-tuning,** PPO led global magnitude by up to 6.51 pp (Section 5.8).

(b) **After equal recovery,** the differences were −0.16 to +0.12 pp. PPO's largest remaining advantage was **0.12 pp** (C100 LeNet-5; supported); PPO was also ahead by 0.07 pp in C10 ResNet-8 (supported); global was ahead by 0.16 pp in C10 SimpleCNN (opposite); the other three settings were inconclusive.

(c) **Gap change and margin.** The change in the PPO–Global gap was opposite in 5 settings, because global recovered much more, and inconclusive on C10 SimpleCNN, where there had been no gap to close. Every post-recovery PPO–Global interval lies inside ±0.5 pp.

Fixed masks and integrity. No pruned weight regrew: in all 360 pruned runs, per-layer zero counts were identical before and after fine-tuning, the mask hash was unchanged, and no masked weight was non-zero. The 6 reproduction runs were bit-identical to the originals, and all 20 integrity checks passed.

Table 5.10: Phase 6 pre-registered criteria.

Criterion	Status	Basis
A. Fairness	**MET**	one protocol and epoch count; shared seed and data stream in every pair; zero-count mismatch 0 in 120 of 120 triples; masks equal the frozen table
B. Recovery	**MET**	recovery supported for all three pruning methods in all 6 settings
C. PPO competitiveness (change of the PPO–LAMP gap)	**MET**	gap moved towards PPO in 5 settings, towards LAMP in none. C concerns the change of the gap, not who is ahead: LAMP remains ahead in 3 settings
D. Architecture generalisation	**MET**	all six settings complete and present in every family
E. Sparsity preservation	**MET**	identical per-layer zero counts and unchanged masks in 360 of 360 pruned runs
F. Reproducibility	**MET**	480 of 480 runs complete; 6 of 6 reproductions bit-identical

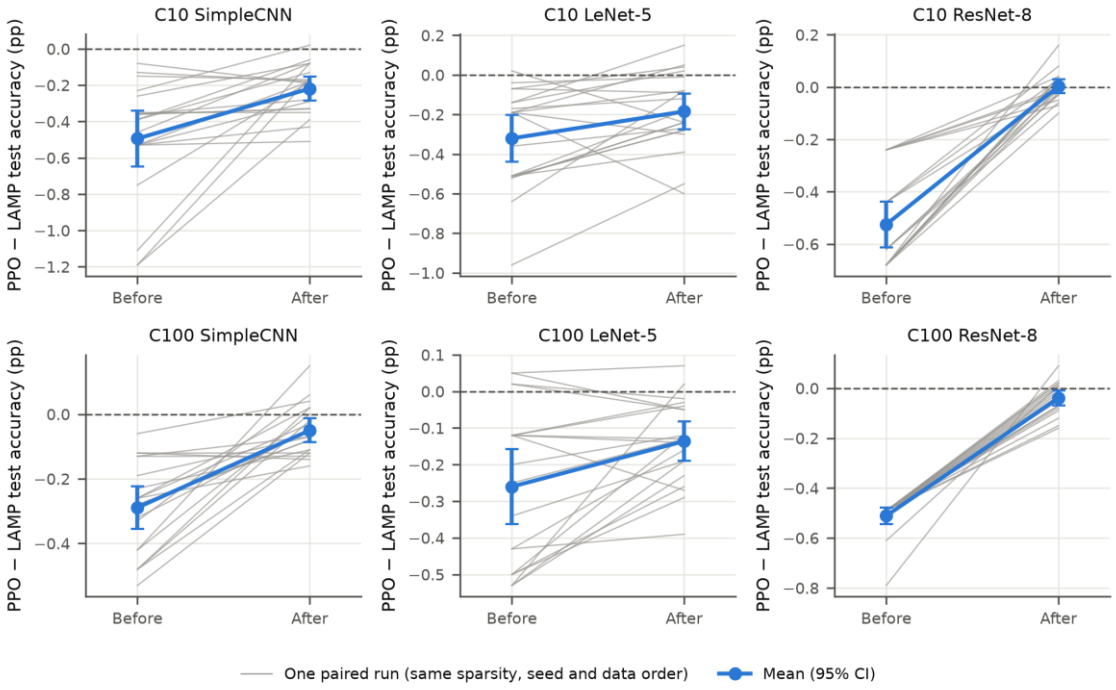


Fig. 5.8: PPO − LAMP test accuracy before and after equal 5-epoch fine-tuning (Phase 6). Grey lines: one paired run (same sparsity, seed and data order); blue: mean with 95% CI; dashed line: zero difference. The ±0.5 pp margin is not drawn; every post-fine-tuning interval lies inside it (Table 5.9).

Interpretation. Equal, modest fine-tuning substantially shrinks the differences among pruning methods at these sparsities: PPO's advantage over global magnitude and LAMP's advantage over PPO both lay mainly *before* fine-tuning. It also removed most of the validation-to-test retention problem of Section 5.9: after recovery, every pruned model

met retention ≥ 0.98 on test, and the PPO retention gap between VAL-SELECT and test narrowed significantly in 4 of 6 settings.

Claim boundary.

(a) PPO and LAMP are not claimed to be statistically equivalent, and LAMP remains significantly, though slightly, ahead in three settings.

(b) The result holds for these networks, sparsities and a 5-epoch budget.

(c) It says nothing about Uniform, ERK or Random after fine-tuning.

5.11 Answers to RQ1–RQ5

Table 5.11: Sensitivity guidance across Phases 3–5: exploration endpoints versus final-model endpoints (settings supported / opposite, of 6).

Phase	Comparison	Exploration endpoints	Final-model endpoint
3	P1 − P0 (prior vs none)	VAL-RL (I) accuracy AUC 3 / 0 (MODERATE); rank AUC 6 / 0; utility AUC 6 / 0; destructive frequency 6 / 0 (STRONG)	excess over matched Global 0 / 0 (NOT SUPPORTED)
3	P1 vs P2 and P3 (specificity)	VAL-RL (I) accuracy AUC 1 / 0 (LIMITED)	—
4	P2 vs P3 and P4 (centred, specific)	rank AUC 5 / 0 (STRONG); VAL-RL (I) accuracy AUC 0 / 4 (REVERSED); destructive 4 / 0	final-selected rank 0 / 0 (NOT SUPPORTED)
4	P2 − P5 (sign-reversed)	rank AUC 6 / 0	—
5	C1 − C2 (prior vs none, constrained reward)	rank AUC 5 / 0; VAL-RL accuracy AUC 5 / 0; time to top-20 / top-10 0 / 0; destructive rate 6 / 0	test accuracy: C1 0.05–0.16 pp below C2 in 6 of 6 (descriptive)

(Table 5.11 is placed here, beside the answer to RQ2, because it draws on Sections 5.4, 5.5 and 5.8.)

RQ1: PPO against uniform and one-shot baselines. Yes, with clear limits. PPO's non-uniform allocations substantially outperform uniform allocation (5 of 6 settings, up to +31.4 pp on C10 SimpleCNN test) and random allocation (6 of 6) at the sparsity PPO chose; against global magnitude and ERK the advantage is mixed. LAMP is significantly more accurate than PPO before recovery: in 5 of 6 settings in Phase 3, and in 6 of 6 for the Phase-4 final pipeline and for Phase-5 C1.

RQ2: sensitivity guidance. It improves the search but not the final model (Table 5.11): better-ranked policies earlier and fewer destructive ones in every phase, also against

equally conservative controls (Phase 4) and under the corrected reward (Phase 5). No final-accuracy benefit was found, and one Phase-4 exploration measure was reversed.

RQ3: retention and archive selection. The terminal PPO policy does not retain the strong policies PPO visits (Section 5.6). Archive selection restores retention on validation ranking, mainly as an effect of the archive rule, not of behavioural cloning. The effect on test accuracy is mixed, with large losses on ResNet-8 under the misaligned reward.

RQ4: reward design. Reward design materially determines how aggressively PPO prunes. R0's optimum over-prunes fragile networks (ResNet-8 lost up to 7.86 pp once search improved). The accuracy-constrained reward largely removed this loss (Table 5.5), with mixed effects elsewhere, and validation feasibility transferred to test in only 67.5% of runs.

RQ5: equal recovery. Equal 5-epoch fixed-mask fine-tuning substantially narrows the differences. The PPO–LAMP gap fell from 0.26–0.53 pp to 0.00–0.22 pp, within the pre-registered ±0.5 pp confidence-interval margin in all six settings, with LAMP still significantly ahead in three; PPO's advantage over global magnitude fell to at most 0.12 pp. PPO's distinctive advantages lay before fine-tuning.

5.12 Chapter Summary

Findings in the order the phases produced them:

(i) **Phase 3.** PPO strongly beat uniform and random pruning, but LAMP was more accurate in five of six settings.

(ii) **Sensitivity.** The prior made the search more efficient in every phase but never improved the final model.

(iii) **Phase 4.** PPO visited top policies but ended on poor ones; an archive recovered them on validation ranking. Better search then exposed a misaligned reward that over-pruned ResNet-8.

(iv) **Phase 5.** An accuracy constraint largely corrected the over-pruning. Validation feasibility transferred to test in only two thirds of runs.

(v) **Phase 6.** After identical fine-tuning the differences between methods became small; LAMP remained slightly ahead in three settings, and PPO's lead over global magnitude essentially disappeared.

Negative and partial results are reported alongside the positive ones: LAMP reversed against PPO (Phase 3); criteria A and D not met, ResNet-8 test losses from archive selection and one reversed exploration measure (Phase 4); criteria A, C and E partially met and F-strong not met (Phase 5); LAMP still ahead in three settings (Phase 6).

Figures. Figures 5.1, 5.2 and 5.6–5.8 are the frozen phase figures, used unchanged. Figures 5.3–5.5 are presentation-only thesis re-renders of the frozen Phase-3 figure D, Phase-4 figure D and Phase-5 figure B, from the same frozen CSVs, with every plotted value verified identical (`figures/rerender_ch5_figures.py`).

Chapter 6 asks whether these results translate into deployment efficiency; Chapter 7 discusses them.

CHAPTER 6
DEPLOYMENT AND SUSTAINABILITY ANALYSIS

6.1 General

Chapter 5 evaluated masked pruning, where compression is measured as weight sparsity. This chapter asks the question the core framework cannot answer:

RQ6. Does model compression translate into real deployment benefit?

The evidence comes from four later studies (Chapter 4, Table 4.7): **Paper 3**, which measures the CPU latency of every structured policy (Sections 6.4–6.6); **Phase 7A**, which realises the learned Phase-5 allocation as a physically smaller network and measures its CPU package energy (Sections 6.7–6.8); **Phase 7B**, which searches directly in the structured space (Sections 6.9–6.12); and the **energy confirmation** study of the frozen Phase-7B model (Section 6.13).

Deployment benefit is analysed at three levels (Table 3.6): compression and memory; computation and latency; and operational energy. The central message is that **an improvement at one level does not guarantee an improvement at the next.** Each level needs its own measurement, and the accuracy cost must be stated alongside it.

Reporting conventions:

(a) Accuracies are validation accuracies on configuration II (VAL-RL / VAL-SELECT, 2,500 images each), after the equal 5-epoch recovery. No test-set number exists for these phases (Section 4.3).

(b) Phase 7A's pre-registered accuracy split is **VAL-RL**; Phase 7B's is **VAL-SELECT**. Both are reported, and the split is always named.

(c) Latency and energy were measured on the laptop CPU of Table 4.11 (Sections 4.17–4.18).

6.2 From Compression to Deployment Efficiency

The core framework produced masked models. A masked network keeps every tensor at its dense shape and stores zeros in place of removed weights. On a general-purpose CPU with dense kernels, those zeros are still loaded and multiplied unless sparse formats and

kernels are used (Hoefler et al., 2021). Weight sparsity is therefore a statement about the model, not about its cost on hardware, and cost indicators such as parameter count, FLOPs and measured speed can disagree with one another (Dehghani et al., 2022). This thesis makes no latency or energy claim for its masked models.

To reach the hardware, the extension realises each policy **structurally**: whole channels and neurons are removed, and a smaller dense network is built (He, Zhang and Sun, 2017; Li et al., 2017; He and Xiao, 2024). The three levels then become measurable: parameter count, parameter memory and serialised size exactly from the model (Level 1); MACs in closed form, checked against hooks, and measured batch-1 CPU latency (Level 2); and CPU package energy per 1,000 inferences from the RAPL hardware counter (Khan et al., 2018), with no TDP model or estimate (Level 3).

6.3 Structured Physical Pruning Setup

The structured search space is the core space realised physically (Section 4.6.2):

(a) **Action set and policy count:** the same six ratios per unit, so there are 1,296 policies per network.

(b) **Keep rule:** each unit keeps max(1, round((1 − r)·C)) channels, those with the largest L1 norm on the dense model.

(c) **SimpleCNN units:** the three convolutions' filters and the hidden layer's neurons.

(d) **ResNet-8 units:** the stem is a **coupled residual pruning group**, which sets the stage-1 residual width across every tensor that writes or reads it; the three stage units remove **block-internal channels**.

(e) **Integrity:** every compacted model was checked to contain no masks, to have analytic parameter and MAC counts, and to reproduce the masked-equivalent logits within 10^{-4}.

Recovery. Accuracy always follows the equal 5-epoch Phase-6 protocol, with recovery seed 600 for every model.

Latency protocol (Section 4.17). Calibration chose configuration A (batch 1, one thread, one performance core); its dense-minus-aggressive latency difference had a signal-to-noise ratio of 12.3 and a relative reduction of 61.6%, passing the thresholds of 5 and 10%.

Each table was timed in five fresh-process sessions with dense anchors and a 10% drift limit.

6.4 Exhaustive CPU Latency Analysis

Paper 3 timed **all 1,296 physically compacted policies** in each of four tables: SimpleCNN and ResNet-8, each with 10 outputs (CIFAR-10) and with 100 outputs (CIFAR-100). That is **5,184 structured policies** in total. Latency depends only on shape, so the tables need no trained weights and no accuracy evaluation. Four session attempts exceeded the drift limit and were repeated; all 20 accepted sessions met it.

Measurement stability. The median between-session coefficient of variation per shape was 1.2–1.6%; the Spearman correlation of latency gains between any two sessions was 0.96–0.995; and raw and anchor-normalised gains agreed at Spearman ≥ 0.997.

Relative latency. The absolute dense latency on the table days was about 16% higher than on the calibration day, while the dense anchors within each table agreed with its identity shape to within 1%. The analysis therefore uses latency *relative to the identity shape of the same table*.

Table 6.1: Exhaustive structured latency tables: cost metrics against measured latency gain (Spearman rank correlation over 1,296 policies per table; latency gain $g_T = 1 - T/T_identity$).

Table	Policies	Parameter reduction vs latency gain	MAC reduction vs latency gain	Pairs where the more parameter-sparse policy is slower	Maximum speed-up (unconstrained)
SimpleCNN, CIFAR-10	1,296	0.32	0.89	39.0%	2.61×
SimpleCNN, CIFAR-100	1,296	0.31	0.89	39.4%	2.59×
ResNet-8, CIFAR-10	1,296	0.55	0.98	30.6%	1.67×
ResNet-8, CIFAR-100	1,296	0.54	0.98	30.8%	1.72×

The maximum speed-up in every table belongs to the most aggressive policy [5, 5, 5, 5]. These maxima describe the latency space only. They are **not** accuracy-preserving speed-ups: no accuracy constraint is applied in Table 6.1 (Section 6.6).

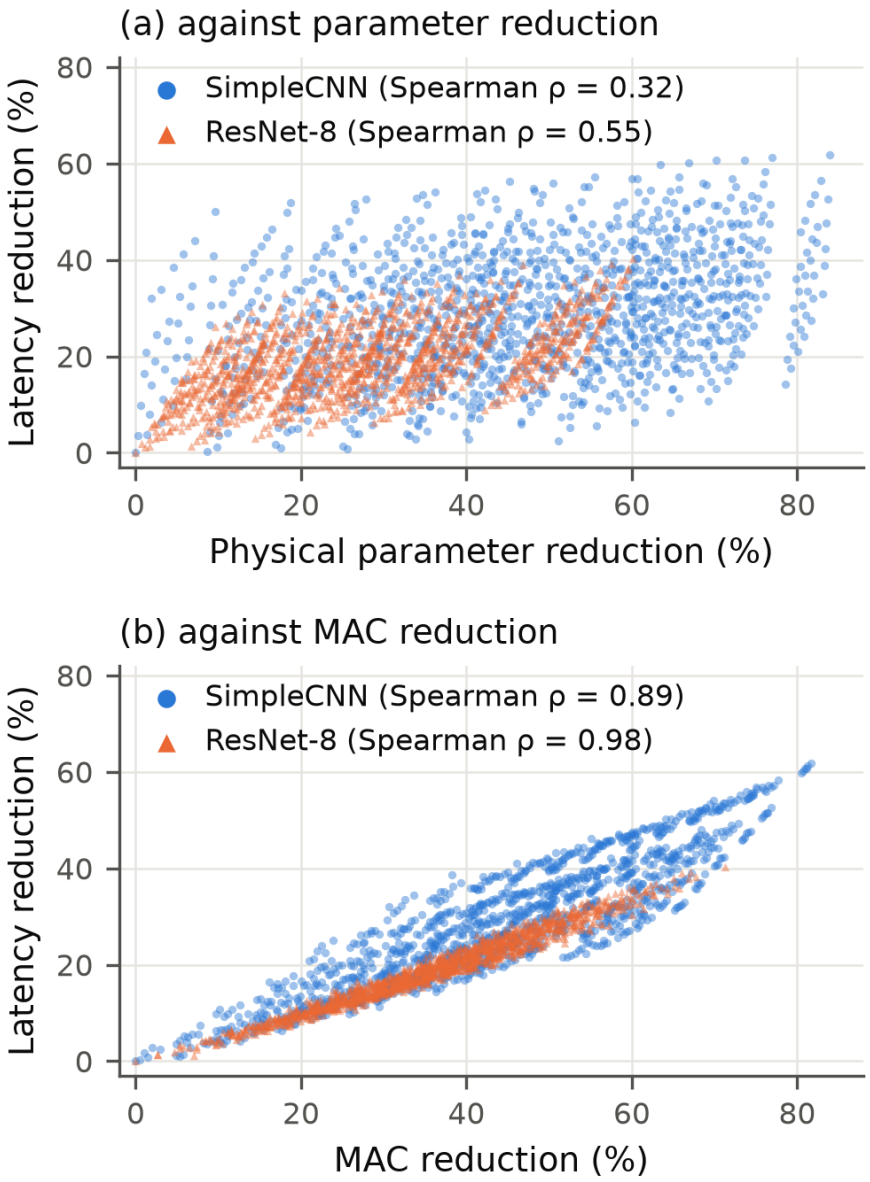


Fig. 6.1: Measured latency reduction against (a) physical parameter reduction and (b) MAC reduction, for all 1,296 structured policies of the CIFAR-10 SimpleCNN and ResNet-8 tables (Paper 3). The CIFAR-100 tables (Table 6.1) behave the same way.

6.5 Parameter Reduction versus MAC Reduction versus Latency

Result. Across all four tables (Table 6.1), the Spearman correlation of latency gain was 0.31–0.55 (weak) with parameter reduction and 0.89–0.98 (strong) with MAC reduction. In **39.0–39.4%** of SimpleCNN policy pairs and **30.6–30.8%** of ResNet-8 pairs, the policy with more parameter reduction was *slower*.

Example (CIFAR-10 SimpleCNN table). Two policies show why the location of pruning matters more than the parameter count.

Table 6.2: Two CIFAR-10 SimpleCNN policies: parameter reduction, MAC reduction and measured speed-up.

Policy	What it prunes	Parameter reduction	MAC reduction	Speed-up
[5, 5, 1, 0]	the first two convolutions at 60%	18.7%	69.3%	**2.08**×
[0, 0, 0, 5] (fc-heavy)	only the 256-unit hidden layer at 60%	51.1%	2.9%	**1.03**×

In SimpleCNN, the hidden layer holds most of the weights but little of the arithmetic, while the early convolutions have few weights operating at high spatial resolution; the fc-heavy policy therefore removes almost three times the parameters for almost no time. The examples describe cost only: pruning the first convolutions at 60% is destructive in this network (Section 5.4).

ResNet-8 correlates more strongly with parameters (0.54–0.55 vs 0.31–0.32) but still has roughly one inverted pair in three.

Interpretation. Parameter sparsity is a weak proxy for CPU latency, and MAC reduction a strong one. Measured latency remains the deployment-grounded quantity, because even MACs leave part of the ordering unexplained (Spearman $\rho < 1$). The same mechanism reappears in Phase 7A (Section 6.8).

Claim boundary. These are batch-1, single-thread results on one laptop CPU. They do not transfer to other CPUs, thread counts, batch sizes or accelerators. No correlation is reported other than Spearman's.

6.6 Accuracy-Feasible Latency Headroom

Table 6.1 shows how much faster a structured network *can* be; deployment needs the speed-up of policies that also keep accuracy.

Feasibility criterion. Paper 3 used the Phase-5 criterion, $q_5 \geq 0.98$ in retained VAL-RL accuracy after the 5-epoch recovery. The exhaustive 5-epoch landscape was stopped for computational cost (Section 4.6.3), so a 1-epoch accuracy proxy was validated first and accepted only if its Spearman correlation with 5-epoch recovery was ≥ 0.90.

Table 6.3: Validation of the 1-epoch accuracy proxy against 5-epoch recovery.

Architecture	Validation pairs	Spearman $\rho(A_1, A_5)$	Decision
SimpleCNN	156	0.844	**rejected** (NO-GO)
ResNet-8	31	0.980	accepted (GO)

Because the SimpleCNN proxy was rejected, **no accuracy-feasibility claim is made for the SimpleCNN latency tables.** Its 2.61× maximum is an unconstrained figure only.

ResNet-8 calibration. The ResNet-8 validation pairs contained too few truly feasible policies for the pre-registered threshold rule, so its fallback applied. A pre-committed calibration then recovered 48 frozen ResNet-8 policies for the full 5 epochs, and the proxy threshold was set by a recall-first rule with a fixed 0.005 margin. The resulting accuracy-feasible latency headroom is given in Table 6.4.

Table 6.4: ResNet-8 (CIFAR-10): calibrated accuracy-feasible latency headroom.

Item	Value
Calibration policies (5-epoch recovered)	48
Truly 5-epoch feasible ($q_5 \geq 0.98$)	14
Proxy threshold τ_{proxy}	0.960
Recall / precision at τ_{proxy}	1.00 / 0.875 (14 true positives, 2 false positives)
Proxy-feasible policies in the 1,296 landscape	52
Their parameter / MAC reduction	0–28.9% / 0–18.4%
Best table speed-up among proxy-feasible policies	**1.098×**
Best speed-up among the 14 truly feasible calibration policies	1.089× ([0, 3, 1, 1])
Proxy-feasible policies with speed-up ≥ 1.10×	0

The 52 proxy-feasible policies are *candidates*, not all truly feasible: only the 48 calibration policies were confirmed by 5-epoch recovery.

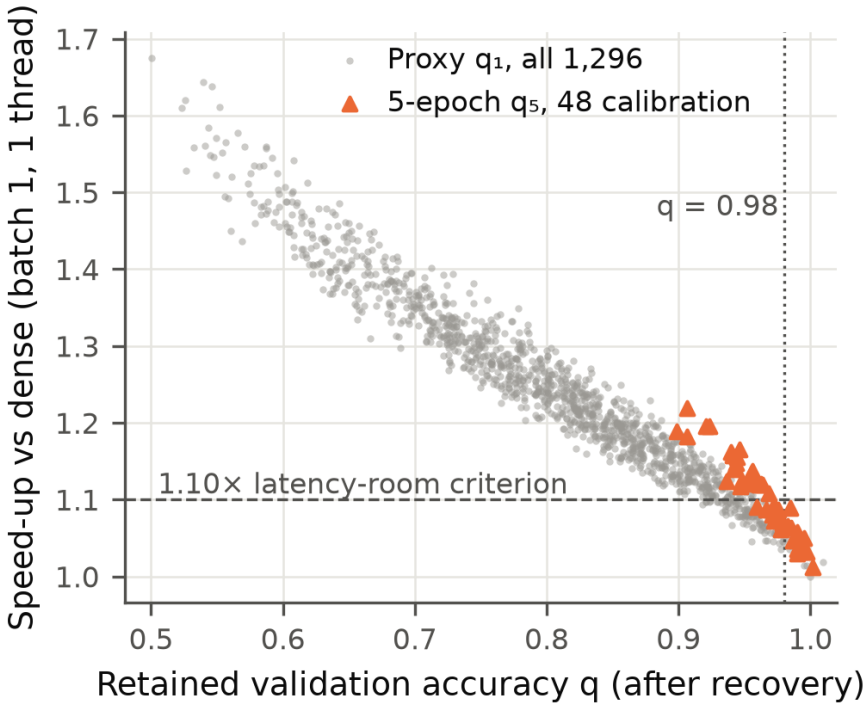


Fig. 6.2: ResNet-8 (CIFAR-10): table speed-up against retained validation accuracy (Paper 3). Grey: the 1-epoch proxy q_1 for all 1,296 policies. Orange: 5-epoch q_5 for the 48 calibration policies. The dotted line is q = 0.98; the dashed line is the 1.10× latency-room criterion.

Decision. The pre-registered latency-room rule required at least 20 proxy-feasible policies **and** at least one with a speed-up ≥ 1.10×. The first condition held (52); the second did not (best 1.098×). **Because the latency-room requirement was not met, no hardware-aware PPO search was started.**

The gap between 1.098× and 1.10× is a protocol decision threshold fixed in advance, not a scientifically meaningful discontinuity. The substantive finding concerns the evidence available under the 2% accuracy constraint: the best proxy-feasible candidate reached a 1.098× speed-up, while the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither satisfied the pre-registered 1.10× GO criterion. The accuracy-feasible latency room found on this CPU is therefore small, and for SimpleCNN it could not be established at all, because its proxy failed.

Claim boundary. No hardware-aware reinforcement-learning result exists. Paper 3 contributes a hardware analysis, not an RL success.

6.7 Phase 7A — Structured Transfer and Energy Evaluation

Question. Does the learned PPO allocation, realised structurally, reduce directly measured CPU energy, and at what accuracy cost?

Design (Section 4.9.6). CIFAR-10, SimpleCNN and ResNet-8, four models each:

(a) **Dense:** the reference, recovered identically.

(b) **PPO-Structured:** the most frequent Phase-5 C1 policy, transferred to structured units: SimpleCNN [0, 2, 5, 5] (3 of 20 seeds), widths [32, 51, 51, 102]; ResNet-8 [1, 1, 1, 3] (9 of 20 seeds), widths [14, 14, 29, 45]. The ResNet-8 mapping is approximate.

(c) **Uniform-Structured:** MAC-matched to PPO, giving common ratios 0.256 and 0.149.

(d) **LAMP-Structured:** from the Phase-6 LAMP mask at PPO's zero count.

Measurement (Section 4.18). After the identical 5-epoch recovery, RAPL package energy was measured over five fresh-process sessions per architecture, with the dense anchor in every session and 100,000 batch-1 inferences per model. The stability pilot passed its 3% rule (dense CV 2.30% for SimpleCNN and 1.74% for ResNet-8).

Table 6.5: Phase 7A: accuracy, compression, latency and measured energy. Accuracy after the equal 5-epoch recovery; Δ in pp against the recovered dense model on VAL-RL, the pre-registered split. Latency, energy and EDP are medians over 5 sessions, paired with the same-session dense anchor.

Architecture	Method	VAL-RL (Δ)	VAL-SELECT	Params ↓	MACs ↓	Latency (ms)	Speed-up	J / 1,000 inf.	Energy ↓	EDP
SimpleCNN	Dense	80.60 (0)	80.00	—	—	0.7313	1.00×	11.05	—	1.000
	PPO-Structured	75.52 (−5.08)	76.00	80.1%	42.6%	0.5711	1.28×	8.54	22.8%	0.603
	Uniform-Structured	79.12 (−1.48)	78.48	44.7%	42.5%	0.5341	1.37×	7.91	27.9%	0.518
	LAMP-Structured	78.84 (−1.76)	79.44	65.5%	19.7%	0.6579	1.12×	9.76	12.1%	0.783
ResNet-8	Dense	78.40 (0)	78.52	—	—	0.6984	1.00×	10.88	—	1.000
	PPO-Structured	75.16 (−3.24)	74.72	25.0%	21.5%	0.6400	1.09×	9.59	13.4%	0.791
	Uniform-Structured	75.60 (−2.80)	76.00	16.1%	19.3%	0.6459	1.07×	9.64	11.8%	0.818
	LAMP-Structured	75.24 (−3.16)	75.72	25.0%	18.2%	0.6490	1.07×	9.82	8.9%	0.828

Energy results.

(a) **Every pruned model used less package energy than the dense model in 5 of 5 sessions.** For each, the exact one-sided Wilcoxon signed-rank test gives $p = 0.031$, the smallest value attainable with five sessions.

(b) **The p-value is secondary.** The pre-registered decision rests on the 5-of-5 consistency and on the median saving exceeding the noise threshold of twice the dense between-session coefficient of variation (4.3% for SimpleCNN; 3.7% for ResNet-8). All six pruned models met both conditions.

(c) PPO's per-session saving ranged from 19.5% to 25.7% on SimpleCNN and from 5.6% to 14.5% on ResNet-8, and the core counter and net-of-idle energy agreed with the package result to within 2 percentage points.

Latency and EDP. Every pruned model was faster than the same-session dense model, and speed-ups track MAC reduction far more closely than parameter reduction (Table 6.5); at nearly equal MACs Uniform was still faster than PPO, so MACs do not explain the whole difference either. Median package power was 14.6–15.6 W for every model. The normalised energy–delay product (EDP; lower is better) combines energy and latency relative to the same-session dense model. It is not a carbon metric.

6.8 Phase 7A — Accuracy–Energy Trade-off

Accuracy cost. After identical recovery, every pruned model lost VAL-RL accuracy (Table 6.5). **The pre-registered ±0.5 pp accuracy-preservation condition was not met by any pruned method.** On VAL-SELECT, PPO's losses were 4.00 pp (SimpleCNN) and 3.80 pp (ResNet-8).

Pre-registered verdicts:

(a) **Primary success, Case A** (accuracy preserved and energy consistently reduced): **not met** for either architecture.

(b) **SimpleCNN:** Case B (energy reduced, with a measurable accuracy trade-off) **and Case D** (a baseline offers the better trade-off).

(c) **ResNet-8:** Case B.

PPO against Uniform on SimpleCNN. At almost identical MACs (42.6% vs 42.5% removed), MAC-matched Uniform-Structured was **3.60 pp more accurate** than PPO-Structured on VAL-RL. It also **used less energy in 5 of 5 sessions** (7.91 vs 8.54 J per 1,000 inferences; median per-session difference 8.0%). **PPO does not dominate Uniform in Phase 7A: on SimpleCNN, Uniform was better on both axes.** PPO concentrated its 80.1% parameter removal in conv3 and the hidden layer, which bought less time than Uniform's broad channel removal at the same MAC count (latency 0.571 vs 0.534 ms), the Section-6.5 mechanism.

Other comparisons (between-method tests are two-sided exact Wilcoxon tests on five session pairs, minimum attainable p = 0.0625, so they are descriptive only):

(a) **PPO vs LAMP on SimpleCNN:** neither dominates. PPO used less energy in 5 of 5 sessions (12.5% median difference) but was 3.32 pp less accurate.

(b) **ResNet-8:** PPO, Uniform and LAMP showed **no clear difference** in the energy–accuracy trade-off. Against Uniform, accuracy differed by 0.44 pp and PPO used less energy in only 3 of 5 sessions; against LAMP, accuracy differed by 0.08 pp and PPO used less energy in 4 of 5.

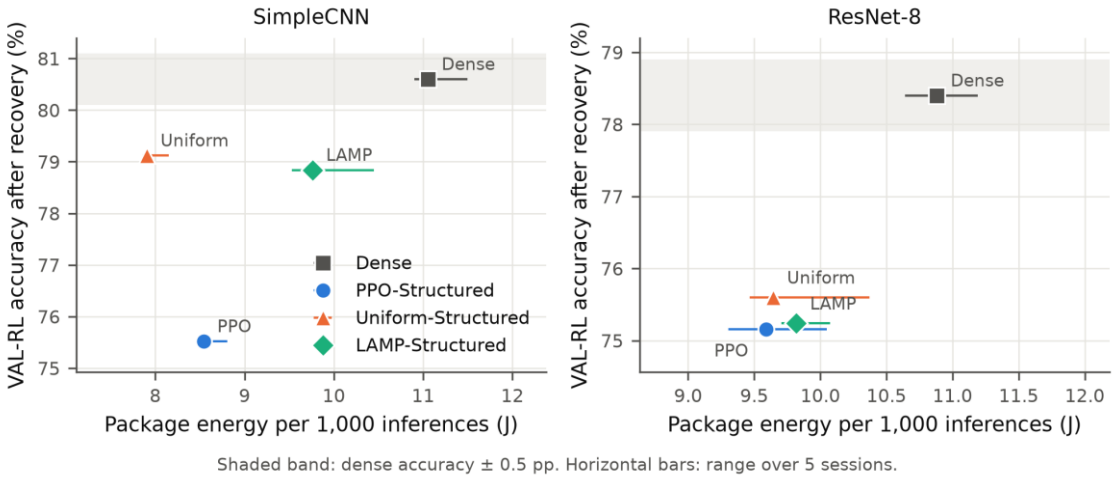


Fig. 6.3: Phase 7A: VAL-RL accuracy after recovery against measured package energy per 1,000 inferences. Points: medians. Horizontal bars: range over the five sessions. Shaded band: dense accuracy ± 0.5 pp; no pruned model lies inside it.

Interpretation. Structured realisation made the compression real at the hardware: parameters, MACs, latency and directly measured package energy all fell, consistently across sessions. The learned masked allocation, however, did not transfer well: its

structured form lost 5.1 and 3.2 pp of VAL-RL accuracy, and on SimpleCNN a plain uniform allocation was both more accurate and more energy-efficient.

Claim boundary.

(a) Phase 7A does not show accuracy-preserving energy reduction.

(b) It does not show that PPO is more energy-efficient than Uniform.

(c) It says nothing about the masked Phase-5/6 models themselves.

(d) The energy is CPU package energy on one laptop, not whole-system or lifecycle energy.

6.9 Motivation for Phase 7B

Phase 7A exposed a design problem, not a measurement problem: a policy learned for *masked* pruning, realised as channel removal and recovered, was neither accurate nor efficient enough, and for ResNet-8 the masked units do not match the structured units.

Phase 7B was therefore designed **after** Phase 7A, as a separate variant whose agent chooses structured actions directly. It was not part of the original framework.

6.10 Phase 7B — Deployment-Oriented Structured PPO

Design (Sections 3.16.3 and 4.9.7; summarised here). The deployment-oriented structured PPO searches directly in the structured space. Its search reward used a *surrogate* (predicted) energy saving, so the name implies no measured energy result.

(a) **Scope, actions and state:** SimpleCNN on CIFAR-10 only; structured ratios per unit, each a physical width configuration; an eight-feature state including centred structured sensitivity, plus the centred sensitivity prior ($\beta = 1.0$).

(b) **Accuracy proxy:** feasibility judged by a 2-epoch recovery proxy on VAL-RL, accepted before the search (Spearman 0.948 against 5-epoch recovery; threshold $\hat{\tau} = 0.98849$; recall 1.00 and precision 0.655 on the validation sample).

(c) **Energy surrogate:** $\hat{E} = -0.117 + 13.762 \cdot T_{table}$ (J per 1,000 inferences, T in ms), fitted to directly measured energy of 200 calibration policies and accepted by cross-validation (Spearman 0.996, median absolute percentage error 1.05%). The first

candidate in the fixed order, MACs alone, had failed (Spearman 0.946 against the required 0.95; lowest-energy-quartile error 8.55% against 5%), so measured latency, not MACs, was the accepted predictor.

(d) **Reward and selection:** a constraint-first reward on the proxy, with $\alpha = 4$ fixed on development seeds; a feasible archive with lowest-predicted-energy selection; 20 confirmatory seeds (700–719).

(e) **Recovery and accuracy criterion:** the equal 5-epoch recovery for every final model; accuracy judged on VAL-SELECT, which is held out from the search and selection.

Status of the surrogate. The surrogate supported the search only. **It is not measured energy, and it is not evidence for the energy of any final Phase-7B model.**

Selection outcome. Every one of the 20 seeds selected a policy. The primary policy, [2, 2, 2, 5], was selected in **5 of 20** seeds; its widths **[26, 51, 102, 102]** remove 20% of the three convolutions' channels and 60% of the hidden neurons. The six other policies, each selected by 1–4 seeds, were also recovered and evaluated. The primary policy's proxy retention, 0.9890, lies just above the threshold of 0.98849: the selections sit at the proxy feasibility boundary.

6.11 Phase 7B — Memory and Computation Results

Table 6.6 compares the recovered Phase-7B model with the dense model, matched baselines and old Phase-7A model.

Table 6.6: Phase 7B: accuracy, memory and computation after the equal 5-epoch recovery (SimpleCNN, CIFAR-10). Memory is parameter memory, Σ numel × element size (float32). Latency and energy were **not measured** (Section 6.13).

Model	Widths	VAL-SELECT (Δ pp)	VAL-RL (Δ pp)	Parameter memory	Memory ↓	MACs (M)	MACs ↓
Dense	[32, 64, 128, 256]	80.00 (0)	80.60 (0)	2,481,448 B	—	10.85	—
New PPO [2, 2, 2, 5]	[26, 51, 102, 102]	**78.88 (−1.12)**	80.00 (−0.60)	908,916 B	**63.4%**	6.94	**36.0%**
Uniform, r = 0.208	[25, 51, 101, 203]	78.76 (−1.24)	79.76 (−0.84)	1,555,908 B	37.3%	6.93	36.2%
LAMP, s = 56.2%	[32, 58, 103, 94]	79.64 (−0.36)	79.12 (−1.48)	909,932 B	63.3%	8.76	19.3%
Old PPO (7A) [0, 2, 5, 5]	[32, 51, 51, 102]	76.00 (−4.00)	75.52 (−5.08)	493,836 B	80.1%	6.23	42.6%

Level 1, memory. The new PPO model needs **63.4% less parameter memory** than the dense model (908,916 vs 2,481,448 bytes). Its serialised checkpoint is 63.3% smaller.

Level 2, computation. It needs **36.0% fewer MACs** (6.94 M vs 10.85 M per image).

Spread across seeds. The seven distinct per-seed selections all removed between 34.1% and 37.5% of the MACs, but their parameter-memory reduction ranged from 29.5% to 63.4%. The search rewarded predicted energy, which tracks latency, and therefore MACs; the selections differ mainly in how many hidden-layer neurons they remove, which changes memory a great deal and MACs very little (Section 6.5).

Matched baselines (descriptive). MAC-matched Uniform removes about the same MACs but much less memory; parameter-matched LAMP removes about the same memory but far fewer MACs (Table 6.6).

Latency. Latency was not measured for Phase 7B, so Level 2 is established for MACs only. The Paper-3 table latency of the policy's shape was a surrogate input, not a Phase-7B measurement.

6.12 Phase 7B — Accuracy Trade-off

Result on the pre-registered split (VAL-SELECT). The new PPO model reached 78.88% against the recovered dense model's 80.00%, a loss of **1.12 pp**. Every other per-seed selection lost between 1.08 and 1.28 pp.

Comparison with the Phase-7A transfer. Measured on the same split and protocol, the old Phase-7A PPO model lost **4.00 pp**, so the redesign reduced the loss by 2.88 pp. Phase 7A's widely quoted 5.08 pp loss is on VAL-RL, Phase 7A's criterion split; comparing it directly with the 1.12 pp VAL-SELECT figure would mix splits. On VAL-RL the new model's loss was 0.60 pp, against 5.08 pp for the old model.

Joint deployment-efficiency success criterion. The pre-registration names it GREEN-SUCCESS (Section 4.9.7).

(a) **Result: met by 0 of 20 seeds.** Every selected policy lost more than the allowed 0.5 pp on VAL-SELECT, which alone decides the outcome.

(b) The latency and energy conditions could not be assessed either, because no measurement session took place (Section 6.13).

Comparative boundaries (descriptive; no energy verdict is possible for any model):

(a) **Against Uniform:** the new PPO model is 0.12 pp more accurate on VAL-SELECT, at similar MACs (36.0% vs 36.2% removed) and with much lower memory (63.4% vs 37.3% removed).

(b) **Against LAMP:** the new PPO model is 0.76 pp less accurate, removes more MACs (36.0% vs 19.3%), and removes about the same parameter memory (63.4% vs 63.3%).

(c) **LAMP-Structured** is the only pruned model within 0.5 pp of dense on VAL-SELECT (−0.36 pp). Its joint criterion is not assessable, because energy was never measured.

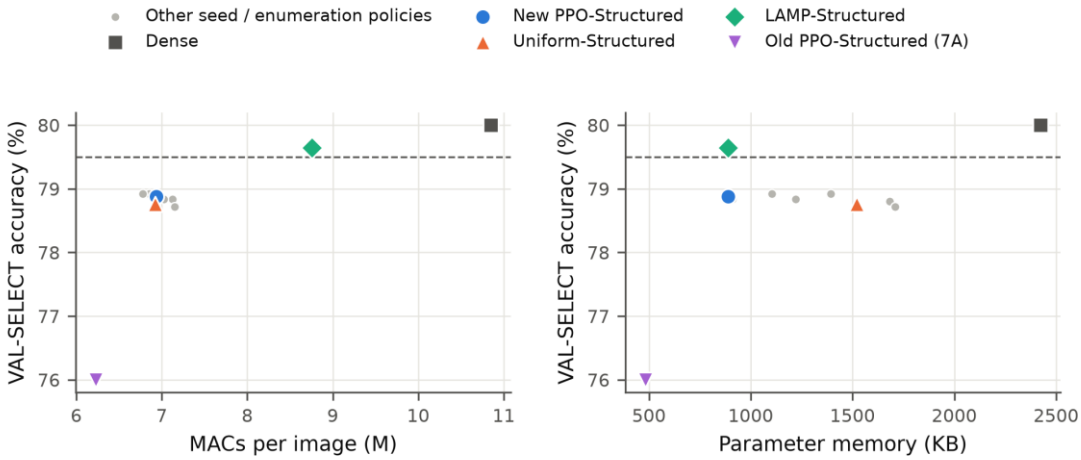


Fig. 6.4: Phase 7B: VAL-SELECT accuracy after the equal 5-epoch recovery against MACs per image (left) and parameter memory (right). Dashed line: dense accuracy − 0.5 pp, the accuracy condition of the joint criterion. Grey points: the other per-seed selections and the enumeration reference.

Interpretation. Learning directly in the structured space improved the memory–computation–accuracy trade-off substantially over transferring a masked policy, but did not reach the 0.5 pp criterion. That the selections sit at the proxy feasibility boundary, with proxy precision 0.655 in validation, is consistent with the shortfall, although the data do not isolate the proxy as its cause.

Claim boundary.

(a) Phase 7B shows no measured latency or energy reduction.

(b) It is not accuracy-preserving.

(c) It does not beat Uniform or LAMP.

(d) The joint deployment-efficiency criterion was not achieved.

6.13 Direct Energy Confirmation and Measurement Failure

Two independent attempts were made to measure Phase-7B energy directly. Both stopped at the pre-registered stability check, before any comparison between models.

Attempt 1, the Phase-7B confirmatory pilot. N may be doubled once, then STOP.

Table 6.7: Phase-7B confirmatory energy pilot (dense model only; limit CV ≤ 3%).

Step	N per block	CV of six dense blocks	Decision
First pilot	100,000	**11.36%**	unstable; N doubled
Doubled-N attempt	200,000	—	voided by recorded user decision after the CPU-load gate stopped it during foreground activity
Permitted rerun, laptop idle	200,000	**4.01%**	unstable → **hardware validation STOPPED**

No confirmatory energy session was run.

Attempt 2, the separate energy confirmation study.

(a) **Protocol.** Written and committed before any data, for the frozen Phase-7B models; all five checkpoint hashes were verified. The workload added a fixed 180 s thermal soak and used 200,000 inferences per block.

(b) **Qualification.** Dense model only, 3 fresh runs × 2 blocks = **6 blocks**. Energy per 1,000 inferences ranged from 9.47 to 15.22 J (mean 12.14 J). The coefficient of variation was **19.92%**, against the **3%** criterion. All runs passed the validity gates, with in-block system load 7.6–10.2%, below the 15% limit.

(c) **Outcome.** The setup was **not qualified**. There was one permitted attempt, and the limit was not relaxed. **No Dense-versus-PPO paired session ran.**

Result. Direct energy confirmation for Phase 7B remains unresolved. No estimate from latency or from the surrogate is substituted.

What the qualification recorded (descriptive). The workload was steady: median latency stayed within 1.042–1.052 ms across the six blocks. Package power was not: it ranged from 9.1 to 14.5 W (core PP0 6.7 to 9.7 W, the rest of the package 2.4 to 5.1 W), and the thermal-zone reading ranged from 72 to 92 °C, with the coolest run giving the lowest energy. The instability therefore lay in the package's power draw under identical work, not in the timing of the inference. The measurements do not identify its source, and only the dense model was measured.

The contrast with Phase 7A. The same software counter on the same laptop had passed its pilot in Phase 7A, with CV 2.30% and 1.74%. The thesis records this contrast without claiming a cause. A software counter on a laptop is not a guaranteed-stable instrument, which is why each study had to qualify it (Jay et al., 2023; Raffin and Trystram, 2025).

6.14 Three-Level Sustainability Assessment

Table 6.8 brings the three levels of Section 6.2 together for Phases 7A and 7B, with the accuracy cost of each.

Table 6.8: Three-level deployment-efficiency assessment.

Level	Metric	Phase 7A evidence (structured transfer)	Phase 7B evidence (deployment-oriented structured PPO)	Conclusion
1. Compression / memory	parameter count / parameter memory; serialised size	parameters −80.1% (SimpleCNN) and −25.0% (ResNet-8)	parameter memory −63.4%; serialised −63.3%	**Established** in both phases
2. Computation / latency	MACs; measured batch-1 latency	MACs −42.6% / −21.5%; **measured** speed-up 1.28× / 1.09×	MACs −36.0%; latency **not measured**	MACs established in both; latency measured for 7A only. Paper 3: MACs are a strong latency proxy, parameters a weak one
3. Operational energy	RAPL package J per 1,000 inferences; EDP	**Measured:** −22.8% / −13.4%, lower in 5/5 sessions; EDP 0.603 / 0.791	**Unresolved:** pilot CV 11.36% → 4.01%; confirmation CV 19.92% (limit 3%)	Real energy reduction demonstrated in 7A only
Accuracy (criterion ±0.5 pp)	recovered validation accuracy	**Failed:** VAL-RL −5.08 / −3.24 pp; no pruned method within 0.5 pp; Uniform dominated PPO on SimpleCNN	**Failed:** VAL-SELECT −1.12 pp; 0/20 seeds met the joint criterion	The accuracy trade-off was too large in 7A and reduced, but not removed, in 7B

Three readings follow. **Level 1 does not imply Level 2:** the Phase-7A SimpleCNN PPO model removed 80.1% of the parameters but only 42.6% of the MACs, and MAC-matched Uniform, with far fewer parameters removed, ran faster. **Level 2 tracked Level 3 in Phase 7A,** because package power was nearly constant across models. **Level 3 must be measured:** in Phase 7B the instrument could not be qualified, and neither the MAC reduction nor the energy surrogate substitutes for a measurement. A deployment saving bought with a material accuracy loss is not presented as a sustainability result.

6.15 Answer to RQ6

Compression can translate into real deployment savings, but not automatically, and not at every level at once.

(i) **Masked sparsity alone gives no hardware evidence.** Physical structured pruning is needed for dense-kernel CPU cost to fall.

(ii) **Even structurally, parameter reduction is a weak guide to latency.** Across 5,184 measured structured policies, Spearman ρ was 0.31–0.55 for parameter reduction and 0.89–0.98 for MAC reduction. In 31–39% of policy pairs the sparser policy was slower.

(iii) **Accuracy-feasible headroom is small.** For ResNet-8, the best proxy-feasible candidate reached a 1.098× speed-up, while the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither satisfied the pre-registered 1.10× GO criterion, so the hardware-aware search was not started.

(iv) **Phase 7A: real energy savings, with material accuracy loss.** Directly measured CPU package energy fell by 22.8% (SimpleCNN) and 13.4% (ResNet-8), with VAL-RL losses of 5.08 and 3.24 pp; on SimpleCNN, MAC-matched uniform allocation was both more accurate and more energy-efficient.

(v) **Phase 7B: a much better trade-off, but neither criterion met.** Parameter memory fell by 63.4% and MACs by 36.0% at a 1.12 pp VAL-SELECT loss (4.00 pp for the transferred policy). No seed met the 0.5 pp accuracy criterion, and direct energy confirmation failed its stability qualification.

Therefore deployment benefit must be established separately for memory, computation, latency and energy, each with its accuracy cost. An improvement at one level is a hypothesis about the next, not evidence for it.

6.16 Chapter Summary

Paper 3 showed that parameter reduction is a weak latency proxy (Spearman ρ 0.31–0.55) and MAC reduction a strong one (Spearman ρ 0.89–0.98), and that the accuracy-feasible latency headroom of ResNet-8 was too small for a hardware-aware search. Phase 7A showed that structured pruning reduces directly measured CPU package energy, but with VAL-RL losses beyond the pre-registered limit and with Uniform better than PPO on SimpleCNN. Phase 7B, designed in response, improved the memory–computation–accuracy trade-off, but no seed met the joint deployment-efficiency criterion, and its energy remains unresolved because both measurement attempts failed the stability check. Chapter 7 discusses these results together with those of Chapter 5.

CHAPTER 7
DISCUSSION

7.1 General

Chapters 5 and 6 reported what happened in the core pruning experiments (Phases 3–6) and when the pruned models were realised and measured on hardware (Paper 3, Phases 7A and 7B, and the energy confirmation study). This chapter interprets that evidence across phases, places it against the literature reviewed in Chapter 2, and states where the proposed framework succeeds and where it does not.

No new numbers are introduced; only the few audited numbers that an argument needs are repeated. Causal language is reserved for effects the experimental design isolated, such as the C1-versus-C2 contrast of Phase 5, or for relations that hold by construction; where only an association was observed, the text says so. The discussion follows the order in which the thesis encountered its problems: the agent and sensitivity (Sections 7.2–7.3); retention, reward and recovery (Sections 7.4–7.7); deployment and energy (Sections 7.8–7.11); and generalisability, limitations and the overall contribution (Sections 7.12–7.15).

7.2 What Adaptive Layer-Wise PPO Actually Contributes

The evidence (Section 5.3). The PPO agent learned visibly non-uniform allocations, pruning large, low-sensitivity layers heavily and keeping small early layers almost intact (Figure 5.2). At the sparsity it chose, these allocations were far more accurate than uniform allocation (5 of 6 settings, up to about +31.4 pp) and random allocation (6 of 6); against global magnitude and ERK the picture was mixed; and LAMP was significantly more accurate than PPO before recovery in 5 of 6 settings in Phase 3, and in 6 of 6 for the later pipelines.

What this establishes.

(a) **The agent solves the allocation problem it was given.** It discovers, without any hand-written allocation rule, that layers should be pruned differently. In this sense the thesis confirms the premise behind RL-based compression (He et al., 2018): layer-wise allocation is learnable from a scalar reward.

(b) **It does not establish superiority.** The thesis does not show that a learned allocation is better than the best deterministic allocation rule. The comparison with LAMP, a cheap and deterministic method, points the other way at the sparsities studied.

Where the agent's value lies. One interpretation, supported by the later phases, is that PPO's value lies in its *flexibility*: Phases 4, 5 and 7B changed the prior, the reward and the action space, while the agent and its training procedure stayed the same, whereas a fixed rule such as LAMP optimises one implicit criterion. This evidence is indirect. The thesis shows that PPO *can* be pointed at a new objective and will pursue it (in Phase 5 this paid off for ResNet-8), not that this flexibility yields a better final model than a tailored deterministic method for any objective tested here. RL pruning should therefore be presented as an adaptive optimisation mechanism for objectives that are hard to express as a single saliency score, not as a replacement for strong deterministic pruning under ordinary magnitude-based compression. Nothing in the thesis implies that PPO is necessary for pruning in general.

7.3 The Role of Sensitivity Information

The pattern (Sections 5.4, 5.5 and 5.8). Across three phases, sensitivity guidance improved the *search*, but not the *final model*. In Phase 3 the prior sampled better-ranked policies earlier, with fewer destructive ones; the primary accuracy-AUC endpoint improved in 3 of 6 settings, and no final-accuracy effect was found in any setting. In Phase 4 the centred prior beat both its shuffled control and a magnitude-matched conservative control on reward-rank AUC, and beat the sign-reversed prior in every setting, but did not improve the final rank. In Phase 5, under the corrected reward, it again improved exploration in 5 of 6 settings. Yet C1's test accuracy was 0.05–0.16 pp *below* C2's in every setting (descriptive).

The strongest defensible reading. Sensitivity is useful as a **search bias**: its measurable contribution lies in exploration guidance, not in predicting which final model will be most accurate. A natural explanation, which the thesis can describe but not prove, is that the prior shapes *which* policies are sampled early, whereas the returned policy is decided by the reward landscape and the selection rule. In these small spaces, where every setting's best-seen policy already had a median rank of 1–4.5, the search finds near-optimal

regions with or without the prior. A better route to the same destination need not change the destination.

Why centring mattered. In Phase 3 the shuffled and constant priors did as well as the correct one on LeNet-5, and all three Phase-3 priors share the same total pressure towards lighter pruning, so part of any benefit could be generic conservatism. Centring removes that net conservative pressure by construction. Its advantage over permuted and magnitude-matched controls on reward-rank AUC therefore provides stronger evidence for a layer-relative exploration effect than Phase 3 did, although not complete causal isolation of layer-specific information. On the accuracy-based exploration measure, however, the *conservative* control was significantly better than the centred prior in 4 of 6 settings, consistent with aggressive actions on insensitive units raising the reward rank of sampled policies while lowering their average accuracy. Centring made the prior more layer-specific; it did not make it a better predictor of final accuracy.

Implication for the literature. Sensitivity is often treated as a direct proxy for how much a layer can be pruned. The dissociation found here suggests evaluating sensitivity-based methods separately on whether they improve the *search* and whether they improve the *outcome*; the thesis found the first, but not the second. This does not mean sensitivity "does not work"; its contribution is of a different kind than often assumed.

7.4 Search Quality versus Policy Retention

The finding (Section 5.6). One of the clearest results of the thesis is that **finding a good policy and ending on it are different problems.** In Phase 4, PPO's best visited policy reached the top 10 of 1,296 in 95–100% of runs, whereas the terminal policy did so in 0% of runs in five settings and 5% in the sixth; the same pattern was present in Phase 3. Archive selection restored retention on validation ranking (80–100% top-10 reach).

A possible mechanism, which was not isolated, is that a stochastic policy-gradient method updating on noisy, high-variance estimates may sample an excellent policy early and then settle on a region of high average reward that does not contain it.

What the finding establishes is methodological. A study that reports only the terminal policy of an RL pruning agent measures retention together with search. Had the thesis reported only terminal policies, it would have concluded that PPO searches poorly; the

exhaustive landscapes showed instead that it searches well and retains poorly. This mirrors the gap between the search phase and the final selection in architecture search (Yu et al., 2020), and the value of archives of visited states in exploration (Ecoffet et al., 2021). RL pruning studies should therefore report the best visited policy, the retained policy and the selection procedure that links them, separately.

Elite behavioural cloning. An imitation term towards the best policies seen so far, in the spirit of self-imitation learning (Oh et al., 2018), improved the final-selected rank in only 3 of 6 settings, against 6 of 6 for the archive rule alone. Pulling the actor towards elite policies during training was thus not equivalent to keeping the elite candidates outside the agent; with one strength ($\eta = 0.1$) and one elite size ($K = 5$) tested, this does not show that behavioural cloning is ineffective in general.

The boundary. Archive selection improves *validation-rank* retention. Its effect on test accuracy was mixed: small gains in four settings, and losses of 5.66 and 4.00 pp on ResNet-8, where the archive retained exactly what the reward ranked highest, an over-pruned model. Archive selection is therefore a retention mechanism, not a generalisation mechanism. Retention amplifies whatever the reward specifies, which leads directly to the next section.

7.5 Reward Design and Over-Pruning

A stronger optimiser exposes the reward (Section 5.7). Once search and retention improved in Phase 4, the agent reliably reached the optimum of the reward R0. That optimum is set by R0's fixed exchange rate between accuracy and sparsity, and on ResNet-8 it is an accuracy-losing policy: with the improved pipeline, ResNet-8 test accuracy fell by 7.86 pp (CIFAR-10) and 4.77 pp (CIFAR-100) below dense. The optimiser did not fail; it optimised what the reward asked for, whereas a weaker search had hidden the misalignment. This is a pruning instance of the observation that more capable optimisers exploit misspecified objectives more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022). Search performance must therefore be judged together with objective quality, and a pruning reward must encode the acceptable accuracy–compression boundary explicitly, because a trade-off reward has no region in which accuracy loss becomes unacceptable.

The accuracy-constrained reward as a correction (Section 5.8). Phase 5 made accuracy a constraint rather than a traded quantity, in the spirit of constrained reinforcement learning (Achiam et al., 2017). Comparing the constrained reward with R0 under the same prior and archive (C1 vs C0) isolates the reward as the manipulated factor: the ResNet-8 accuracy loss fell from 7.32 to 1.18 pp (CIFAR-10) and from 4.05 to 1.16 pp (CIFAR-100), at markedly more moderate sparsity. On SimpleCNN it was slightly *worse* than R0, LAMP remained more accurate in all six settings, and the constraint held on test in only 67.5% of runs. A validation-based accuracy constraint is therefore an effective **optimisation constraint**: it controls over-pruning during search. It is not a **generalisation guarantee**.

7.6 Validation-to-Test Transfer

The gap (Section 5.9). 119 of 120 constrained policies passed the constraint on the selection split, but only 81 of 120 remained feasible on the test set, and in CIFAR-100 ResNet-8 none of 20 did. Three explanations are plausible, although none was isolated by the experiments:

(i) **Validation sampling variability.** The sampling noise of a 2,500-image validation split is of the same order as the 2% relative margin.

(ii) **Selection at the boundary.** The constrained reward drives the agent to the *most compressed* feasible policy, which sits close to the threshold by design.

(iii) **Architecture and dataset dependence.** The failures concentrated in the LeNet-5 settings and in CIFAR-100 ResNet-8, where the feasible region is small and the dense accuracy is low.

Constraint-based pruning therefore needs margins, uncertainty-aware constraints or larger selection splits before a validation-certified constraint can be read as a promise about unseen data. Equal recovery removed most of the gap for this pipeline (Phase 6), but that is a property of recovery, not of the constraint.

7.7 Why LAMP Remains a Strong Benchmark, and Why Equal Fine-Tuning Changes the Rankings

7.7.1 LAMP

LAMP (Lee et al., 2021), deterministic and search-free, was the strongest method before recovery throughout the core framework. Its strength calibrates the thesis rather than invalidating it. One untested interpretation is that LAMP's rescaled magnitude score, which sets layer-wise sparsity automatically (Lee et al., 2021), already captures much of what a learned allocation can achieve under an ordinary compression objective.

For simple compression objectives, a strong deterministic layer-adaptive baseline can therefore be difficult to beat, and RL must justify its additional cost: repeated pruned-model evaluations (512 per run in this thesis), many seeds, reward design, prior calibration, archive management and selection splits, against one pass over the weights for LAMP. An adaptive search becomes the more natural tool when the objective includes accuracy constraints, deployment metrics or several competing goals; the thesis does not show that this advantage materialises, and Phase 7B does not prove it (Section 7.10).

7.7.2 Equal fine-tuning

What changed (Section 5.10). Phase 6 changed the comparison more than any other experiment. Before recovery, LAMP led PPO by 0.26–0.53 pp and PPO led global magnitude by up to 6.51 pp. After an identical five-epoch, fixed-mask recovery, the PPO–LAMP difference fell to 0.00–0.22 pp, within the pre-registered ±0.5 pp confidence-interval margin in all six settings, and PPO's advantage over global magnitude fell to at most 0.12 pp. This is a decision rule, not a formal equivalence test, and LAMP remained slightly but significantly ahead in three settings. The methods are therefore not equivalent; their differences became small.

One interpretation. Much of the apparent advantage of one pruning rule over another reflects how much accuracy the *initial* masked model preserves: recovery appears to compensate most where the initial loss was largest (global magnitude recovered the most and LAMP the least). Because the methods differ only in their masks, with shared seeds and data streams, the narrowing within each setting is an effect of equal recovery; the mechanism inside fine-tuning is not isolated.

This is the thesis's most important methodological implication for pruning comparisons. Methods should be compared after equal post-pruning recovery budgets, not only immediately after pruning; otherwise the comparison partly measures how much each method damages the network before the damage is repaired. This is consistent with work showing that retraining choices can change pruning conclusions (Blalock et al., 2020; Renda, Frankle and Carbin, 2020; Le and Hua, 2021). The claim holds for small CNNs at the moderate sparsities PPO selected under the accuracy constraint (mean 15.6–56.9% in Phase 6), with a five-epoch budget. It does not extend to very high sparsity, ImageNet-scale models, transformers or large models, where recovery may compensate far less.

7.8 Compression versus Real Deployment Efficiency

Masked versus structured pruning. Masked pruning suits search and analysis, because every policy is a masked copy of the same network and recovery can preserve the mask exactly. But masked zeros stay inside dense tensors: without sparse kernels, a masked model is not smaller or faster on a CPU (Hoefler et al., 2021), and the thesis makes no latency or energy claim for its masked models. Only structured pruning, which changes tensor shapes, lets parameters, MACs, latency and energy fall physically. It is coarser, however, and in Phase 7A the structured realisation of the masked allocation cost several points of accuracy.

Sparsity is not speed (Sections 6.4–6.5). Paper 3's exhaustive tables showed that parameter reduction correlated only weakly with measured CPU latency gain and MAC reduction strongly, and that in roughly a third of policy pairs or more the more parameter-sparse policy was slower. In SimpleCNN the reason is arithmetic rather than inference: the hidden fully connected layer holds most of the weights but very little of the computation, and the early convolutions are the reverse. This is consistent with the observation that cost indicators can disagree (Dehghani et al., 2022); the thesis quantifies the disagreement exhaustively for small structured CNNs on one CPU.

Accuracy-feasible headroom (Section 6.6). Raw latency headroom existed, but under an accuracy constraint the useful headroom shrank sharply. For ResNet-8, the best proxy-feasible candidate reached 1.098× and the best confirmed five-epoch-feasible calibration policy reached 1.089×; neither satisfied the pre-registered 1.10× GO criterion, and the

1.098× candidate was never confirmed feasible. For SimpleCNN none could be established, because its accuracy proxy failed validation. **Unconstrained speed-ups say little about the speed-ups available to a model that must keep its accuracy.**

7.9 Accuracy–Efficiency Trade-offs and the Three Levels of Efficiency

The deployment results are best read through the three levels of Chapter 3: compression and memory (Level 1), computation and latency (Level 2), and measured operational energy (Level 3). **Each level needs its own evidence:**

(i) **Memory does not guarantee latency.** In Phase 7A, the SimpleCNN PPO model removed 80.1% of the parameters but was slower than a uniform model that removed far fewer, and Paper 3 shows the same weak link across 5,184 shapes.

(ii) **MACs predict latency better, but do not replace measurement.** The MAC–latency correlation is strong but below 1, and at nearly equal MACs Uniform was still faster than PPO in Phase 7A.

(iii) **Latency does not mathematically guarantee energy.** Energy is power times time, so a faster model saves energy only if its average power does not rise to compensate. In Phase 7A average package power was nearly the same for every model, so energy savings followed latency savings; that is an observation about this platform, not a law.

Accuracy must be reported at every level. The pre-registered criteria of Phases 7A and 7B made this explicit by requiring accuracy within 0.5 pp of dense. **No pruned model met that requirement in either phase, except LAMP-Structured in Phase 7B, whose energy could not be measured.** Every deployment saving in the thesis therefore comes with an accuracy cost, and is reported with it.

7.10 Interpreting the Sustainability Experiments

Phase 7A: real energy savings, but not a better model (Sections 6.7–6.8). Phase 7A firmly establishes that physically structured pruning reduced **directly measured** CPU package energy per inference, in all five sessions. But energy savings alone are not evidence of a better model: the SimpleCNN PPO model lost 5.08 pp of VAL-RL accuracy, and a MAC-matched uniform model was both more accurate and lower in

energy. It shows that structural compression reaches the hardware, not successful sustainability-oriented optimisation by PPO, and that a transferred masked allocation is a poor structured allocation, plausibly because the masked policy concentrated pruning in the hidden layer, where it costs little accuracy *as a mask* but is neither accurate nor computationally efficient as channel removal (consistent with Paper 3, not isolated).

Phase 7B: what improved (Sections 6.10–6.12). Phase 7B used a **deployment-oriented structured PPO**: its actions were physical channel ratios, and its search was guided by a validated energy surrogate rather than by measured energy. It substantially improved the memory–computation–accuracy trade-off: parameter memory fell by 63.4% and MACs by 36.0%, and the selection-split accuracy loss was 1.12 pp, against 4.00 pp for the old policy on the same split, supporting the design lesson that **an allocation should be learned in the space where it will be deployed.**

Phase 7B: what it did not solve. It lost more than the 0.5 pp the pre-registration allowed, and 0 of 20 seeds met the joint deployment-efficiency criterion. Its latency was not measured, and its energy is unresolved. Against the MAC-matched uniform and parameter-matched LAMP structured baselines it showed no clear advantage overall (Section 6.12). Phase 7B improved part of the deployment objective; it did not solve the full objective.

The energy surrogate. The surrogate ranked calibration policies very well (Spearman 0.996), which made the search affordable. But surrogate validity does not turn predicted energy into measured energy: its error for a particular final model is unknown, and the final model's energy was never measured. Surrogates can reduce search cost; final deployment claims require direct validation on the final model.

The measurement failure (Section 6.13). Phase 7B's energy remained unresolved because the measurement setup repeatedly failed its pre-registered stability check (3% limit), while latency stayed stable within about one percent; the thesis does not claim to know the cause. In the present setup, package-energy readings were therefore substantially less stable than latency measurements; the thesis does not generalise this beyond the tested system. Because the same counter on the same machine had passed its check in Phase 7A, qualification must be repeated for every measurement campaign. This is a limitation of the experimental environment. **It is evidence neither for nor against**

the Phase-7B model's energy use: no model comparison was made, and only the dense model was measured. Studies of software energy meters likewise report that readings need careful protocol design and validation (Khan et al., 2018; Jay et al., 2023; Raffin and Trystram, 2025).

On sustainability wording. The thesis does not establish that any of its models is environmentally beneficial, lower in carbon footprint or suitable for "green" deployment. It establishes measured operational CPU energy savings for the Phase-7A structured models, with their accuracy costs, and, for Phase 7B, memory and computation reductions with energy unresolved. Lifecycle, training-energy and carbon accounting were outside its scope.

7.11 What the Negative Results Establish

The negative and partial results, which carry much of the thesis's methodological weight, are summarised in Table 7.1.

Table 7.1: Negative and partial results and what each establishes.

Negative or partial result	What it establishes
PPO did not beat LAMP before recovery (5 of 6, then 6 of 6 settings)	RL must justify its cost through flexibility or richer objectives
Sensitivity did not improve final accuracy in any phase	Search improvement and outcome improvement must be evaluated separately
Archive selection had mixed test effects	Retention is not generalisation; it amplifies whatever the reward specifies
Validation feasibility transferred to test in 67.5% of runs	An optimisation constraint is not a generalisation guarantee
Hardware-aware RL was NO-GO	Accuracy-feasible latency room can be too small to justify a latency-driven search
Uniform beat PPO on SimpleCNN in Phase 7A	A transferred masked allocation is a poor structured allocation; deployment metrics can reverse rankings
Phase 7B missed the 0.5 pp criterion (0 of 20 seeds)	Learning in the deployment space narrows, but does not close, the accuracy gap at these compression levels
Phase-7B energy remained unresolved	Energy claims require a qualified instrument; MAC reductions and surrogates do not substitute for measurement

Each of these results came from an experiment whose decision criteria were fixed in advance, and each was kept as observed. Several changed the design of the next phase: retention led to the archive, over-pruning to the constraint, and the lossy transfer to Phase

7B. The positive findings are credible partly *because* the negative ones are visible; that is a logical point about evidential weight, not an empirical causal claim.

7.12 Generalisability

The evidence base. The core framework used CIFAR-10 and CIFAR-100, three small CNNs, four pruning units per network and an action space of 1,296 policies; the deployment extension used a subset of these settings on one CPU platform (an Intel Core i7-1260P laptop, batch 1, one thread).

Within that scope, the main qualitative results recur across both datasets and all three architectures; the deployment findings are specific to one platform. **Outside that scope** the thesis provides no evidence, whether for ImageNet-scale data, larger CNNs, transformers or large language models, other hardware, or other batch sizes or thread counts. At larger scale the policy space can no longer be enumerated, so retention and search quality cannot be measured against ground truth and the case for a learned search may become stronger; recovery may compensate less at high sparsity; and latency and energy relations are hardware-specific. Larger-scale and multi-platform validation is future work (Chapter 8).

7.13 Limitations

The limitations below bound every claim in Chapters 5–7. They are grouped by kind.

A. Model and dataset scale. CIFAR-scale data and small CNNs only. Four pruning units and six ratios per network: the space was small enough to enumerate, which made exhaustive ground truth possible but makes the RL problem easier than at realistic scale. The sustainability phases used CIFAR-10 only; Phase 7B, SimpleCNN only.

B. Optimisation. PPO's search cost is substantial. Sensitivity guidance improved exploration but not final accuracy, and one Phase-4 exploration measure reversed. LAMP remained stronger before recovery and slightly ahead in three settings after it. The results depend on reward design: no Phase-4 redesigned reward met its criterion, and the constrained reward made SimpleCNN slightly worse. Elite behavioural cloning was tested at one configuration only.

C. Validation methodology. Validation-to-test transfer of the accuracy constraint was imperfect (67.5% test feasibility, 0% in CIFAR-100 ResNet-8), and the fixed relative threshold ($\tau = 0.98$) and the small selection splits make feasibility sensitive to sampling noise. The deployment phases used a single recovery seed per model. One dense reference (CIFAR-10 SimpleCNN) used an older training recipe. Phase 6 was not a fully independent test confirmation, since the test sets had been read for earlier frozen models. The ±0.5 pp rule is a confidence-interval decision rule, not a formal equivalence test. Recovery used one fixed five-epoch protocol; longer or tuned recovery might change the rankings.

D. Hardware. One CPU platform, one batch size and one thread configuration; no GPU, mobile, embedded or accelerator evaluation. The SimpleCNN accuracy proxy failed validation, so its accuracy-feasible latency headroom is unknown, and the exhaustive 5-epoch recovered landscape was stopped for compute cost.

E. Energy. CPU package energy only, with no DRAM, whole-system or external power measurement. The Phase-7B measurement setup failed its stability qualification repeatedly, so no Phase-7B energy or latency claim is made. The energy surrogate and the accuracy proxy are model-selection tools, not measurements.

F. Sustainability. No carbon accounting, no lifecycle or embodied-energy assessment, and no analysis of training or search energy. Phase-7A deployment-scale energy figures are projections, not results.

7.14 Overall Scientific Contribution

The central contribution. The novelty of this thesis does not lie in a claim that PPO beats existing pruning methods; it does not. It lies in **an integrated, systematically audited diagnostic framework, with pre-specified and pre-registered decision criteria across its later experimental stages**. It follows one pruning approach from allocation to measured hardware energy, isolating a separate question at each stage (Table 7.2).

Table 7.2: Stages of the diagnostic framework, the question each isolates, and the thesis evidence.

Stage	Question	Thesis evidence	Research question
1. Adaptive layer-wise allocation	Can a learned agent decide *where* to prune?	Yes: far better than uniform or random; not better than LAMP	RQ1
2. Sensitivity-guided exploration	Does layer sensitivity help the search?	Search yes, outcome no	RQ2
3. Policy retention	Does the agent keep what it finds?	No, unless an archive retains it (validation ranking)	RQ3
4. Reward alignment	Does the reward encode the desired trade-off?	A trade-off reward over-prunes; a constraint controls it, without a generalisation guarantee	RQ4
5. Equal recovery	Do method differences survive identical fine-tuning?	Largely not, at these sparsities	RQ5
6. Deployment translation	Does compression become latency?	Through MACs and structure, not through parameter count	RQ6
7. Direct energy validation	Does it become measured energy, at what accuracy cost?	Yes in Phase 7A, with material accuracy loss; unresolved in Phase 7B	RQ6

The six research questions therefore form a chain rather than a list of independent results: each answer changed what the next question had to control for, from learnability (RQ1) through guidance, retention, reward and recovery to re-measurement at each hardware level (RQ6). Effective pruning is **not a single optimisation problem** but a chain of coupled problems, and a weakness at any link can hide or reverse the effect of improvements elsewhere: a good search under a misaligned reward produced worse models; a retained policy did not generalise; an allocation that was good as a mask was poor as a structure; and a strong MAC reduction remained unconfirmed as an energy saving. **This multi-stage view, supported at each stage by experiments whose decision criteria were fixed in advance, and reported including their negative results, is the thesis's central scientific contribution.** Its practical implications for pruning research are drawn together in Section 8.4.

7.15 Chapter Summary

PPO contributes a flexible, learnable layer-wise allocation that clearly beats uniform and random allocation but not the strong deterministic LAMP baseline; its value lies in adaptability to changing objectives rather than universal superiority. Sensitivity acts as a search bias that improves exploration but not the final model. An external archive restores

validation-rank retention without improving generalisation. A stronger optimiser exposed a misaligned trade-off reward; an accuracy constraint controlled over-pruning but did not guarantee test feasibility. Equal recovery narrowed method differences to small values. In deployment, sparsity is not speed, accuracy-feasible headroom is much smaller than unconstrained headroom, Phase 7A's measured energy savings carried a material accuracy cost, and the deployment-oriented structured PPO of Phase 7B improved the memory–computation–accuracy trade-off but missed the accuracy criterion, with its energy unresolved. Chapter 8 concludes the thesis and sets out the future work these findings motivate.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 General

This chapter answers the six research questions, states the contributions and the thesis-level conclusion, summarises the limitations and sets out future work. It introduces no new evidence: every number below was reported and audited in Chapters 5 and 6.

The thesis set out to learn layer-wise pruning allocations with Proximal Policy Optimisation (PPO), guided by layer sensitivity, and to find out what such a framework needs in order to produce a better and cheaper model. The research proceeded as a sequence of experiments, each designed in response to what the previous one revealed. The core framework, which used masked pruning and answered RQ1–RQ5, comprised: adaptive layer-wise PPO pruning on three small CNNs and two datasets, against five one-shot baselines at identical sparsity; sensitivity-guided exploration, first with an absolute and then with a centred prior; a policy-retention analysis against exhaustive ground truth; archive selection on held-out validation data; a reward-misalignment diagnosis; an accuracy-constrained reward; and equal fine-tuning of every compared model. The deployment extension, added afterwards and answering RQ6, comprised an exhaustive CPU latency analysis of structured policies, a direct energy evaluation of the structurally realised allocation, a deployment-oriented structured PPO that chose physical channel allocations using an accuracy proxy and a latency-based energy surrogate, and a pre-registered attempt to confirm the final model's energy directly.

8.2 Conclusions by Research Question

RQ1. Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP?

Yes against simple strategies; no against LAMP. PPO learned meaningful non-uniform allocations, pruning large, low-sensitivity layers heavily and protecting small early layers. At the sparsity it chose, it was much more accurate than uniform pruning in five of six settings, by up to approximately +31.4 percentage points of test accuracy (CIFAR-10 SimpleCNN), and more accurate than random pruning in all six. The comparison with

global magnitude and ERK was mixed. Before equal recovery, LAMP was significantly more accurate than PPO in five of six settings in the first comparison and in all six for the later pipelines. The learned allocator is effective, but not universally better than LAMP.

RQ2. Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?

It improves search measures, not the final model. Across Phases 3–5, sensitivity guidance improved one or more search or exploration measures: better-ranked policies were sampled earlier, and destructive policies less often. These improvements did not translate into reliable gains in final predictive accuracy. The centred form provided stronger evidence for a layer-relative exploration effect, although one accuracy-based exploration measure favoured a conservative control, and under the corrected reward the prior-guided pipeline was marginally less accurate on test than the identical pipeline without the prior. Its role is narrower than often assumed, not absent.

RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention?

PPO does not retain them; an archive restores retention on validation ranking. The terminal policy reached the top 10 of 1,296 policies in 0% of runs in five settings and 5% in the sixth, although the best visited policy was there in 95–100% of runs. Archive-based selection raised top-10 reach to 80–100%, mainly through the archive rule itself rather than through imitation during training. Its effect on test accuracy was mixed, with large losses on ResNet-8, where the archive retained exactly the over-pruned policies the reward preferred. Archive selection solves a retention problem, not a generalisation problem.

RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off?

Reward design materially controls how aggressively PPO prunes. The original reward trades accuracy and sparsity at a fixed rate, and its optimum over-prunes fragile networks; this became visible only once search and retention improved (on CIFAR-10 ResNet-8, test accuracy fell 7.86 percentage points below dense). Under an otherwise identical pipeline, the accuracy-constrained reward cut the mean loss on CIFAR-10 ResNet-8 from

7.32 to 1.18 percentage points, at more moderate sparsity. But it was slightly worse on SimpleCNN, LAMP remained more accurate, and its validation-certified feasibility held on the test set in only 67.5% of runs. A validation constraint controls the search; it does not guarantee behaviour on unseen data.

RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods?

Equal five-epoch, fixed-mask recovery substantially narrowed the differences between methods. The PPO–LAMP difference fell from 0.26–0.53 to 0.00–0.22 percentage points, within the pre-registered ±0.5 percentage-point confidence-interval decision margin in all six settings. This margin is a decision rule, not a formal statistical equivalence test, and LAMP remained significantly, though slightly, ahead in three settings. PPO's advantage over global magnitude, large before recovery, fell to at most 0.12 percentage points. Methods should therefore be compared under equal recovery budgets.

RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy?

Not automatically.

(a) **Latency proxies.** Across every structured policy measured, parameter reduction was a weak proxy for CPU latency (Spearman 0.31–0.55), and MAC reduction a much stronger one (0.89–0.98). Measured latency remained necessary.

(b) **ResNet-8 headroom.** Under the accuracy constraint, the best proxy-feasible candidate reached a 1.098× speed-up and the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither met the pre-registered 1.10× GO rule, so no hardware-aware search was started.

(c) **Phase 7A, measured energy with an accuracy cost.** The structured realisation of the learned allocation reduced directly measured CPU package energy by 22.8% (SimpleCNN) and 13.4% (ResNet-8), consistently across sessions, but lost 5.08 and 3.24 percentage points of VAL-RL accuracy; on SimpleCNN, MAC-matched uniform pruning was both more accurate and lower in energy.

(d) **Phase 7B, a better trade-off without an energy result.** The deployment-oriented structured PPO reduced parameter memory by 63.4% and MACs by 36.0%, at a validation-accuracy cost of 1.12 percentage points. No seed (0 of 20) met the joint deployment-efficiency criterion, its latency was not measured, and its energy remained unresolved because the measurement setup failed its stability qualification.

Compression can produce real deployment savings, but each dimension must be measured separately, with its accuracy cost.

8.3 Main Contributions

The six thesis contributions (TC1–TC6) are stated in Table 8.1 in final form, each with its boundary.

Table 8.1: Thesis contributions and their boundaries.

#	Contribution	Boundary
TC1	**Adaptive PPO-based layer-wise pruning**, evaluated against Uniform, Random, global magnitude, ERK and LAMP at identical sparsity on two datasets and three architectures	Strongly better than uniform and random allocation; not superior to LAMP
TC2	**An analysis of sensitivity-guided exploration**, with absolute and centred priors against shuffled, constant, magnitude-matched and sign-reversed controls	Improves the search; no consistent final-accuracy gain
TC3	**A diagnosis of policy retention in RL pruning**, against exhaustive ground truth, and evaluation of archive-based selection	Validation-retention benefit; mixed effect on test accuracy
TC4	**A diagnosis of reward misalignment**, and evaluation of an accuracy-constrained, validation-aware reward	Better control of over-pruning; test feasibility not guaranteed
TC5	**An equal-recovery methodology:** identical fixed-mask fine-tuning, shared seeds and data, at identical sparsity	Small post-recovery differences do not constitute formal equivalence
TC6	**A deployment-efficiency analysis** that separates parameter compression, MAC reduction, measured latency and directly measured operational energy, including a qualified energy-measurement protocol and its reported failure	Phase-7A energy savings carry a material accuracy cost; Phase-7B energy remains unresolved

Together, they examine adaptive pruning at each stage from allocation to hardware validation, with decision criteria fixed in advance across the later experimental stages.

8.4 Overall Thesis Conclusion

Adaptive neural-network pruning is not only a problem of selecting which weights or layers to remove. It is a chain of connected decisions: how pruning is allocated across layers, how the search is guided, whether good policies are retained, whether the reward matches the true objective, how post-pruning recovery is controlled, and whether compression translates into hardware benefit.

Improvement at one stage does not guarantee success at the next. Better search exposed a poor reward. Better validation selection transferred to the test set only partly. Lower parameter count did not mean lower latency. **Lower energy came with accuracy loss exceeding the pre-registered limit**, and a simpler allocation did better. And the structured model with the best trade-off reduced memory and computation substantially, but its energy could not be measured reliably.

Implications for practice. Compare methods only after an equal recovery budget; report the best visited policy separately from the returned one; write the acceptable accuracy loss into the objective; confirm any constraint on held-out data; and treat memory, computation, latency and energy as separate claims, each measured and reported with its accuracy cost.

This multi-stage view is the central conclusion of the thesis. A pruning result becomes a deployment-efficiency result only when every link of the chain has its own evidence and accuracy is reported alongside each efficiency gain. The negative results are part of that conclusion, not exceptions to it: each identified the next link to control.

8.5 Limitations

The limitations (Section 7.13) are summarised by group.

(a) **A. Scale.** All experiments used CIFAR-10 and CIFAR-100, with no ImageNet-scale validation. Each pruning space was small enough to enumerate, which enabled exact ground truth but makes the search easier than at realistic scale.

(b) **B. Models.** Three small CNNs (SimpleCNN, a CIFAR adaptation of LeNet-5 and ResNet-8); no larger CNN, transformer or language model. The deployment extension used a subset of these settings, and its final agent SimpleCNN only.

(c) **C. Optimisation.** PPO's search is far more expensive than a deterministic method; its results depend on reward design; LAMP remained stronger before recovery; and sensitivity guidance did not improve final accuracy.

(d) **D. Evaluation.** Validation-certified constraints transferred imperfectly to the test set; feasibility was sensitive to the fixed threshold and the size of the validation splits; the later recovery and deployment experiments used a single recovery seed per model; and recovery used one fixed five-epoch protocol.

(e) **E. Hardware.** All latency and energy results come from one laptop CPU, at batch 1 and one thread, with no GPU, mobile, embedded or accelerator evaluation.

(f) **F. Energy.** Only CPU package energy was measured, with no memory or whole-system energy. The Phase-7B measurements failed their stability qualification, so there is no direct energy result for the final structured model.

(g) **G. Sustainability.** No carbon accounting, lifecycle analysis or assessment of training and search energy was carried out. The thesis is a sustainability-oriented evaluation of operational inference energy, not an environmental assessment.

8.6 Future Work

Each direction below follows from a specific finding or limitation of the thesis.

Short-term

(i) **Native structured RL pruning from the beginning.** Translating a masked policy into a structured network after the search was costly in accuracy (Phase 7A), and learning in the structured space narrowed that cost (Phase 7B). Future frameworks should search in the deployed structure from the start, including residual-coupled channel groups.

(ii) **Hybrid LAMP and RL.** Deterministic layer-adaptive scores could initialise, constrain or bias the RL search, combining LAMP's cheap reliability with the agent's ability to pursue objectives that one score cannot express.

(iii) **Better feasibility proxies.** The validation-to-test gap and the modest precision of the Phase-7B accuracy proxy call for calibrated predictors of post-recovery accuracy and for selection margins rather than exact thresholds.

(iv) **Stable external energy instrumentation.** The laptop's software energy counter failed its stability qualification. Future energy studies should use a logging wall-power meter, controlled thermal conditions and fixed processor clocks where possible, and qualify the instrument before any model comparison.

Medium-term

(v) **Multi-objective optimisation.** Explicit Pareto-front search over accuracy, memory, MACs, latency and measured energy, compared with deterministic baselines on the same fronts, would show trade-offs that a scalar reward hides.

(vi) **Improved generalisation constraints.** Multiple validation subsets, uncertainty-aware thresholds and more robust constraint formulations would address the finding that validation feasibility did not guarantee test feasibility.

(vii) **Larger architectures.** ResNet-18/34/50, MobileNet- and EfficientNet-class networks, or other CNNs whose pruning spaces cannot be enumerated, where a learned search may have more to offer.

(viii) **Larger datasets.** ImageNet-scale or comparable data, to test whether the retention, reward and recovery findings hold when each evaluation is much more costly.

(ix) **Edge and mobile deployment.** Mobile CPUs, ARM processors, embedded GPUs and accelerators, each with its own latency and energy behaviour.

(x) **GPU evaluation.** Whether the relationship between parameter reduction, MAC reduction and latency found on one CPU holds, weakens or reverses on GPUs.

Long-term

(xi) **Hardware-in-the-loop multi-objective RL.** Rewards using directly measured latency and energy rather than surrogates, attempted only after stable energy instrumentation (item (iv)) is available and with explicit accuracy constraints.

(xii) **Reliable lifecycle and carbon analysis.** Carbon-intensity conversion and lifecycle accounting become meaningful only once trustworthy direct energy measurements exist. Until then, no carbon impact should be claimed; this thesis makes no such claim.

8.7 Final Remarks

The thesis does not establish PPO as a universally superior pruning method. It shows that adaptive pruning must be evaluated as a complete pipeline, from search through recovery to deployment: RL pruning is useful as a flexible allocator when its objective is explicitly aligned with the desired accuracy boundary, simpler deterministic methods remain stronger for ordinary compression objectives, and hardware-level validation must precede efficiency claims. The value of the work lies as much in the links it found to be weak as in those it showed to be sound.

REFERENCES

Abdelfattah, M. S., Mehrotra, A., Dudziak, Ł., and Lane, N. D. (2021). Zero-cost proxies for lightweight NAS, Proc. International Conference on Learning Representations (ICLR).

Achiam, J., Held, D., Tamar, A., and Abbeel, P. (2017). Constrained policy optimization, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 70, pp. 22–31.

Alwani, M., Wang, Y., and Madhavan, V. (2022). DECORE: Deep compression with reinforcement learning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12339–12349. DOI: 10.1109/cvpr52688.2022.01203

Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R. et al. (2021). What matters for on-policy deep actor-critic methods? A large-scale study, Proc. International Conference on Learning Representations (ICLR).

Blalock, D., Gonzalez Ortiz, J. J., Frankle, J., and Guttag, J. (2020). What is the state of neural network pruning? Proc. Machine Learning and Systems (MLSys), Vol. 2, pp. 129–146.

Cai, H., Gan, C., Wang, T., Zhang, Z., and Han, S. (2020). Once-for-all: Train one network and specialize it for efficient deployment, Proc. International Conference on Learning Representations (ICLR).

Camci, E., Gupta, M., Wu, M., and Lin, J. (2022). QLP: Deep Q-learning for pruning deep neural networks, IEEE Transactions on Circuits and Systems for Video Technology, Vol. 32, No. 10, pp. 6488–6501. DOI: 10.1109/tcsvt.2022.3167951

Chen, T., Liang, L., Ding, T., Zhu, Z., and Zharkov, I. (2023). OTOv2: Automatic, generic, user-friendly, Proc. International Conference on Learning Representations (ICLR).

Cheng, H., Zhang, M., and Shi, J. Q. (2024). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 12, pp. 10558–10578. DOI: 10.1109/tpami.2024.3447085

Chin, T.-W., Ding, R., Zhang, C., and Marculescu, D. (2020). Towards efficient model compression via learned global ranking, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1515–1525. DOI: 10.1109/cvpr42600.2020.00159

Dehghani, M., Tay, Y., Arnab, A., Beyer, L., and Vaswani, A. (2022). The efficiency misnomer, Proc. International Conference on Learning Representations (ICLR).

Desislavov, R., Martínez-Plumed, F., and Hernández-Orallo, J. (2023). Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning, Sustainable Computing: Informatics and Systems, Vol. 38, Article No. 100857. DOI: 10.1016/j.suscom.2023.100857

Diao, E., Wang, G., Zhang, J., Yang, Y., Ding, J., and Tarokh, V. (2023). Pruning deep neural networks from a sparsity perspective, Proc. International Conference on Learning Representations (ICLR).

Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. (2021). First return, then explore, Nature, Vol. 590, No. 7847, pp. 580–586. DOI: 10.1038/s41586-020-03157-9

El Halabi, M., Srinivas, S., and Lacoste-Julien, S. (2022). Data-efficient structured pruning via submodular optimization, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L. et al. (2020). Implementation matters in deep RL: A case study on PPO and TRPO, Proc. International Conference on Learning Representations (ICLR).

Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. (2020). Rigging the lottery: Making all tickets winners, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 119, pp. 2943–2952.

Fang, G., Ma, X., Song, M., Mi, M. B., and Wang, X. (2023). DepGraph: Towards any structural pruning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16091–16101. DOI: 10.1109/cvpr52729.2023.01544

Fang, G., Ma, X., Mi, M. B., and Wang, X. (2024). Isomorphic pruning for vision models, Computer Vision – ECCV 2024, Lecture Notes in Computer Science, Springer, pp. 232–250. DOI: 10.1007/978-3-031-73404-5_14

Frankle, J. and Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks, Proc. International Conference on Learning Representations (ICLR).

Frantar, E. and Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Frantar, E. and Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one-shot, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 202, pp. 10323–10337.

Gadhikar, A. H., Mukherjee, S., and Burkholz, R. (2023). Why random pruning is all we need to start sparse, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 202, pp. 10542–10570.

Ganjdanesh, A., Gao, S., and Huang, H. (2024). Jointly training and pruning CNNs via learnable agent guidance and alignment, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16058–16069. DOI: 10.1109/cvpr52733.2024.01520

Gu, S., Yang, L., Du, Y., Chen, G., Walter, F., Wang, J. et al. (2024). A review of safe reinforcement learning: Methods, theories and applications, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 12, pp. 11216–11235. DOI: 10.1109/tpami.2024.3457538

Han, S., Pool, J., Tran, J., and Dally, W. (2015). Learning both weights and connections for efficient neural network, Advances in Neural Information Processing Systems (NeurIPS), Vol. 28.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition, Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. DOI: 10.1109/cvpr.2016.90

He, Y. and Xiao, L. (2024). Structured pruning for deep convolutional neural networks: A survey, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 5, pp. 2900–2919. DOI: 10.1109/tpami.2023.3334614

He, Y., Zhang, X., and Sun, J. (2017). Channel pruning for accelerating very deep neural networks, Proc. IEEE International Conference on Computer Vision (ICCV), pp. 1398–1406. DOI: 10.1109/iccv.2017.155

He, Y., Lin, J., Liu, Z., Wang, H., Li, L.-J., and Han, S. (2018). AMC: AutoML for model compression and acceleration on mobile devices, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 815–832. DOI: 10.1007/978-3-030-01234-2_48

Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., and Pineau, J. (2020). Towards the systematic reporting of the energy and carbon footprints of machine learning, Journal of Machine Learning Research, Vol. 21, No. 248, pp. 1–43.

Hoefler, T., Alistarh, D., Ben-Nun, T., Dryden, N., and Peste, A. (2021). Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks, Journal of Machine Learning Research, Vol. 22, No. 241, pp. 1–124.

Holm, S. (1979). A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics, Vol. 6, pp. 65–70.

Hou, Z., Qin, M., Sun, F., Ma, X., Yuan, K., Xu, Y. et al. (2022). CHEX: CHannel EXploration for CNN model compression, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12277–12288. DOI: 10.1109/cvpr52688.2022.01197

Humble, R., Shen, M., Latorre, J. A., Darve, E., and Alvarez, J. (2022). Soft masking for cost-constrained channel pruning, Computer Vision – ECCV 2022, Lecture Notes in Computer Science, Springer, pp. 641–657. DOI: 10.1007/978-3-031-20083-0_38

Iofinova, E., Peste, A., Kurtz, M., and Alistarh, D. (2022). How well do sparse ImageNet models transfer? Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12256–12266. DOI: 10.1109/cvpr52688.2022.01195

Jay, M., Ostapenco, V., Lefevre, L., Trystram, D., Orgerie, A.-C., and Fichel, B. (2023). An experimental comparison of software-based power meters: Focus on CPU and GPU, Proc. IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp. 106–118. DOI: 10.1109/ccgrid57682.2023.00020

Khan, K. N., Hirki, M., Niemi, T., Nurminen, J. K., and Ou, Z. (2018). RAPL in action: Experiences in using RAPL for power measurements, ACM Transactions on

Modeling and Performance Evaluation of Computing Systems, Vol. 3, No. 2, pp. 1–26. DOI: 10.1145/3177754

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images, Technical Report, University of Toronto, Toronto, Canada.

Kuzmin, A., Nagel, M., van Baalen, M., Behboodi, A., and Blankevoort, T. (2023). Pruning vs quantization: Which is better? Advances in Neural Information Processing Systems (NeurIPS), Vol. 36.

Kwon, W., Kim, S., Mahoney, M. W., Hassoun, J., Keutzer, K., and Gholami, A. (2022). A fast post-training pruning framework for transformers, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Le, D. H. and Hua, B.-S. (2021). Network pruning that matters: A case study on retraining variants, Proc. International Conference on Learning Representations (ICLR).

LeCun, Y., Denker, J. S., and Solla, S. A. (1989). Optimal brain damage, Advances in Neural Information Processing Systems (NeurIPS), Vol. 2.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition, Proceedings of the IEEE, Vol. 86, No. 11, pp. 2278–2324. DOI: 10.1109/5.726791

Lee, J., Park, S., Mo, S., Ahn, S., and Shin, J. (2021). Layer-adaptive sparsity for the magnitude-based pruning, Proc. International Conference on Learning Representations (ICLR).

Lee, N., Ajanthan, T., and Torr, P. (2019). SNIP: Single-shot network pruning based on connection sensitivity, Proc. International Conference on Learning Representations (ICLR).

Li, B., Wu, B., Su, J., and Wang, G. (2020). EagleEye: Fast sub-net evaluation for efficient neural network pruning, Computer Vision – ECCV 2020, Lecture Notes in Computer Science, Springer, pp. 639–654. DOI: 10.1007/978-3-030-58536-5_38

Li, H., Kadav, A., Durdanovic, I., Samet, H., and Graf, H. P. (2017). Pruning filters for efficient ConvNets, Proc. International Conference on Learning Representations (ICLR).

Li, Y., Adamczewski, K., Li, W., Gu, S., Timofte, R., and Van Gool, L. (2022). Revisiting random channel pruning for neural network compression, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 191–201. DOI: 10.1109/cvpr52688.2022.00029

Liebenwein, L., Baykal, C., Carter, B., Gifford, D., and Rus, D. (2021). Lost in pruning: The effects of pruning neural networks beyond test accuracy, Proc. Machine Learning and Systems (MLSys), Vol. 3, pp. 93–138.

Liu, S., Chen, T., Chen, X., Shen, L., Mocanu, D. C., Wang, Z. et al. (2022). The unreasonable effectiveness of random pruning: Return of the most naive baseline

for sparse training, Proc. International Conference on Learning Representations (ICLR).

Liu, S., Chen, T., Zhang, Z., Chen, X., Huang, T., Jaiswal, A. K. et al. (2023). Sparsity may cry: Let us fail (current) sparse neural networks together! Proc. International Conference on Learning Representations (ICLR).

Liu, Z., Sun, M., Zhou, T., Huang, G., and Darrell, T. (2019). Rethinking the value of network pruning, Proc. International Conference on Learning Representations (ICLR).

Luccioni, A. S., Viguier, S., and Ligozat, A.-L. (2023). Estimating the carbon footprint of BLOOM, a 176B parameter language model, Journal of Machine Learning Research, Vol. 24, No. 253, pp. 1–15.

Luccioni, S., Jernite, Y., and Strubell, E. (2024). Power hungry processing: Watts driving the cost of AI deployment? Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp. 85–99. DOI: 10.1145/3630106.3658542

Ma, N., Zhang, X., Zheng, H.-T., and Sun, J. (2018). ShuffleNet V2: Practical guidelines for efficient CNN architecture design, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 122–138. DOI: 10.1007/978-3-030-01264-9_8

Ma, X., Fang, G., and Wang, X. (2023). LLM-Pruner: On the structural pruning of large language models, Advances in Neural Information Processing Systems (NeurIPS), Vol. 36.

Menghani, G. (2023). Efficient deep learning: A survey on making deep learning models smaller, faster, and better, ACM Computing Surveys, Vol. 55, No. 12, pp. 1–37. DOI: 10.1145/3578938

Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. (2017). Pruning convolutional neural networks for resource efficient inference, Proc. International Conference on Learning Representations (ICLR).

Molchanov, P., Mallya, A., Tyree, S., Frosio, I., and Kautz, J. (2019). Importance estimation for neural network pruning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 11256–11264. DOI: 10.1109/cvpr.2019.01152

Nonnenmacher, M., Pfeil, T., Steinwart, I., and Reeb, D. (2022). SOSP: Efficiently capturing global correlations by second-order structured pruning, Proc. International Conference on Learning Representations (ICLR).

Oh, J., Guo, Y., Singh, S., and Lee, H. (2018). Self-imitation learning, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 80, pp. 3878–3887.

Pan, A., Bhatia, K., and Steinhardt, J. (2022). The effects of reward misspecification: Mapping and mitigating misaligned models, Proc. International Conference on Learning Representations (ICLR).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. et al. (2019). PyTorch: An imperative style, high-performance deep learning library, Advances in Neural Information Processing Systems (NeurIPS), Vol. 32.

Patterson, D., Gonzalez, J., Holzle, U., Le, Q., Liang, C., Munguia, L.-M. et al. (2022). The carbon footprint of machine learning training will plateau, then shrink, Computer, Vol. 55, No. 7, pp. 18–28. DOI: 10.1109/mc.2022.3148714

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations, Journal of Machine Learning Research, Vol. 22, No. 268, pp. 1–8.

Raffin, G. and Trystram, D. (2025). Dissecting the software-based measurement of CPU energy consumption: A comparative analysis, IEEE Transactions on Parallel and Distributed Systems, Vol. 36, No. 1, pp. 96–107. DOI: 10.1109/tpds.2024.3492336

Renda, A., Frankle, J., and Carbin, M. (2020). Comparing rewinding and fine-tuning in neural network pruning, Proc. International Conference on Learning Representations (ICLR).

Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M. et al. (2023). From words to watts: Benchmarking the energy costs of large language model inference, Proc. IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–9. DOI: 10.1109/hpec58863.2023.10363447

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation, Proc. International Conference on Learning Representations (ICLR).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347.

Schwartz, R., Dodge, J., Smith, N. A., and Etzioni, O. (2020). Green AI, Communications of the ACM, Vol. 63, No. 12, pp. 54–63. DOI: 10.1145/3381831

Shen, M., Yin, H., Molchanov, P., Mao, L., Liu, J., and Alvarez, J. M. (2022). Structural pruning via latency-saliency knapsack, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Skalse, J., Howe, N., Krasheninnikov, D., and Krueger, D. (2022). Defining and characterizing reward gaming, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Strubell, E., Ganesh, A., and McCallum, A. (2019). Energy and policy considerations for deep learning in NLP, Proc. 57th Annual Meeting of the Association for Computational Linguistics, pp. 3645–3650. DOI: 10.18653/v1/p19-1355

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. (2024). A simple and effective pruning approach for large language models, Proc. International Conference on Learning Representations (ICLR).

Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A. et al. (2019). MnasNet: Platform-aware neural architecture search for mobile, Proc. IEEE/CVF

Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2815–2823. DOI: 10.1109/cvpr.2019.00293

Tran, C., Fioretto, F., Kim, J.-E., and Naidu, R. (2022). Pruning has a disparate impact on model accuracy, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Tschand, A., Rajan, A. T. R., Idgunji, S., Ghosh, A., Holleman, J., Kiraly, C. et al. (2025). MLPerf Power: Benchmarking the energy efficiency of machine learning systems from μWatts to MWatts for sustainable AI, Proc. IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 1201–1216. DOI: 10.1109/hpca61900.2025.00092

Wang, K., Liu, Z., Lin, Y., Lin, J., and Han, S. (2019). HAQ: Hardware-aware automated quantization with mixed precision, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8604–8612. DOI: 10.1109/cvpr.2019.00881

Wang, X., Na, C., Strubell, E., Friedler, S., and Luccioni, S. (2023). Energy and carbon considerations of fine-tuning BERT, Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 9058–9069. DOI: 10.18653/v1/2023.findings-emnlp.607

Wang, Z. and Li, C. (2022). Channel pruning via lookahead search guided reinforcement learning, Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 3513–3524. DOI: 10.1109/wacv51458.2022.00357

Wen, W., Wu, C., Wang, Y., Chen, Y., and Li, H. (2016). Learning structured sparsity in deep neural networks, Advances in Neural Information Processing Systems (NeurIPS), Vol. 29.

Wilcoxon, F. (1945). Individual comparisons by ranking methods, Biometrics Bulletin, Vol. 1, No. 6, p. 80. DOI: 10.2307/3001968

Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K. et al. (2022). Sustainable AI: Environmental implications, challenges and opportunities, Proc. Machine Learning and Systems (MLSys), Vol. 4, pp. 795–813.

Xia, M., Gao, T., Zeng, Z., and Chen, D. (2024). Sheared LLaMA: Accelerating language model pre-training via structured pruning, Proc. International Conference on Learning Representations (ICLR).

Xu, P., Shao, W., Chen, M., Tang, S., Zhang, K., Gao, P. et al. (2024). BESA: Pruning large language models with blockwise parameter-efficient sparsity allocation, Proc. International Conference on Learning Representations (ICLR).

Yang, H., Yin, H., Shen, M., Molchanov, P., Li, H., and Kautz, J. (2023). Global vision transformer pruning with Hessian-aware saliency, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 18547–18557. DOI: 10.1109/cvpr52729.2023.01779

Yang, T.-J., Chen, Y.-H., and Sze, V. (2017). Designing energy-efficient convolutional neural networks using energy-aware pruning, Proc. IEEE Conference on

Computer Vision and Pattern Recognition (CVPR), pp. 6071–6079. DOI: 10.1109/cvpr.2017.643

Yang, T.-J., Howard, A., Chen, B., Zhang, X., Go, A., Sandler, M. et al. (2018). NetAdapt: Platform-aware neural network adaptation for mobile applications, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 289–304. DOI: 10.1007/978-3-030-01249-6_18

Yin, L., Wu, Y., Zhang, Z., Hsieh, C.-Y., Wang, Y., Jia, Y. et al. (2024). Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 235, pp. 57101–57115.

Youden, W. J. (1950). Index for rating diagnostic tests, Cancer, Vol. 3, No. 1, pp. 32–35. DOI: 10.1002/1097-0142(1950)3:1<32::aid-cncr2820030106>3.0.co;2-3

Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. et al. (2022). The surprising effectiveness of PPO in cooperative multi-agent games, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Yu, K., Sciuto, C., Jaggi, M., Musat, C., and Salzmann, M. (2020). Evaluating the search phase of neural architecture search, Proc. International Conference on Learning Representations (ICLR).

Yu, S., Mazaheri, A., and Jannesari, A. (2022). Topology-aware network pruning using multi-stage graph embedding and reinforcement learning, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 162, pp. 25656–25667.

Yu, S., Yao, Z., Gholami, A., Dong, Z., Kim, S., Mahoney, M. W. et al. (2022). Hessian-aware pruning and optimal neural implant, Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 3665–3676. DOI: 10.1109/wacv51458.2022.00372

Zhang, Y., Yao, Y., Ram, P., Zhao, P., Chen, T., Hong, M. et al. (2022). Advancing model pruning via bi-level optimization, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Zoph, B. and Le, Q. V. (2017). Neural architecture search with reinforcement learning, Proc. International Conference on Learning Representations (ICLR).

APPENDIX A
ARCHITECTURES, HYPERPARAMETERS AND REPRODUCIBILITY

These appendices give supporting detail for Chapters 3–6, generated from the frozen phase records by `tools/build_appendices.py`. Each table names its source record. Complete per-seed, per-session and per-comparison records (all statistical families, raw energy blocks, per-seed policies) are preserved in the repository and are summarised, not reproduced, here. The central results and conclusions are in the main chapters.

A.1 Pruning units

Table A.1: Pruning units, weight shapes, parameters and MACs per image (CIFAR-10 output layer).

Architecture	Unit	Role	Modules	Weight shapes	Weights	Share of prunable	MACs per image
simplecnn	features.0	prunable (PPO decision step)	features.0	32x3x3x3	864	0.14%	884,736
simplecnn	features.3	prunable (PPO decision step)	features.3	64x32x3x3	18,432	2.99%	4,718,592
simplecnn	features.6	prunable (PPO decision step)	features.6	128x64x3x3	73,728	11.94%	4,718,592
simplecnn	classifier.1	prunable (PPO decision step)	classifier.1	256x2048	524,288	84.93%	524,288
simplecnn	classifier.3	excluded (never pruned)	classifier.3	10x256	2,560	—	2,560
lenet5	conv1	prunable (PPO decision step)	features.0	6x3x5x5	450	0.74%	352,800
lenet5	conv2	prunable (PPO decision step)	features.3	16x6x5x5	2,400	3.94%	240,000
lenet5	fc1	prunable (PPO decision step)	classifier.1	120x400	48,000	78.78%	48,000
lenet5	fc2	prunable (PPO decision step)	classifier.3	84x120	10,080	16.54%	10,080
lenet5	classifier.5	excluded (never pruned)	classifier.5	10x84	840	—	840
resnet8	stem	prunable (PPO decision step)	conv1	16x3x3x3	432	0.56%	442,368
resnet8	stage1	prunable (PPO decision step)	layer1.conv1;layer1.conv2	16x16x3x3;16x16x3x3	4,608	6.01%	4,718,592
resnet8	stage2	prunable (PPO decision step)	layer2.conv1;layer2.conv2;layer2.shortcut.0	32x16x3x3;32x32x3x3;32x16x1x1	14,336	18.69%	3,670,016
resnet8	stage3	prunable (PPO decision step)	layer3.conv1;layer3.conv2;layer3.shortcut.0	64x32x3x3;64x64x3x3;64x32x1x1	57,344	74.74%	3,670,016
resnet8	fc	excluded (never pruned)	fc	10x64	640	—	640

Source: `cpu_prunable_units.csv`.

A.2 Dense reference models

Table A.2: Dense references over three training seeds.

Setting	Seeds	Reference	Val. (ref.)	Test (ref.)	Val. mean ± SD	Test mean ± SD	Parameters	Recipe
cifar10 simplecnn	1	cnn_baseline_FIXED.pth	77.32	77.03	77.32	77.03	620,362	historical: Adam 1e-3, 10 epochs, final epoch (not retrained)
cifar10 lenet5	3	lenet5_seed0.pth	64.32	64.96	65.63 ± 1.94	66.21 ± 1.51	62,006	Phase-2 protocol: Adam 0.001 cosine, 30 epochs, best-validation epoch
cifar10 resnet8	3	resnet8_seed0.pth	78.72	78.69	78.62 ± 0.23	78.69 ± 0.51	78,042	Phase-2 protocol: Adam 0.001 cosine, 30 epochs, best-validation epoch
cifar100 lenet5	3	lenet5_seed0.pth	32.58	32.28	32.15 ± 0.42	31.42 ± 0.75	69,656	Phase-2 protocol: Adam 0.001 cosine, 30 epochs, best-validation epoch
cifar100 resnet8	3	resnet8_seed0.pth	43.48	43.99	43.07 ± 0.54	43.58 ± 0.53	83,892	Phase-2 protocol: Adam 0.001 cosine, 30 epochs, best-validation epoch
cifar100 simplecnn	3	simplecnn_seed0.pth	49.08	50.5	48.78 ± 0.95	49.84 ± 0.65	643,492	Phase-2 protocol: Adam 0.001 cosine, 30 epochs, best-validation epoch

Source: `cpu_baselines.csv`.

A.3 Recovery protocol

Table A.3: Equal recovery protocol (Phase 6; reused in Paper 3, Phases 7A and 7B).

Configuration key	Value
`optimizer`	Adam
`lr`	0.0001
`betas`	[0.9, 0.999]
`eps`	1e-08
`weight_decay`	0.0
`schedule`	CosineAnnealingLR(T_{max}=epochs, eta_min=0), stepped per epoch
`batch_size`	128
`augmentation`	random crop 32 (zero pad 4) + horizontal flip
`batchnorm`	train mode during fine-tuning (running statistics updated identically for every method); eval mode for every evaluation; no separate recalibration
`selection`	none: the model after the final epoch

Source: `phase6_config.json (protocol)`. *The PPO hyper-parameters are given in Sections 3.17 and 4.8.*

A.4 Pre-registration commits

Table A.4: Pre-registration and freeze commits.

Experiment	Pre-registration record	Commit that first added it
Phase 2 dense training	`cpu_dense_training_protocol.md`	`366d21ae`
Phase 3	`phase3_preregistration.md`	`2dbcc770`
Phase 4	`phase4_preregistration.md`	`f52e87d9`
Phase 5	`phase5_preregistration.md`	`3cbaec1a`
Phase 6	`phase6_preregistration.md`	`b1b5396f`
Paper 3 H0	`h0_protocol.md`	`c2dcb22a`
Paper 3 H2	`paper3_preregistration.md`	`45936846`
Paper 3 proxy	`paper3_proxy_protocol.md`	`ec7a7f27`
Paper 3 ResNet-8 calibration	`resnet8_threshold_calibration_protocol.md`	`5b127804`
Phase 7A	`preregistration.json`	`b8c27665`
Phase 7B design	`phase7b_design.md`	`f5abe039`
Phase 7B final	`phase7b_final_preregistration.json`	`0e101d55`
7B energy confirmation	`protocol.json`	`df233be0`

Commits verified with `git log --diff-filter=A` (Section 4.19).

A.5 Software environments

Table A.5: Software and hardware environments.

Item	Core framework (Phases 2–6)	Deployment extension (Paper 3, 7A, 7B, energy confirmation)
CPU	Intel(R) Core(TM) i5-6400 CPU @ 2.70GHz	12th Gen Intel(R) Core(TM) i7-1260P
OS	Windows-10-10.0.22621-SP0 (Windows 11 build 22621)	Microsoft Windows 11 Home Single Language 10.0.26200
Python	3.11.9	3.11.9
PyTorch	2.7.1+cpu	2.7.1+cpu
Stable-Baselines3	2.8.0	2.8.0 (pinned environment)
Gymnasium / NumPy	1.2.3 / 2.2.2	pinned environment
CUDA	False	not used

Sources: `phase5_environment.json`; `environment.json`; `requirements_locked.txt`.

APPENDIX B
CORE PPO AND SENSITIVITY SUPPLEMENTARY RESULTS

B.1 Phase 3 results by setting and method

Table B.1: Phase 3 mean test accuracy and sparsity by setting and method (20 runs; baselines at each PPO run's zero count).

Setting	Method	Test mean	Test SD	Sparsity mean (%)	Drop vs dense (pp)
C10 SimpleCNN	PPO P0	76.44	0.49	55.06	0.59
C10 SimpleCNN	PPO P1	76.51	0.41	56.87	0.52
C10 SimpleCNN	PPO P2	76.24	0.61	55.95	0.79
C10 SimpleCNN	PPO P3	76.36	0.80	56.04	0.67
C10 SimpleCNN	uniform (matched to P0)	45.08	10.44	55.06	31.95
C10 SimpleCNN	global (matched to P0)	76.51	0.19	55.06	0.52
C10 SimpleCNN	lamp (matched to P0)	76.99	0.03	55.06	0.04
C10 SimpleCNN	erk (matched to P0)	75.70	0.32	55.06	1.34
C10 SimpleCNN	random (matched to P0)	16.84	5.34	55.06	60.19
C10 LeNet-5	PPO P0	61.67	2.64	51.54	3.29
C10 LeNet-5	PPO P1	62.17	1.04	53.20	2.79
C10 LeNet-5	PPO P2	62.54	1.31	48.59	2.42
C10 LeNet-5	PPO P3	62.83	1.36	47.45	2.13
C10 LeNet-5	uniform (matched to P0)	45.38	9.56	51.54	19.58
C10 LeNet-5	global (matched to P0)	58.94	2.35	51.54	6.03
C10 LeNet-5	lamp (matched to P0)	63.31	0.64	51.54	1.65
C10 LeNet-5	erk (matched to P0)	59.74	2.10	51.54	5.22
C10 LeNet-5	random (matched to P0)	12.99	2.92	51.54	51.97
C10 ResNet-8	PPO P0	76.95	0.78	28.35	1.74
C10 ResNet-8	PPO P1	76.87	0.69	30.31	1.82
C10 ResNet-8	PPO P2	77.00	1.26	26.96	1.69
C10 ResNet-8	PPO P3	76.49	1.79	29.20	2.20
C10 ResNet-8	uniform (matched to P0)	62.79	4.18	28.35	15.90
C10 ResNet-8	global (matched to P0)	69.57	2.82	28.35	9.12
C10 ResNet-8	lamp (matched to P0)	76.80	1.37	28.35	1.89
C10 ResNet-8	erk (matched to P0)	77.01	0.94	28.35	1.68
C10 ResNet-8	random (matched to P0)	12.59	1.67	28.35	66.10
C100 SimpleCNN	PPO P0	49.66	0.59	53.28	0.84
C100 SimpleCNN	PPO P1	49.48	0.53	54.32	1.02
C100 SimpleCNN	PPO P2	49.52	0.54	53.39	0.98
C100 SimpleCNN	PPO P3	49.50	0.53	54.21	1.00
C100 SimpleCNN	uniform (matched to P0)	19.96	1.33	53.28	30.54

Setting	Method	Test mean	Test SD	Sparsity mean (%)	Drop vs dense (pp)
C100 SimpleCNN	global (matched to P0)	49.67	0.23	53.28	0.84
C100 SimpleCNN	lamp (matched to P0)	50.30	0.04	53.28	0.20
C100 SimpleCNN	erk (matched to P0)	45.31	1.26	53.28	5.19
C100 SimpleCNN	random (matched to P0)	2.37	0.30	53.28	48.13
C100 LeNet-5	PPO P0	31.19	0.23	34.59	1.10
C100 LeNet-5	PPO P1	30.81	1.04	36.98	1.47
C100 LeNet-5	PPO P2	31.15	0.58	34.37	1.13
C100 LeNet-5	PPO P3	31.05	0.81	35.03	1.23
C100 LeNet-5	uniform (matched to P0)	27.51	1.85	34.59	4.77
C100 LeNet-5	global (matched to P0)	31.32	0.40	34.59	0.96
C100 LeNet-5	lamp (matched to P0)	31.86	0.20	34.59	0.42
C100 LeNet-5	erk (matched to P0)	30.88	0.97	34.59	1.40
C100 LeNet-5	random (matched to P0)	3.29	0.31	34.59	28.99
C100 ResNet-8	PPO P0	43.01	0.09	9.46	0.98
C100 ResNet-8	PPO P1	43.13	0.38	8.76	0.86
C100 ResNet-8	PPO P2	43.01	0.07	9.23	0.98
C100 ResNet-8	PPO P3	43.09	0.31	8.62	0.90
C100 ResNet-8	uniform (matched to P0)	42.78	0.93	9.46	1.21
C100 ResNet-8	global (matched to P0)	42.74	0.02	9.46	1.25
C100 ResNet-8	lamp (matched to P0)	43.76	0.09	9.46	0.23
C100 ResNet-8	erk (matched to P0)	43.81	0.14	9.46	0.18
C100 ResNet-8	random (matched to P0)	4.25	0.36	9.46	39.74

Source: `phase3_summary.csv`.

B.2 Sensitivity vectors

Table B.2: Normalised loss sensitivity S_l by setting and the Phase-5 centred prior coefficients.

Setting	Config. I VAL-RL (Phases 3–4)	Config. II VAL-RL (Phase 5)	Phase-5 P2 coefficients $c_l = S_l - \bar{S}$
C10 SimpleCNN	1.000, 0.032, 0.009, 0.000	1.000, 0.030, 0.012, 0.000	0.740, −0.230, −0.249, −0.260
C10 LeNet-5	1.000, 0.422, 0.046, 0.000	1.000, 0.440, 0.042, 0.000	0.629, 0.070, −0.329, −0.371
C10 ResNet-8	1.000, 0.138, 0.331, 0.000	1.000, 0.145, 0.320, 0.000	0.634, −0.221, −0.046, −0.366
C100 SimpleCNN	1.000, 0.103, 0.014, 0.000	1.000, 0.119, 0.010, 0.000	0.718, −0.164, −0.272, −0.282
C100 LeNet-5	1.000, 0.587, 0.000, 0.086	1.000, 0.564, 0.000, 0.059	0.594, 0.159, −0.406, −0.347
C100 ResNet-8	0.987, 0.310, 1.000, 0.000	1.000, 0.309, 0.997, 0.000	0.423, −0.268, 0.421, −0.577

Phase 7B structured sensitivity (C10 SimpleCNN, VAL-RL (2,500 images), units conv1, conv2, conv3, fc1): normalised 0.898, 1.000, 0.341, 0.000; centred 0.338, 0.440, −0.219, −0.560.

Sources: `phase3_frozen_inputs.json` *(configuration I);* `phase5_config.json` *(configuration II);* `structured_sensitivity.json` *(Phase 7B).*

APPENDIX C
ARCHIVE, REWARD AND RECOVERY DIAGNOSTICS

C.1 Phase 4 search quality and retention

Table C.1: Phase 4 search quality by variant (VAL-RL (I) ranks among 1,296 policies; medians over 20 runs).

Setting	Variant	Selected rank	Selected V_{SELECT} rank	Best-seen rank	Best-seen top-10	Selected top-10	Selected top-20	Test mean	Sparsity mean
C10 SimpleCNN	S0	92	78	1	1.00	0.00	0.00	76.62	57.74
C10 SimpleCNN	S1	3	1	1	1.00	1.00	1.00	76.77	56.06
C10 SimpleCNN	S2	46	38	1	1.00	0.10	0.10	76.76	56.25
C10 SimpleCNN	S2A	3	1	1	1.00	1.00	1.00	76.76	56.06
C10 LeNet-5	S0	69	66	1	1.00	0.00	0.00	62.32	54.89
C10 LeNet-5	S1	5.5	1.5	1	1.00	1.00	1.00	63.20	52.94
C10 LeNet-5	S2	69	66	1	1.00	0.00	0.05	62.08	56.78
C10 LeNet-5	S2A	6	1	1	1.00	1.00	1.00	63.22	52.79
C10 ResNet-8	S0	96	87	2.5	0.95	0.00	0.00	76.48	32.20
C10 ResNet-8	S1	3.5	2.5	2.5	0.95	0.90	1.00	71.52	45.09
C10 ResNet-8	S2	42	28	1	1.00	0.35	0.40	71.77	43.58
C10 ResNet-8	S2A	1	1	1	1.00	1.00	1.00	70.83	47.78
C100 SimpleCNN	S0	65	70	1	1.00	0.00	0.00	49.62	52.86
C100 SimpleCNN	S1	6	2	1	1.00	1.00	1.00	49.89	54.07
C100 SimpleCNN	S2	62	61	1	1.00	0.00	0.10	49.69	53.66
C100 SimpleCNN	S2A	6	1	1	1.00	1.00	1.00	49.84	54.20
C100 LeNet-5	S0	173	140	1	1.00	0.05	0.05	30.28	41.32
C100 LeNet-5	S1	7	2	1	1.00	0.80	1.00	30.71	46.46
C100 LeNet-5	S2	48.5	34.5	1	1.00	0.00	0.00	29.70	48.76
C100 LeNet-5	S2A	7	2	1	1.00	0.85	1.00	30.72	46.30
C100 ResNet-8	S0	114	137	2	0.95	0.00	0.00	43.22	8.53
C100 ResNet-8	S1	10	2.5	2	0.95	0.75	1.00	39.41	27.26
C100 ResNet-8	S2	45	55.5	2	0.90	0.00	0.05	42.82	13.78
C100 ResNet-8	S2A	10	2	2	0.90	0.80	1.00	39.22	27.71

Setting	Variant	Selected rank	Selected V_{SELECT} rank	Best-seen rank	Best-seen top-10	Selected top-10	Selected top-20	Test mean	Sparsity mean
C10 SimpleCNN	Phase-3 P0 (reference, seeds	100.5	—	1.5	—	0.00	0.00	76.44	—
C10 LeNet-5	Phase-3 P0 (reference, seeds	74.5	—	2	—	0.00	0.00	61.67	—
C10 ResNet-8	Phase-3 P0 (reference, seeds	122	—	4	—	0.00	0.00	76.95	—
C100 SimpleCNN	Phase-3 P0 (reference, seeds	65	—	1	—	0.00	0.00	49.66	—
C100 LeNet-5	Phase-3 P0 (reference, seeds	199	—	4.5	—	0.00	0.00	31.19	—
C100 ResNet-8	Phase-3 P0 (reference, seeds	114	—	2	—	0.00	0.00	43.01	—

Source: `phase4_search_quality.csv`.

C.2 Phase 5 selected policies

Table C.2: Phase 5 selected policies by setting and condition (20 runs each).

Cond.	Setting	Sparsity mean	Sparsity median	Test mean	Drop vs dense	Feasible VAL-RL/VAL-SELECT/test (of 20)	Mean q_{test}	Fallbacks
C0	C10 SimpleCNN	56.10	56.10	76.75	0.28	20/20/20	0.9964	0
C0	C10 LeNet-5	53.89	54.00	63.14	1.83	0/0/0	0.9719	0
C0	C10 ResNet-8	46.23	48.78	71.37	7.32	0/0/0	0.9070	0
C0	C100 SimpleCNN	52.90	52.66	50.28	0.22	20/20/20	0.9956	0
C0	C100 LeNet-5	46.32	46.25	30.67	1.61	0/4/0	0.9501	0
C0	C100 ResNet-8	25.88	23.53	39.94	4.05	0/0/0	0.9079	0
C1	C10 SimpleCNN	56.93	57.59	76.45	0.58	20/20/20	0.9924	0
C1	C10 LeNet-5	38.57	38.42	63.83	1.13	20/20/9	0.9827	0
C1	C10 ResNet-8	24.50	24.74	77.51	1.18	20/20/20	0.9850	0
C1	C100 SimpleCNN	53.09	53.67	50.07	0.43	20/20/20	0.9915	0
C1	C100 LeNet-5	30.60	31.01	31.79	0.49	20/19/12	0.9848	1
C1	C100 ResNet-8	15.62	16.08	42.83	1.16	20/20/0	0.9736	0
C2	C10 SimpleCNN	56.21	56.26	76.50	0.53	20/20/20	0.9931	0
C2	C10 LeNet-5	36.92	36.17	63.99	0.97	20/20/15	0.9851	0
C2	C10 ResNet-8	23.92	24.09	77.57	1.12	20/20/20	0.9857	0
C2	C100 SimpleCNN	51.86	53.24	50.13	0.37	20/20/20	0.9926	0
C2	C100 LeNet-5	30.13	30.77	31.86	0.42	20/19/14	0.9870	1
C2	C100 ResNet-8	15.45	16.08	42.90	1.09	20/20/1	0.9752	0

Source: `phase5_constraint_results.csv`.

C.3 Phase 6 recovery by setting and method

Table C.3: Phase 6 test accuracy before and after equal recovery (means of 20 runs).

Setting	Method	Sparsity	Test before	Test after	Recovery	Gap to dense after	Retention ≥ 0.98 before / after	Zero-count change
C100 LeNet-5	dense	0.00	32.28	32.39	0.11	−0.114	1.00 / 1.00	0
C100 LeNet-5	global	30.60	31.43	32.24	0.81	0.042	0.20 / 1.00	0
C100 LeNet-5	lamp	30.60	32.05	32.49	0.44	−0.213	1.00 / 1.00	0
C100 LeNet-5	ppo	30.60	31.79	32.36	0.57	−0.077	0.60 / 1.00	0
C100 ResNet-8	dense	0.00	43.99	44.08	0.09	−0.086	1.00 / 1.00	0
C100 ResNet-8	global	15.62	39.35	44.04	4.69	−0.050	0.00 / 1.00	0
C100 ResNet-8	lamp	15.62	43.34	44.06	0.72	−0.071	1.00 / 1.00	0
C100 ResNet-8	ppo	15.62	42.83	44.02	1.20	−0.032	0.00 / 1.00	0
C100 SimpleCNN	dense	0.00	50.50	50.60	0.10	−0.102	1.00 / 1.00	0
C100 SimpleCNN	global	53.09	49.75	50.51	0.76	−0.005	1.00 / 1.00	0
C100 SimpleCNN	lamp	53.09	50.36	50.58	0.22	−0.081	1.00 / 1.00	0
C100 SimpleCNN	ppo	53.09	50.07	50.53	0.46	−0.031	1.00 / 1.00	0
C10 LeNet-5	dense	0.00	64.96	65.10	0.14	−0.143	1.00 / 1.00	0
C10 LeNet-5	global	38.57	62.41	64.83	2.42	0.133	0.00 / 1.00	0
C10 LeNet-5	lamp	38.57	64.15	65.06	0.91	−0.103	1.00 / 1.00	0
C10 LeNet-5	ppo	38.57	63.83	64.88	1.04	0.082	0.45 / 1.00	0
C10 ResNet-8	dense	0.00	78.69	78.83	0.14	−0.143	1.00 / 1.00	0
C10 ResNet-8	global	24.50	71.00	78.67	7.67	0.025	0.00 / 1.00	0
C10 ResNet-8	lamp	24.50	78.03	78.73	0.70	−0.044	1.00 / 1.00	0
C10 ResNet-8	ppo	24.50	77.51	78.74	1.23	−0.048	1.00 / 1.00	0
C10 SimpleCNN	dense	0.00	77.03	80.08	3.05	−3.050	1.00 / 1.00	0
C10 SimpleCNN	global	56.93	76.54	79.89	3.35	−2.861	1.00 / 1.00	0
C10 SimpleCNN	lamp	56.93	76.94	79.95	3.01	−2.920	1.00 / 1.00	0
C10 SimpleCNN	ppo	56.93	76.45	79.73	3.28	−2.701	1.00 / 1.00	0

Source: `phase6_pre_post_accuracy.csv`.

APPENDIX D
STRUCTURED PRUNING AND HARDWARE PROTOCOL

D.1 Latency-measurement calibration (H0)

Table D.1: H0 calibration: per-configuration decision quantities.

Config.	Signal (ms)	Noise (ms)	SNR	Relative reduction	Positive in all sessions	Usable
A	0.4127	0.0335	12.32	61.6%	True	True
B	0.1222	0.0287	4.26	31.8%	True	False
C	15.4531	1.6070	9.62	63.2%	True	True

Source: `h0_decision.json`. *Rule: SNR ≥ 5, reduction ≥ 10%, positive in every session; order A, C, B.*

D.2 Latency-table session log

Table D.2: Latency-table session log (anchor drift; limit 10%).

Table	Session	Attempt	Load before (%)	Load after (%)	Anchor drift	Accepted
resnet8_c10	1	1	4.1	3.0	27.1%	False
resnet8_c10	1	2	2.6	2.6	9.0%	True
resnet8_c100	1	1	2.7	2.2	9.5%	True
simplecnn_c10	1	1	2.8	2.4	7.5%	True
simplecnn_c100	1	1	2.2	5.1	40.6%	False
simplecnn_c100	1	2	4.7	2.6	7.1%	True
resnet8_c100	2	1	2.1	3.3	8.8%	True
resnet8_c10	2	1	2.3	2.4	7.6%	True
simplecnn_c10	2	1	2.3	2.2	8.8%	True
simplecnn_c100	2	1	3.1	2.0	8.4%	True
resnet8_c100	3	1	2.3	3.4	8.7%	True
simplecnn_c10	3	1	2.7	2.3	7.9%	True
resnet8_c10	3	1	2.5	2.3	9.7%	True
simplecnn_c100	3	1	2.7	2.6	8.4%	True
resnet8_c100	4	1	2.1	3.1	9.6%	True
resnet8_c10	4	1	2.7	3.1	10.6%	False
resnet8_c10	4	2	5.4	2.4	7.4%	True
simplecnn_c100	4	1	2.6	3.4	14.0%	False
simplecnn_c100	4	2	3.1	1.9	8.6%	True
simplecnn_c10	4	1	2.2	2.8	8.9%	True
resnet8_c10	5	1	3.3	2.8	8.4%	True
simplecnn_c10	5	1	2.6	2.6	9.4%	True
simplecnn_c100	5	1	2.7	3.0	9.1%	True
resnet8_c100	5	1	3.2	4.2	6.8%	True

Source: `h2_latency_freeze.json(session_log)`.

D.3 Accuracy-proxy validation and ResNet-8 calibration

Table D.3: Accuracy-proxy validation (1-epoch against 5-epoch recovery).

Architecture	Validation pairs	Spearman $\rho(A_1, A_5)$	MAE of q	Top-10 overlap	q_5-feasible pairs	GO
cifar10_simplecnn	156	0.844	0.0088	3	112	False
cifar10_resnet8	31	0.980	0.0679	10	2	True

Table D.4: ResNet-8 threshold calibration and latency-room decision.

Item	Value
Calibration policies / truly feasible	48 / 14
τ_{proxy}	0.9601
Recall / precision (TP, FP, FN, TN)	1.00 / 0.875 (14, 2, 0, 32)
Proxy-feasible landscape policies	52
Best proxy-feasible speed-up	1.098×
Policies ≥ 1.10×	0
GO to PPO	False

Sources: `paper3_proxy_decision.json`, `resnet8_threshold_calibration_decision.json`. *The best confirmed five-epoch-feasible calibration policy reached 1.089× ([0, 3, 1, 1];* `resnet8_threshold_calibration_report.md`*).*

APPENDIX E
ENERGY EVALUATION AND QUALIFICATION RECORDS

E.1 Phase 7A energy session summaries

Table E.1: Phase 7A energy: session summaries (five sessions per architecture).

Architecture	Method	Median J / 1,000	CV over sessions	Median saving	Min–max saving	Sessions below dense	One-sided Wilcoxon p	Consistent reduction
simplecnn	dense	11.054	2.14%	0.0%	0.0–0.0%	0	—	False
simplecnn	ppo	8.541	1.83%	22.8%	19.5–25.7%	5	0.031	True
simplecnn	uniform	7.906	1.54%	27.9%	27.3–31.2%	5	0.031	True
simplecnn	lamp	9.760	3.59%	12.1%	6.8–15.1%	5	0.031	True
resnet8	dense	10.882	1.84%	0.0%	0.0–0.0%	0	—	False
resnet8	ppo	9.592	2.84%	13.4%	5.6–14.5%	5	0.031	True
resnet8	uniform	9.645	3.63%	11.8%	4.7–13.9%	5	0.031	True
resnet8	lamp	9.819	1.46%	8.9%	7.4–12.3%	5	0.031	True

Source: `energy_summary.csv`.

E.2 Phase 7B recovered final models

Table E.2: Phase 7B: all recovered final models (5-epoch recovery).

Id	Roles	Widths	VAL-SELECT	Δ VAL-SELECT	VAL-RL	Memory ↓	MACs ↓	Joint criterion
0000	dense	[32, 64, 128, 256]	80.00	0.00	80.60	0.0%	0.0%	NO (memory, MACs failed; latency and energy not measured)
2225	new_ppo, seed701, seed702, seed707, seed712, seed718	[26, 51, 102, 102]	78.88	−1.12	80.00	63.4%	36.0%	NO (accuracy failed; latency and energy not measured)
uniform_r0.208	uniform	[25, 51, 101, 203]	78.76	−1.24	79.76	37.3%	36.2%	NO (accuracy failed; latency and energy not measured)
lamp_s56.2	lamp	[32, 58, 103, 94]	79.64	−0.36	79.12	63.3%	19.3%	not assessable (hardware validation stopped)
3125	seed700, enumeration_reference	[22, 58, 102, 102]	78.84	−1.16	79.64	62.4%	34.3%	NO (accuracy failed; latency and energy not measured)
3132	seed703, seed704, seed709	[22, 58, 90, 205]	78.92	−1.08	79.60	42.5%	36.8%	NO (accuracy failed; latency and energy not measured)
2224	seed705, seed714, seed716	[26, 51, 102, 154]	78.84	−1.16	79.84	49.6%	35.3%	NO (accuracy failed; latency and energy not measured)
3130	seed706, seed708	[22, 58, 90, 256]	78.80	−1.20	79.56	30.6%	36.2%	NO (accuracy failed; latency and energy not measured)
2221	seed710, seed711, seed713, seed719	[26, 51, 102, 230]	78.72	−1.28	79.68	29.5%	34.1%	NO (accuracy failed; latency and energy not measured)
3134	seed715, seed717	[22, 58, 90, 154]	78.92	−1.08	79.52	54.4%	37.5%	NO (accuracy failed; latency and energy not measured)
phase7a_ppo	old_ppo	[32, 51, 51, 102]	76.00	−4.00	75.52	80.1%	42.6%	NO (accuracy failed; latency and energy not measured)

Source: `final_summary.csv`. The last column is the pre-registered joint deployment-efficiency criterion (internal file name GREEN-SUCCESS). Latency and energy were not measured.

E.3 Energy-measurement qualification

Table E.3: Dense-only stability pilots and qualification (energy per 1,000 inferences, J).

Measurement	N per block	Six dense blocks	CV	Limit	Decision
7A pilot, simplecnn	100,000	11.518, 11.150, 11.081, 11.253, 10.963, 10.768	2.30%	3%	passed
7A pilot, resnet8	100,000	10.793, 10.489, 10.555, 10.667, 10.358, 10.312	1.74%	3%	passed
7B surrogate-calibration pilot	20,000	12.200, 11.198, 11.107, 11.729, 11.433, 11.420	3.47%	4%	stable
7B confirmatory pilot (pilot_n100000)	100,000	13.088, 11.414, 11.204, 15.125, 12.265, 12.273	11.36%	3%	unstable
7B confirmatory pilot (pilot_n200000)	200,000	11.814, 12.074, 11.039, 11.539, 11.025, 11.033	4.01%	3%	unstable
7B energy-confirmation qualification	200,000	11.417, 14.278, 12.962, 15.221, 9.514, 9.473	19.92%	3%	not qualified

Sources: `pilot_n100000.json` *(Phase 7A);* `pilot_n20000.json` *and* `statistics.json` *(Phase 7B);* `measurement_qualification.json` *(energy confirmation). A doubled-N Phase-7B pilot attempt stopped by the load gate was declared void by a recorded decision of the author and is not listed.*

E.4 Energy-confirmation qualification blocks

Table E.4: Energy-confirmation qualification blocks (descriptive diagnostics).

Run	Position	J / 1,000	Package W	Median latency (ms)	System load (%)	CPU-2 performance (%)	Thermal zone start → end (°C)
qual1	0	11.417	11.16	1.0456	8.4	130	86 → 76
qual1	1	14.278	13.76	1.0513	9.2	133	77 → 83
qual2	0	12.962	12.72	1.0479	8.9	131	86 → 84
qual2	1	15.221	14.51	1.0517	10.2	135	79 → 92
qual3	0	9.514	9.40	1.0420	7.6	129	76 → 81
qual3	1	9.473	9.09	1.0430	7.9	125	72 → 73

Source: `measurement_qualification.json (blocks)`*. No cause of the instability is claimed, and no Dense-versus-PPO session was run.*